

Rancang bangun Aplikasi Manajemen Hotel Kapsul multi-Cabang berbasis Website (Tamu.id)

Oleh :
Aliif Arief Maulana
21/479029/SV/19418

INTISARI

Digitalisasi mendorong transformasi signifikan dalam industri perhotelan, termasuk pada segmen hotel kapsul yang menonjolkan efisiensi ruang dan biaya. Sayangnya, keunggulan fisik tersebut belum selalu diiringi sistem digital yang efisien. Penelitian ini berfokus pada perancangan dan pengembangan *Tamu.id*, sebuah platform manajemen hotel kapsul berbasis web yang komprehensif dan terintegrasi. Sistem ini dibangun dengan arsitektur multi-pengguna (superadmin waralaba, admin hotel, dan tamu) guna mendukung pengelolaan operasional secara terpusat. Fitur utama mencakup sinkronisasi pemesanan dari *Online Travel Agent* (OTA), pengaturan harga dinamis berbasis waktu, dan *Content Management System* (CMS) untuk mengelola informasi properti. Untuk menunjang pertumbuhan bisnis, aplikasi menyediakan fitur promosi berupa *referral* dan voucher diskon yang dapat dikonfigurasi secara fleksibel. Pengembangan sistem dilakukan menggunakan metodologi *Kanban* agar proses iterasi berjalan fleksibel dan adaptif terhadap perubahan kebutuhan selama siklus pengembangan. Dari sisi pengguna, sistem dirancang mengutamakan kemudahan dan keamanan, dimulai dari proses login yang cepat melalui integrasi Google OAuth2 tanpa password. Untuk mencegah terjadinya *race condition* dalam pemesanan, diterapkan mekanisme *optimistic locking*. Proses pembayaran berlangsung secara *real-time* melalui QRIS yang diintegrasikan dengan layanan Midtrans menggunakan *webhook*. Inovasi utama dari sistem ini adalah implementasi *chained QR Code Rotation* untuk regenerasi kunci kamar digital yang dapat bekerja bahkan dalam kondisi perangkat IoT offline, memastikan akses yang aman dan andal. Evaluasi sistem mencakup pengujian fungsional melalui metode *blackbox* yang menunjukkan seluruh fitur berfungsi sesuai perancangan awal. Selain itu, dilakukan uji penerimaan pengguna menggunakan kuesioner dengan skala Likert yang menunjukkan tingkat kelayakan sangat tinggi: 94,6% untuk *role user*, 96% untuk *superadmin*, dan 100% untuk *admin*. Hasil dari penelitian ini adalah sebuah sistem yang tidak hanya meningkatkan efisiensi operasional hotel kapsul, tetapi juga memberikan pengalaman digital yang modern dan *seamless* bagi pengguna, baik dari sisi pengelola maupun tamu.

Kata kunci: Sistem Informasi, Reservasi, Website, Hotel Kapsul, OAuth2, Webhook, E-Ticketing, Kode QR, QRIS, Harga Dinamis, Booking Online, IoT, CMS

Design and Development of a Web-Based Multi-Tenant Capsule Hotel Management Application (Tamu.id)

by:
Aliif Arief Maulana
21/479029/SV/19418

ABSTRACT

Digitalization has driven significant transformation within the hospitality industry, including in the capsule hotel segment which emphasizes spatial and cost efficiency. Unfortunately, these physical advantages are not always accompanied by efficient digital systems. This study focuses on the design and development of *Tamu.id*, a comprehensive and integrated web-based capsule hotel management platform. The system is built with a multi-user architecture (franchise superadmin, hotel admin, and guest) to support centralized operational management. Core features include booking synchronization from *Online Travel Agents* (OTA), time-based dynamic pricing configuration, and a *Content Management System* (CMS) for property information management. To support business growth, the application also provides promotional features such as referrals and discount vouchers, which can be configured flexibly. System development was carried out using a *Kanban* approach to ensure a flexible and adaptive iterative process in response to evolving requirements throughout the development cycle. From the user perspective, the system prioritizes simplicity and security, starting with a fast login process through Google OAuth2 integration without requiring passwords. To prevent race conditions during the booking process, an *optimistic locking* mechanism is implemented. Payments are processed in *real-time* via QRIS, integrated with the Midtrans service using a *webhook* mechanism. A key innovation in this system is the implementation of *chained QR Code Rotation* for regenerating digital room keys, which can function even when IoT devices are offline—ensuring secure and reliable access. System evaluation includes functional testing using the *blackbox* method, which demonstrated that all features operate as initially designed. In addition, user acceptance testing was conducted using a Likert scale questionnaire, which showed a very high level of feasibility: 94.6% for user roles, 96% for superadmin, and 100% for admin. The outcome of this study is a system that not only enhances the operational efficiency of capsule hotels but also delivers a modern and seamless digital experience for both hotel operators and guests.

Keywords: Information System, Website, Reservation, Capsule Hotel, OAuth2, Webhook, E-Ticketing, QR Code, QRIS, Dynamic Pricing, Online Booking, IoT, CMS

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era digitalisasi, industri perhotelan global telah mengalami transformasi fundamental yang didorong oleh adopsi teknologi informasi. Sistem Manajemen Properti (PMS) telah menjadi pusat operasional bagi hotel modern, mengintegrasikan fungsi-fungsi krusial mulai dari reservasi, manajemen *front office*, hingga pelaporan komprehensif. Implementasi PMS yang efektif kini bukan lagi pilihan semata, melainkan sebuah keharusan strategis bagi hotel untuk dapat bersaing, meningkatkan efisiensi operasional, dan memberikan pengalaman yang memuaskan bagi tamu mereka [1]. Seiring dengan evolusi teknologi, sistem ini terus berkembang menjadi platform terpadu yang mampu mengelola berbagai kanal distribusi dan strategi bisnis yang kompleks.

Hotel kapsul merupakan bentuk akomodasi inovatif yang berasal dari Jepang dan kini telah menyebar luas ke berbagai negara, termasuk Indonesia. Konsepnya yang menawarkan ruang tidur minimalis namun fungsional menjadikannya pilihan populer di kalangan wisatawan, khususnya mereka yang mencari penginapan hemat biaya dan efisien dalam penggunaan ruang. Di Indonesia, pertumbuhan hotel kapsul sangat pesat di kota-kota besar seperti Jakarta, Bandung, dan Yogyakarta, dengan contoh populer seperti Bobobox, Digital Airport Hotel, dan Tab Capsule Hotel [2]. Pertumbuhan ini didorong oleh meningkatnya mobilitas urban dan pergeseran perilaku konsumen ke arah digital, yang menuntut akomodasi praktis serta terjangkau di era modern ini.

Meskipun model bisnis hotel kapsul secara inheren efisien, banyak pengelola masih menghadapi tantangan operasional yang signifikan akibat proses pemesanan yang manual dan sistem pembayaran yang belum sepenuhnya terintegrasi. Ketidaktertarikan pelanggan untuk melakukan pemesanan langsung dan ketergantungan yang tinggi pada *Online Travel Agency* (OTA) menyebabkan hilangnya potensi margin keuntungan akibat komisi tinggi. Selain itu, kurangnya otomatisasi pada *check-in/check-out* serta mekanisme pemberian diskon (*referral* dan *voucher*) yang belum terdigitalisasi menambah kompleksitas dan inefisiensi operasional. Sebuah solusi terpadu yang mampu meminimalkan intervensi manusia, mengoptimalkan setiap tahapan operasional, dan meningkatkan adopsi pemesanan langsung menjadi sangat mendesak.

Dua penelitian sebelumnya memberikan landasan yang relevan untuk mengatasi isu-isu tersebut. Allard dkk. (2024) telah menunjukkan bahwa sistem reservasi hotel berbasis web dapat secara signifikan meningkatkan efisiensi proses pemesanan kamar dan sekaligus mengurangi tingkat kesalahan manual yang sering terjadi. Senada dengan itu, Iskandar dkk. (2025) mengkonfirmasi bahwa semua komponen dalam sistem reservasi hotel berbasis web

dapat berfungsi dengan baik, menegaskan potensi besar dari solusi digital semacam ini. Penelitian-penelitian ini menggarisbawahi bahwa penerapan teknologi informasi yang komprehensif terbukti dapat secara substansial meningkatkan kinerja organisasi dan efisiensi operasional dalam industri perhotelan secara keseluruhan [3]. Pengalaman dari studi-studi ini menjadi pijakan penting dalam perancangan solusi yang lebih canggih.

Dalam konteks tantangan operasional dan potensi solusi teknologi, PT Horus Technology sebagai pengembang sistem Tamu.id, sebuah platform hotel kapsul modern, menghadapi permasalahan krusial yang harus diselesaikan. Perusahaan ini menyadari bahwa untuk tetap kompetitif dan memberikan layanan terbaik, mereka perlu mengatasi masalah-masalah seperti rendahnya pemesanan langsung, tingginya biaya komisi OTA, dan proses manual yang memakan waktu. Kebutuhan akan platform yang efisien dan menarik bagi pengguna menjadi prioritas utama PT Horus Technology dalam mengembangkan Tamu.id sebagai solusi akomodasi kapsul terdepan. Oleh karena itu, penelitian ini hadir sebagai upaya untuk memberikan solusi inovatif yang menjawab kebutuhan spesifik PT Horus Technology.

Untuk mengatasi permasalahan yang disebutkan dan mencapai efisiensi operasional yang lebih tinggi, penelitian ini akan mengembangkan dan mengimplementasikan sistem manajemen hotel kapsul terintegrasi yang inovatif pada platform Tamu.id. Salah satu fokus utamanya adalah optimalisasi pembayaran melalui integrasi QRIS (Quick Response Code Indonesian Standard), yang memungkinkan transaksi cepat dan aman serta mempermudah pemantauan *real-time* bagi pengelola, mengingat adopsi QRIS yang masif di Indonesia [4, 5]. Dalam arsitektur pembayaran digital, webhook juga akan dimanfaatkan sebagai mekanisme notifikasi *real-time* yang esensial dari penyedia layanan pembayaran (misalnya Midtrans) ke sistem hotel, secara drastis mengurangi kebutuhan verifikasi manual dan menekan biaya server dengan menghindari *polling* yang tidak efisien [6].

Sistem ini juga akan mencakup implementasi *check-in/check-out* otomatis berbasis QR Code dan teknologi IoT, di mana kunci pintu dapat divalidasi secara *offline* menggunakan struktur data *customized LinkedList* dan *Unix timestamp*. Desain ini memastikan reliabilitas akses bahkan tanpa koneksi internet dalam batas waktu tertentu, karena setiap *node* menyimpan data QR Code unik beserta *Unix timestamp* yang mendefinisikan masa berlaku kunci, selaras dengan arsitektur keamanan sistem akses IoT [7]. Selain itu, strategi harga dinamis akan diintegrasikan untuk memaksimalkan pendapatan dengan menyesuaikan harga kamar berdasarkan permintaan pasar, musim, atau waktu tertentu, meningkatkan fleksibilitas dan daya saing hotel. Fitur referral dan voucher juga akan disematkan untuk mendorong pemesanan langsung dan mengurangi ketergantungan pada OTA, menawarkan insentif unik bagi pelanggan.

Keamanan dan kenyamanan dalam proses autentikasi pengguna menjadi aspek penting dalam pengembangan aplikasi berbasis web. Dengan mengintegrasikan OAuth2, pengguna

dapat melakukan login ke sistem manajemen hotel kapsul menggunakan akun Google mereka tanpa perlu membuat akun baru atau mengingat kata sandi tambahan, yang tidak hanya meningkatkan keamanan tetapi juga memberikan pengalaman pengguna yang lebih baik [8]. Untuk memastikan pengelolaan properti yang akurat, sistem akan mampu melakukan sinkronisasi dua arah dengan berbagai OTA melalui *channel manager* terpusat dan dilengkapi Content Management System (CMS) yang fleksibel. Terakhir, pengembangan sistem ini akan didasari oleh metode Kanban, yang memungkinkan proses kerja iteratif dan responsif terhadap perubahan kebutuhan, serta mempromosikan transparansi dan kolaborasi tim untuk memastikan produk akhir memenuhi kebutuhan spesifik pengguna secara efektif [9].

Singkatnya, penelitian ini bertujuan untuk mengembangkan platform Tamu.id yang komprehensif dan efisien, yang mampu mengatasi masalah operasional hotel kapsul melalui otomatisasi, integrasi pembayaran modern, manajemen kunci cerdas, strategi harga fleksibel, dan pengalaman pengguna yang ditingkatkan, dengan harapan dapat mengurangi ketergantungan pada OTA dan meningkatkan profitabilitas dan kepuasan pelanggan.

1.2 Rumusan Masalah

Rumusan masalah yang dapat diambil dari latar belakang yang sudah dijabarkan sebelumnya antara lain:

1. Bagaimana mendesain dan mengimplementasikan sistem manajemen hotel kapsul berbasis web yang terintegrasi dan *Seamless*?
2. Bagaimana mengintegrasikan sistem pembayaran digital yang mudah dan aman?
3. Bagaimana memastikan sistem yang dikembangkan dapat diterima oleh pengguna?

1.3 Tujuan dan Manfaat Proyek Akhir

1.3.1 Tujuan Proyek Akhir

Berdasarkan rumusan masalah yang telah ditetapkan, tujuan dari proyek akhir ini adalah sebagai berikut:

1. Merancang dan membangun sebuah sistem manajemen hotel kapsul berbasis web yang terintegrasi, mulai dari pengelolaan operasional terpusat (ketersediaan kamar, kebersihan, dan harga dinamis) hingga pengalaman pengguna yang *seamless* melalui autentikasi *passwordless* (OAuth2) dan sistem kunci digital berbasis QR Code yang andal bahkan saat *offline* (dengan implementasi *LinkedList*).
2. Mengintegrasikan sistem pembayaran digital yang aman dan mudah melalui *payment gateway* Midtrans, dengan memanfaatkan *webhook* untuk notifikasi transaksi *real-time* dan menyediakan QRIS sebagai metode pembayaran utama yang universal bagi pengguna.

3. Mengembangkan fitur-fitur yang dirancang untuk meningkatkan penerimaan dan kepuasan pengguna, yang mencakup antarmuka yang ramah pengguna (*user-friendly*), sistem ulasan dan peringkat (*rating and review*), serta program loyalitas seperti *voucher* dan *affiliate marketing* yang memberikan keuntungan finansial bagi tamu.

1.3.2 Manfaat Proyek Akhir

Penyelesaian proyek akhir ini diharapkan dapat memberikan manfaat yang signifikan bagi berbagai pihak terkait, yang dapat dirincikan sebagai berikut:

A. Manfaat bagi Pengelola Hotel (Tamu.id)

- **Peningkatan Efisiensi Operasional:** Sistem terpusat memungkinkan pengelolaan data reservasi dari berbagai sumber (termasuk OTA), status kebersihan kamar, proses pindah kamar, dan penerapan harga dinamis secara otomatis. Hal ini mengurangi pekerjaan manual, meminimalkan *human error*, dan mempercepat alur kerja harian.
- **Peningkatan Pendapatan dan Profitabilitas:** Dengan platform reservasi mandiri yang kaya fitur (seperti *voucher* dan promosi), ketergantungan pada Online Travel Agency (OTA) yang membebankan komisi tinggi dapat dikurangi. Fitur *affiliate marketing* dengan kode *referral* juga membuka kanal pemasaran baru yang berpotensi meningkatkan okupansi.
- **Pengambilan Keputusan Berbasis Data:** Adanya fitur *rating* dan ulasan dari tamu memberikan masukan langsung yang berharga untuk perbaikan kualitas layanan. Data operasional yang terkumpul juga dapat dianalisis untuk membuat keputusan bisnis yang lebih strategis.

B. Manfaat bagi Tamu (Pengguna Akhir)

- **Pengalaman Pengguna yang Lebih Baik:** Proses dari pemesanan hingga *check-in* menjadi lebih mudah dan cepat. Pengguna dapat melakukan reservasi melalui antarmuka yang responsif, *login* tanpa kata sandi menggunakan akun Google, melakukan pembayaran dengan QRIS, dan masuk ke kamar hanya dengan memindai QR Code dari ponsel mereka.
- **Keamanan dan Kepercayaan:** Integrasi dengan *payment gateway* terpercaya dan mekanisme autentikasi modern (OAuth2, JWT) memberikan jaminan keamanan data dan transaksi. Sistem ulasan yang transparan juga membantu calon tamu dalam membuat keputusan yang lebih baik.
- **Keuntungan Tambahan:** Tamu dapat memperoleh keuntungan finansial melalui diskon yang didapat dari kode *referral* atau bahkan mendapatkan komisi dengan berpartisipasi dalam program afiliasi, memberikan nilai lebih dari sekadar menginap.

C. Manfaat dari Sisi Pengembangan Teknologi

- **Implementasi Arsitektur Modern:** Proyek ini menjadi studi kasus penerapan arsitektur sistem berbasis web modern yang menggabungkan *backend (Rest API)*, *payment gateway* (Midtrans), protokol otentikasi (OAuth2, JWT), dan integrasi dengan perangkat IoT (*smart lock*) secara efisien dan aman.
- **Solusi Inovatif untuk Masalah Offline:** Penggunaan struktur data *LinkedList* untuk sinkronisasi kredensial QR Code ke perangkat IoT merupakan sebuah solusi konkret dan inovatif untuk mengatasi potensi masalah konektivitas internet pada sistem kontrol akses.

1.4 Batasan Masalah

Karena keterbatasan waktu dan sumber daya, Tamu.id memiliki beberapa batasan masalah yang perlu diperhatikan, yaitu:

1. Aplikasi hanya dapat diakses pada web browser dan membutuhkan jaringan internet.
2. Pengguna tidak dapat melakukan pemesanan tanpa login ke dalam sistem.
3. Pengguna tidak dapat melakukan pemesanan lebih dari satu jenis Pod dalam satu waktu dengan pemesanan via Platform Tamu.id.
4. Integrasi Payment gateway yang digunakan adalah Midtrans dengan metode pembayaran QRIS saja menimbang QRIS adalah standar QR Code yang dikeluarkan oleh Bank Indonesia dan hampir semua bank dan e-wallet di Indonesia sudah mendukung QRIS.
5. Karena keterbatasan dokumen perizinan dan waktu belum ada integrasi API otomatis via webhook dengan online travel agent (OTA) seperti Traveloka, Agoda, dan Booking.com.
6. Integrasi API Backend dengan IoT diilustrasikan dengan menggunakan aplikasi web simulasi.

1.5 Sistematika Penulisan

Sistematika penulisan laporan Proyek Akhir ini disusun secara sistematis agar pembaca dapat memahami alur pembahasan dengan jelas dan runtut. Penulisan dibagi ke dalam beberapa bab, mulai dari pendahuluan hingga lampiran, dengan rincian sebagai berikut:

BAB I : PENDAHULUAN

Bab ini memberikan gambaran umum mengenai latar belakang permasalahan yang melatarbelakangi dilakukannya proyek akhir, perumusan masalah yang akan dikaji, tujuan yang hendak dicapai, serta manfaat yang diharapkan baik secara teoritis maupun praktis. Selain itu, bab ini juga memuat batasan masalah yang menjadi ruang lingkup kajian agar fokus pembahasan lebih terarah. Di akhir bab, dijelaskan pula sistematika penulisan yang memberikan panduan mengenai struktur dan isi masing-masing bab dalam laporan ini.

BAB II : STUDI PUSTAKA

Bab ini membahas berbagai teori, konsep, dan referensi dari penelitian atau sumber ilmiah terdahulu yang relevan dengan topik proyek akhir. Kajian pustaka ini mencakup pembahasan mengenai sistem e-ticketing, mekanisme reservasi daring, pemanfaatan teknologi QRCode, serta ulasan terhadap perangkat teknologi yang digunakan dalam pengembangan sistem, baik dari sisi perangkat lunak maupun perangkat keras. Studi pustaka ini menjadi dasar pijakan dalam merumuskan solusi dan pendekatan teknis yang digunakan dalam proyek akhir ini.

BAB III : METODE PENELITIAN

Bab ini menjelaskan pendekatan dan metode yang digunakan selama pelaksanaan proyek akhir. Penjelasan mencakup tahapan-tahapan dalam pengembangan sistem, mulai dari perencanaan, perancangan, implementasi, hingga pengujian. Penggunaan metode Kanban dijelaskan sebagai strategi manajemen proyek yang fleksibel dan adaptif. Selain itu, dibahas pula integrasi sistem secara teknis, seperti penerapan one-time token untuk penggabungan otentikasi OAuth2 milik Google dengan sistem login konvensional menggunakan email dan sandi, integrasi API dan webhook untuk sistem pembayaran otomatis, serta rancangan struktur data linked list berbasis timestamp sebagai mekanisme rotasi kunci pada sistem QRCode yang terintegrasi dengan perangkat IoT secara offline dan fault-tolerant.

BAB IV : HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil dari proses implementasi sistem yang telah dirancang pada bab sebelumnya. Uraian dilakukan secara sistematis mulai dari hasil pengembangan antarmuka pengguna, proses backend, serta integrasi sistem secara keseluruhan. Hasil pengujian terhadap sistem turut disertakan dalam bab ini, meliputi User Acceptance Testing (UAT) untuk menilai kesesuaian sistem terhadap kebutuhan pengguna, Black Box Testing untuk menguji fungsionalitas sistem untuk mengevaluasi performa sistem dalam kondisi beban tinggi. Pembahasan disusun secara analitis untuk mengkaji sejauh mana sistem memenuhi tujuan dan spesifikasi yang telah dirumuskan sebelumnya.

BAB V : PENUTUP

Bab ini merupakan bagian akhir dari laporan proyek akhir yang berisi kesimpulan dari seluruh proses pelaksanaan proyek, mulai dari identifikasi masalah hingga hasil pengujian sistem. Kesimpulan disusun berdasarkan temuan yang diperoleh selama pelaksanaan proyek. Selain itu, bab ini juga memuat saran-saran yang dapat digunakan sebagai acuan bagi pengembangan sistem lanjutan atau penelitian selanjutnya yang relevan dengan topik ini, dengan harapan dapat menyempurnakan dan memperluas cakupan sistem yang telah dikembangkan.

DAFTAR PUSTAKA : DAFTAR PUSTAKA

Bagian ini memuat seluruh referensi dan sumber literatur yang digunakan sebagai landasan teori dan acuan teknis dalam penyusunan laporan proyek akhir. Daftar pustaka dapat berupa buku, jurnal ilmiah, artikel daring, dokumentasi perangkat lunak, maupun sumber lainnya yang kredibel dan relevan.

LAMPIRAN : LAMPIRAN

Bagian lampiran menyajikan dokumen-dokumen pendukung yang tidak dicantumkan dalam bab utama, namun memiliki relevansi penting terhadap keseluruhan isi laporan. Lampiran dapat mencakup tangkapan layar sistem, potongan kode program penting, dokumentasi pengujian, konfigurasi sistem, diagram tambahan, serta dokumen lainnya yang mendukung validitas dan keterlacakan hasil proyek akhir.

BAB II

TINJAUAN PUSTAKA

2.1 Studi Pustaka

Laporan proyek akhir ini didasarkan pada berbagai penelitian terdahulu yang relevan dengan pengembangan aplikasi berbasis web untuk reservasi hotel. Fokus penelitian ini mencakup integrasi QR Code untuk sistem kunci digital, penggunaan OAuth2 sebagai metode otentikasi tanpa kata sandi, serta implementasi *webhook* untuk integrasi pembayaran. Studi pustaka ini bertujuan memberikan wawasan terkini terkait teknologi yang digunakan untuk membangun sistem penjualan tiket yang efisien, aman, dan mudah digunakan.

Penelitian sebelumnya [10] berjudul *Rancang Bangun Sistem Informasi Reservasi Hotel "Hotel Hebat" Berbasis Website* mengembangkan sistem informasi reservasi hotel yang memungkinkan pengguna untuk melakukan pemesanan kamar secara online. Sistem ini dibangun dengan menggunakan bahasa pemrograman PHP dan database MySQL. Hasil penelitian menunjukkan bahwa sistem informasi reservasi hotel ini dapat meningkatkan efisiensi dalam proses pemesanan kamar, mengurangi kesalahan manual, dan memberikan kemudahan bagi pengguna dalam mengakses informasi terkait ketersediaan kamar.

Selain itu, penelitian lain [11] berjudul *Rancang Bangun Sistem Reservasi Hotel pada Hotel Larona Berbasis Website* juga mengembangkan sistem informasi reservasi hotel berbasis web. Penelitian ini menggunakan metode pengembangan perangkat lunak Waterfall dan bahasa pemrograman PHP dengan database MySQL. Hasil penelitian menunjukkan bahwa semua komponen dalam sistem informasi reservasi hotel ini dapat berfungsi dengan baik.

Pada penelitian ini [12] yang melakukan perancangan *Sistem Informasi Pemesanan Tiket Bus Damri di Bandara menggunakan QR Code dan Web Base* bahwa digitalisasi dalam konteks pemesanan tiket dapat mempermudah baik dari sisi pengguna maupun pihak penyedia layanan. penelitian ini [12] dilakukan dengan tujuan untuk meningkatkan visibilitas dan efisiensi dalam proses pemesanan tiket bus Damri di Bandara XYZ. Hasil penelitian menunjukkan bahwa penggunaan QR Code dalam sistem pemesanan tiket bus Damri di Bandara Soekarno-Hatta dapat meningkatkan visibilitas dengan sistem informasi yang berbasis website dan efisiensi dengan QR Code dalam proses pengelolaan tiket bus Damri di Bandara XYZ.

Penelitian juga dilakukan oleh [13] dengan topik *Perancangan Aplikasi Pemesanan Tiket Masuk Pada Central Park Zoo Medan*, sistem ini dibangun dengan memadukan antara aplikasi berbasis web dan mobile yaitu di sisi admin menggunakan aplikasi berbasis web dan di sisi pengguna menggunakan aplikasi berbasis mobile, aplikasi web dibangun dengan

menggunakan bahasa pemrograman PHP dan database MySQL, sedangkan aplikasi mobile dibangun dengan menggunakan bahasa pemrograman Java berbasis Android. Hasil penelitian menunjukkan bahwa aplikasi pemesanan tiket masuk pada Central Park Zoo Medan dapat mempermudah pengunjung dalam melakukan pemesanan tiket secara online dengan berbagai metode pembayaran dan mempermudah pihak pengelola dalam melakukan pengelolaan data pengunjung serta mempercepat proses validasi tiket masuk.

Dengan sudut pandang yang lebih menarik pada penelitian [14] dengan judul *Development of IoT-Based E-ticket Selling and Management System with QR Code Scanner (Tickets.now)* yang mengintegrasikan teknologi Internet of Things (IoT) dalam pengembangan sistemnya yaitu dengan menggunakan ESP32-Cam dengan modul WiFi sebagai QR Code Scanner berbeda dengan penelitian lain yang menggunakan perangkat lunak kamera seperti smartphone ataupun webcam. Hasil penelitian menunjukkan bahwa penggunaan teknologi IoT dalam sistem e-ticketing juga dapat dilakukan dan tentunya memiliki kelebihan dan kekurangan masing-masing menyesuaikan dengan sumber daya yang dimiliki.

Mirip seperti penelitian dengan IoT sebelumnya, pada penelitian ini [15] dengan judul *Implementasi Pembayaran Dan Palang Otomatis Pada Sistem Smart Parking Di Lahan Parkir Menggunakan Metode QR Code* memiliki pembahasan mengenai IoT yang lebih komprehensif ditunjukkan dengan integrasi perangkat *Hardware* yang lebih lengkap, selain itu juga tentunya menggunakan QR Code sebagai basis Tiket dengan website yang dikembangkan dengan PHP dan MySQL serta TriPay sebagai payment gateway dalam sistem smart parking. Hasil penelitian menunjukkan bahwa implementasi pembayaran dan palang otomatis pada sistem smart parking di lahan parkir menggunakan metode QR Code dapat mempermudah pengguna dalam melakukan pembayaran dan mempercepat proses keluar masuk lahan parkir ditunjukkan dengan hasil uji coba fungsionalitas manual baik secara software yaitu website Admin dan website User untuk kemudian melakukan pembayaran, serta hardware yaitu palang otomatis berbasis QR Code dengan pengujian QoS (*Quality of Service*), Pengujian *Throughput*, dan Pengujian *Packet Loss*.

Penelitian berkaitan dengan payment gateway tripay juga dilakukan [16] dengan judul *Pembangunan Sistem Transaksi Penjualan Barang dan Jasa (Studi Kasus: Startup Botanika Kota)* yang membuat website untuk mendigitalisasi UMKM dalam proses operasinya dengan konsep MVC (model, view, controller) menggunakan Yii Framework, MySQL, dan Payment Gateway Tripay, metode pengembangan dilakukan secara *Waterfall*. Hasil penelitian menunjukkan bahwa pembangunan sistem transaksi penjualan barang dan jasa pada startup Botanika Kota dapat meningkatkan efektifitas dan efisiensi dalam operasionalnya ditunjukkan dengan pengujian unit menggunakan cyclomatic complexity paling tinggi bernilai 10 dan uji fungsi terhadap 3 sampel komponen dan integrasi perangkat lunak bernilai 100% valid.

Penelitian [17] juga dilakukan dengan judul *Implementasi Backend System Untuk*

Integrasi Payment Gateway Pada Sistem Pembayaran Kost Menggunakan Express.js yang bertujuan untuk mempermudah proses pembayaran kost dengan menggunakan Express.js sebagai backend system, MySQL sebagai database, dan Payment Gateway Midtrans, yang menarik dari penelitian ini adalah fokusnya yang ada pada membandingkan keamanan dari sisi integrasi payment gateway khususnya midtrans yaitu via *snap* tanpa backend dan *core API* menggunakan backend. Hasil penelitian menunjukkan bahwa implementasi backend system untuk integrasi payment gateway pada sistem pembayaran kost menggunakan Express.js dapat meningkatkan keamanan sistem pembayaran kost dibandingkan dengan sistem pembayaran menggunakan midtrans snap tanpa backend, disini secara tidak langsung menyebutkan bahwa sistem dengan backend yang mengimplemenasikan webhook lebih baik dalam hal keamanan maupun efektifitas dan efisiensi.

Ada juga penelitian [18] dengan judul *Sistem Pengesahan Dokumen dengan QR Code dan Memanfaatkan JSON Web Token (JWT) untuk Mengurangi Penyimpanan: Studi Kasus Universitas Esa Unggul Kampus Bekasi* yang bertujuan untuk mempermudah proses pengesahan dokumen dengan menggunakan QR Code dan JSON Web Token (JWT) untuk mengurangi penyimpanan, penelitian ini menggunakan metode Waterfall dengan menggunakan PHP, MySQL, Laragon, Codeigniter, dan JWT. Hasil penelitian menunjukkan bahwa sistem pengesahan dokumen dengan QR Code dan memanfaatkan JSON Web Token (JWT) untuk mengurangi penyimpanan dapat mempermudah proses pengesahan dokumen dan mengurangi penyimpanan data yang tidak perlu dengan kemampuan JWT yang dapat menyimpan data dalam bentuk token stateless yang aman dan dapat diverifikasi keabsahannya tanpa perlu selalu terhubung dengan database.

Terakhir penelitian [19] dengan judul *Student Mobile Attendance Application Using QRCode and Integrated with SSO at Universitas Sebelas Maret* yang bertujuan untuk mempermudah proses absensi mahasiswa oleh dosen via Aplikasi Android yang terintegrasi dengan SSO (Single Sign On) dalam proses autentikasinya, pengembangan aplikasi dilakukan dengan menggunakan metode SDLC dengan bahasa pemrograman Java sedangkan untuk autentikasinya itulah yang mengimplementasikan JWT lalu untuk absensi menggunakan QR Code. Hasil penelitian menunjukkan bahwa aplikasi absensi mahasiswa menggunakan QR Code dan terintegrasi dengan SSO di Universitas Sebelas Maret dapat mempermudah proses absensi mahasiswa oleh dosen dan mempercepat proses autentikasi mahasiswa.

Berikut adalah pada tabel 2.1 merupakan ringkasan dari berbagai penelitian yang telah dilakukan sebelumnya, yang menunjukkan fokus, metode, teknologi yang digunakan, dan hasil yang diperoleh dari masing-masing penelitian tersebut.

Tabel 2.1 Tabel Perbandingan Studi Pustaka

Penelitian	Fokus	Metode	Teknologi	Hasil
Allard dkk. (2024) [10]	Sistem reservasi hotel berbasis web	Rancang Bangun	PHP, MySQL	Meningkatkan efisiensi pemesanan kamar & mengurangi kesalahan manual.
Iskandar S. (2025) [11]	Sistem reservasi hotel berbasis web	Waterfall	PHP, MySQL	Semua komponen berfungsi dengan baik dalam sistem reservasi hotel.
Gintya P. (2020) [12]	Digitalisasi tiket bus Damri	Studi Kasus	Web, QR Code	Digitalisasi tiket meningkatkan visibilitas & efisiensi layanan.
Akbar dkk.(2023) [13]	Penggunaan tiket online untuk pengelolaan pengunjung	Studi Kasus	Web (PHP, MySQL), Mobile (Java, Android)	Aplikasi tiket online mempermudah pengelolaan pengunjung & validasi tiket.
Nayef dkk. (2023) [14]	Integrasi IoT dalam e-ticketing	Pengembangan Sistem (IoT)	ESP32-Cam, QR Code	Integrasi IoT dalam e-ticketing memungkinkan otomatisasi lebih lanjut.
Assidiqie dkk. (2022) [15]	Penerapan smart parking berbasis QR Code	Pengujian Kinerja (QoS, Throughput, Packet Loss)	PHP, MySQL, TriPay, QR Code, IoT	QR Code & otomatisasi smart parking mempercepat pembayaran & akses.
Rachmand dkk. (2023) [16]	Digitalisasi transaksi UMKM	Waterfall, Uji Unit (Cyclomatic Complexity)	YII Framework, MySQL, TriPay	Digitalisasi transaksi UMKM meningkatkan efisiensi operasional.
Juan dkk. (2024) [17]	Perbandingan integrasi Midtrans (Snap vs Core API)	Perbandingan Teknis	Express.js, MySQL, Midtrans (Core API)	Backend webhook lebih aman dan efisien dibandingkan Snap tanpa backend.

Lanjutan di halaman berikutnya...

Penelitian	Fokus	Metode	Teknologi	Hasil
Nixon dkk. (2024) [18]	Verifikasi QR Code menggunakan JWT	Waterfall	PHP, MySQL, Codeigniter, JWT, QR Code	QR Code + JWT mengurangi penyimpanan data & mempermudah verifikasi.
Winarno dkk. (2018) [19]	Implementasi SSO dengan QR Code	SDLC, Implementasi SSO	Java (Android), SSO, JWT, QR Code	Integrasi QR Code & SSO mempercepat autentikasi & absensi mahasiswa.

Dari berbagai penelitian tersebut, terlihat adanya benang merah dan persamaan dengan penelitian yang dilakukan kali ini, terutama pada implementasi dan integrasi teknologi. Beberapa studi sebelumnya, seperti [14] yang mengintegrasikan IoT dalam *e-ticketing*, [15] yang mengintegrasikan *hardware* pada *smart parking*, [16] yang mengintegrasikan *payment gateway* TriPay pada UMKM, dan [17] yang membandingkan keamanan integrasi *payment gateway* Midtrans melalui *webhook*, menunjukkan bahwa implementasi dan integrasi teknologi yang tepat dapat memberikan nilai tambah yang signifikan bagi sistem yang dikembangkan. Persamaan ini mengukuhkan relevansi pendekatan yang digunakan dalam pengembangan Tamu.id.

Namun, penelitian ini memiliki keunikan dan keunggulan signifikan dibandingkan studi-studi pustaka yang ada. Keunggulan utama terletak pada integrasi berbagai teknologi secara komprehensif yang belum ditemukan secara utuh dalam satu sistem sebelumnya. Ini mencakup implementasi OAuth2 untuk autentikasi yang lebih aman dan tanpa kata sandi, pemanfaatan *webhook* untuk notifikasi pembayaran *real-time* yang efisien dan aman, serta sistem kunci digital berbasis QR Code dengan regenerasi kunci yang mengimplementasikan *Customized LinkedList* dan *Unix timestamp*. Struktur data ini secara spesifik dirancang untuk mengatasi masalah *offline fault tolerance* pada integrasi IoT kunci digital, sebuah aspek krusial yang jarang dibahas secara mendalam. Selain itu, penerapan harga dinamis, fitur *referral* dan *voucher* yang terintegrasi langsung ke dalam *platform*, serta penggunaan metode Kanban dalam proses pengembangan, semakin memperkaya keunikan penelitian ini. Kombinasi inovasi ini bertujuan untuk secara holistik meningkatkan profitabilitas hotel kapsul dan kepuasan pelanggan pada *platform* Tamu.id.

2.2 Dasar Teori

Pembuatan proyek akhir ini tentunya merujuk pada beberapa teori dan konsep yang relevan. Studi pustaka ini bertujuan untuk memberikan landasan teoritis yang mendukung pengembangan proyek akhir ini. Beberapa konsep yang menjadi dasar dalam proyek akhir ini antara lain:

2.2.1 Hotel Kapsul

Hotel kapsul adalah jenis akomodasi yang menawarkan ruang tidur kecil dan efisien, biasanya dalam bentuk kapsul atau pod. Konsep ini berasal dari Jepang dan telah menyebar ke berbagai negara di seluruh dunia. Hotel kapsul dirancang untuk memberikan pengalaman menginap yang nyaman dengan harga terjangkau, terutama bagi pelancong yang mencari akomodasi sementara [2], istilah kamar/room dalam hotel konvensional bila dalam hotel kapsul berubah sesuai dengan bentuknya sebut saja pada platform Tamu.id kamar disebut sebagai Pod, dan dalam satu kapsul hotel terdiri dari beberapa jenis Pod dan tiap jenis pod memiliki spesifikasi dan jumlah Pod yang berbeda-beda.

2.2.2 Sistem Informasi

Sistem informasi adalah kumpulan komponen yang saling berinteraksi untuk mengumpulkan, mengelola, memproses, dan menyebarkan informasi guna mendukung pengambilan keputusan yang efektif dan efisien [20]. Sistem informasi terdiri dari beberapa elemen utama, yaitu perangkat keras, perangkat lunak, manusia, data, dan prosedur yang saling berhubungan untuk mencapai tujuan tertentu.

2.2.3 Website

Website adalah sekumpulan halaman web yang saling terhubung dan dapat diakses melalui internet menggunakan protokol HTTP atau HTTPS [21]. Website dirancang untuk menyediakan informasi, layanan, atau hiburan kepada pengguna. Dalam konteks modern, website menjadi elemen penting dalam kehidupan sehari-hari, mulai dari e-commerce, pendidikan, hingga media sosial.

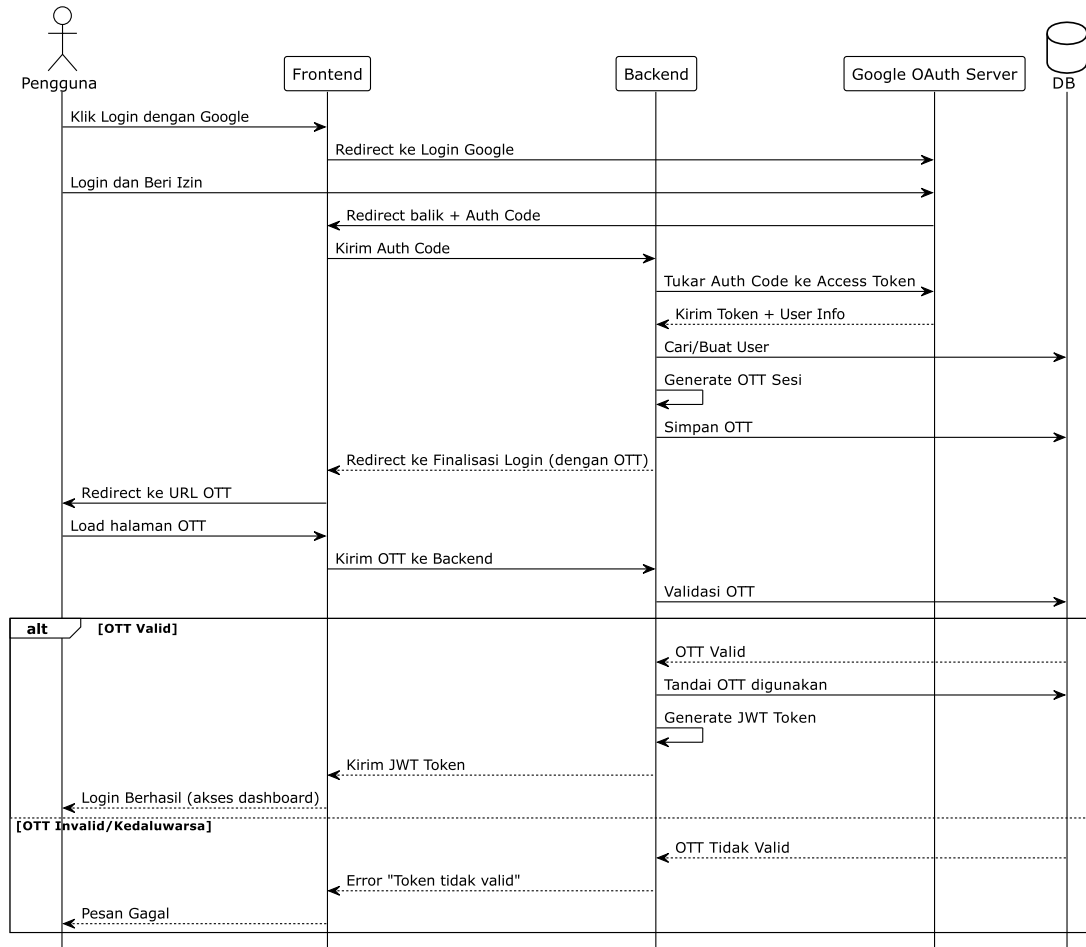
Website terdiri dari dua komponen utama: front-end dan back-end. Front-end mengacu pada tampilan antarmuka pengguna yang dapat diakses melalui browser, sedangkan back-end mencakup server, database, dan logika aplikasi yang mendukung fungsi website tersebut.

Perkembangan teknologi web, seperti HTML5, CSS3, dan JavaScript, telah memungkinkan website menjadi lebih interaktif dan responsif. Selain itu, keberadaan sistem manajemen konten (CMS) seperti WordPress dan Joomla memudahkan pengguna tanpa latar belakang teknis untuk mengelola konten website mereka.

2.2.4 OAuth 2.0

OAuth 2.0 adalah protokol otorisasi yang dirancang untuk memberikan akses terbatas ke sumber daya pengguna di server tanpa perlu membagikan kredensial mereka secara langsung [22]. Protokol ini biasanya digunakan dalam aplikasi web, perangkat mobile, dan layanan pihak ketiga lainnya.

Diagram Sekuens Login Google dengan (OTT)



Gambar 2.1 Ilustrasi Protokol OAuth 2.0

Seperti pada gambar 2.1, prinsip kerja OAuth 2.0 melibatkan beberapa entitas, yaitu resource owner (pemilik sumber daya), client (aplikasi pihak ketiga), authorization server (server otorisasi), dan resource server (server sumber daya). Dalam skenario OAuth 2.0, pengguna memberikan izin kepada aplikasi pihak ketiga untuk mengakses data mereka melalui token akses yang diterbitkan oleh server otorisasi.

OAuth 2.0 sering digunakan oleh platform besar seperti Google, Facebook, dan GitHub untuk memungkinkan integrasi dengan layanan mereka. Keamanan dalam OAuth 2.0 sangat bergantung pada implementasi yang benar, termasuk penggunaan HTTPS, pengelolaan token dengan benar, dan mitigasi serangan seperti CSRF.

2.2.5 Webhook

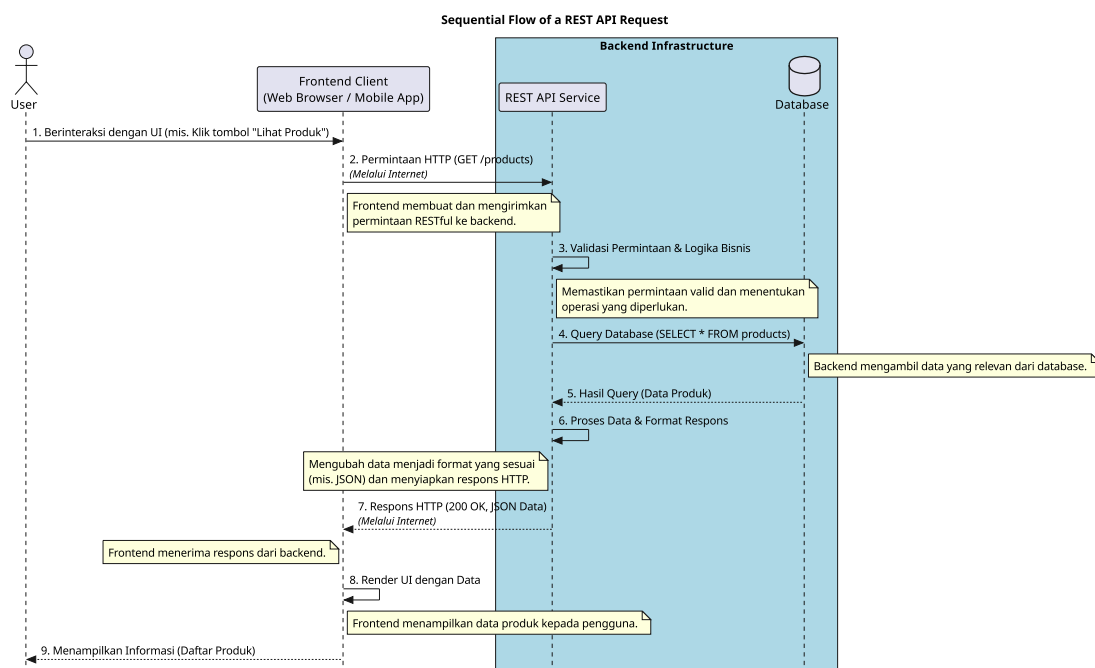
Webhook adalah mekanisme komunikasi antara server yang memungkinkan pemberitahuan secara real-time melalui HTTP POST ketika suatu peristiwa tertentu terjadi [23]. Webhook sering digunakan dalam pengintegrasian aplikasi untuk memastikan data tetap sinkron tanpa perlu polling terus-menerus.

Dalam praktiknya, webhook bekerja dengan cara mengirimkan payload berupa data JSON atau XML ke URL tertentu yang telah ditentukan oleh pengguna. Misalnya, layanan pembayaran online dapat menggunakan webhook untuk memberi tahu merchant ketika pembayaran berhasil dilakukan.

Keuntungan utama webhook adalah efisiensi karena server hanya mengirimkan data saat diperlukan, dibandingkan dengan polling yang memeriksa perubahan secara berkala. Namun, implementasi webhook memerlukan pengamanan ekstra untuk mencegah akses tidak sah, seperti menggunakan token validasi atau enkripsi data.

2.2.6 Rest API (*Representational State Transfer Application Programming Interface*)

REST API (Representational State Transfer Application Programming Interface) adalah arsitektur layanan web yang dirancang untuk mendukung komunikasi antara klien dan server melalui protokol HTTP. REST diperkenalkan oleh Roy Fielding dalam disertasinya pada tahun 2000, dengan prinsip utama bahwa setiap sumber daya pada server diidentifikasi melalui URI (Uniform Resource Identifier) [24].



Gambar 2.2 Ilustrasi REST API

Seperti pada gambar 2.2, REST API memanfaatkan metode HTTP seperti GET, POST, PUT, dan DELETE untuk melakukan operasi pada sumber daya. Pendekatan ini memungkinkan pengembangan layanan web yang skalabel, sederhana, dan mudah diintegrasikan dengan berbagai platform. REST API banyak digunakan dalam pengembangan aplikasi modern karena sifatnya yang ringan dan kompatibel dengan format data seperti JSON dan XML [25]. Beberapa karakteristik utama REST API meliputi:

1. Client-Server: Memisahkan antara klien dan server, memungkinkan evolusi independen

dari kedua komponen.

2. Stateless: Setiap permintaan dari klien ke server harus berisi semua informasi yang dibutuhkan untuk memahami dan memproses permintaan tersebut.
3. Cacheability: Respons dapat ditandai sebagai dapat di-cache atau tidak untuk mengoptimalkan efisiensi.
4. Layered System : Klien tidak perlu mengetahui detail implementasi server, memungkinkan fleksibilitas arsitektur.

REST API telah menjadi standar de facto untuk menghubungkan berbagai aplikasi, terutama dalam ekosistem aplikasi berbasis cloud dan microservices [26].

2.2.7 Software Development Life Cycle dengan Metode Kanban Board

Software Development Life Cycle (SDLC) adalah kerangka kerja sistematis untuk merencanakan, membangun, menguji, dan memelihara perangkat lunak. Dalam penelitian ini, alur kerja pengembangan dikelola menggunakan metode *Kanban Board*, yang menekankan pada pengelolaan tugas secara visual, efisien, dan adaptif terhadap perubahan kebutuhan.

Kanban Board menyediakan pendekatan visual dalam mengelola tugas melalui tiga kolom utama: *To Do*, *In Progress*, dan *Complete*. Setiap tugas direpresentasikan sebagai kartu yang berpindah antar kolom sesuai dengan statusnya. Pendekatan ini membantu membatasi pekerjaan yang berjalan secara bersamaan (*Work In Progress/WIP*), menjaga aliran kerja yang berkelanjutan, serta memberikan transparansi terhadap progres proyek [9].

Penerapan *Kanban Board* dalam SDLC dilakukan secara bertahap, dimulai dari penempatan aktivitas seperti analisis kebutuhan dan perancangan sistem pada kolom *To Do*, pengembangan fitur pada *In Progress*, hingga penyelesaian tugas yang dipindahkan ke kolom *Complete*. Skema ini mendukung proses pengembangan perangkat lunak yang lebih terstruktur, dapat dipantau dengan mudah, dan memungkinkan perbaikan berkelanjutan berdasarkan evaluasi yang dilakukan secara berkala [27].

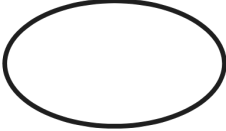
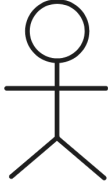
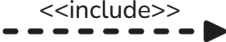
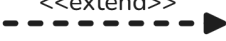
2.2.8 Unified Modeling Language (UML)

Unified Modeling Language (UML) merupakan bahasa pemodelan standar yang digunakan untuk menentukan, memvisualisasikan, membangun, dan mendokumentasikan artefak perangkat lunak serta sistem lainnya. UML memfasilitasi pemahaman antar tim dalam proses pengembangan dengan menyediakan serangkaian notasi yang konsisten dan formal. Komponen utama dalam UML mencakup diagram struktural seperti diagram kelas dan diagram komponen, serta diagram perilaku seperti diagram aktivitas dan diagram use case. UML dirancang untuk mendukung berbagai metodologi pengembangan perangkat lunak dan menjadi alat penting dalam rekayasa perangkat lunak modern [28].

2.2.9 Use Case Diagram

Use case diagram digunakan untuk memahami interaksi antara sistem dan aktor, yang merupakan entitas eksternal yang berinteraksi dengan sistem tanpa menjadi bagian dari sistem. Diagram ini menggambarkan fungsi-fungsi utama sistem, yang disebut sebagai use case, serta hubungan antara aktor dan use case. Elemen-elemen dalam use case diagram meliputi aktor, use case, dan hubungan yang menghubungkan keduanya, membantu menyederhanakan representasi kebutuhan fungsional sistem [28], berikut pada tabel 2.2 adalah notasi yang digunakan pada use case diagram.

Tabel 2.2 Notasi pada *Use Case Diagram*

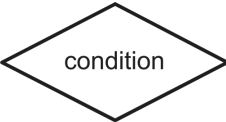
Notasi	Nama	Keterangan
	<i>Use Case</i>	Mewakili serangkaian langkah atau fungsi yang dijalankan sistem untuk menghasilkan nilai atau hasil yang dirasakan oleh aktor. Digunakan untuk menggambarkan perilaku sistem dari perspektif pengguna.
	<i>Actor</i>	Entitas eksternal (manusia atau sistem lain) yang berinteraksi dengan sistem. Aktor menjalankan peran tertentu dalam kaitannya dengan satu atau lebih use case.
	<i>Association</i>	Garis yang menghubungkan aktor dengan use case untuk menunjukkan adanya interaksi atau komunikasi di antara keduanya.
	<i>Include</i>	Hubungan yang menunjukkan bahwa suatu use case selalu menyertakan perilaku dari use case lain. Digunakan untuk memecah use case kompleks menjadi bagian yang lebih kecil dan dapat digunakan ulang.
	<i>Extend</i>	Menunjukkan bahwa suatu use case dapat memperluas perilaku use case lain secara opsional, tergantung pada kondisi tertentu. Cocok untuk skenario tambahan atau pengecualian.
	<i>System Boundary</i>	Kotak atau bingkai yang membatasi cakupan sistem. Semua use case yang ada di dalamnya merupakan bagian dari sistem yang sedang dimodelkan.

	<i>Generalization</i>	Menggambarkan hubungan pewarisan antar aktor atau use case. Aktor atau use case anak mewarisi perilaku dari induknya, dan dapat menambahkan perilaku khusus.
	<i>Package</i>	Digunakan untuk mengelompokkan elemen-elemen dalam diagram agar lebih modular, terstruktur, dan mudah dipahami. Cocok saat sistem kompleks dan terdiri dari banyak komponen.
	<i>Comment</i>	Memberikan penjelasan tambahan atau catatan khusus yang tidak tergambar langsung dari simbol diagram. Membantu dokumentasi dan pemahaman konteks sistem.

2.2.10 Activity Diagram

Activity diagram digunakan untuk menggambarkan alur kerja atau proses bisnis dalam suatu sistem. Diagram ini menampilkan aktivitas-aktivitas yang dilakukan dan urutan alurnya. Setiap aktivitas merepresentasikan tindakan tertentu yang dilakukan oleh aktor atau sistem. Elemen penting dalam activity diagram meliputi nodes (kegiatan atau tindakan), edges (aliran), decision points, dan forks. Diagram ini sangat bermanfaat untuk menganalisis alur proses dan mengidentifikasi area untuk optimasi [29], berikut pada tabel 2.3 adalah notasi yang digunakan pada activity diagram.

Tabel 2.3 Notasi pada *Activity Diagram*

Notasi	Nama	Keterangan
	<i>Initial Node</i>	Menandai titik awal dari sebuah proses atau aktivitas dalam diagram. Hanya ada satu initial node dalam satu alur proses.
	<i>Final Node</i>	Menunjukkan titik akhir dari keseluruhan proses. Alur aktivitas berhenti sepenuhnya saat mencapai elemen ini.
	<i>Activity</i>	Merepresentasikan satu aksi, langkah, atau tugas spesifik yang dilakukan dalam alur kerja sistem atau pengguna.
	<i>Decision/Condition</i>	Menggambarkan percabangan alur berdasarkan kondisi tertentu. Setiap cabang berasal dari keputusan logis (true/false atau pilihan lain).

	<i>Swimlane</i>	Membagi aktivitas berdasarkan pelaku atau unit yang bertanggung jawab, seperti pengguna, sistem, atau komponen lainnya. Membantu memperjelas siapa yang melakukan apa.
--	-----------------	--

2.2.11 Entities Relationship Diagram (ERD)

Entity Relationship Diagram (ERD) digunakan untuk memodelkan struktur data dalam suatu sistem. Diagram ini merepresentasikan entitas, atribut, dan hubungan antar entitas. Entitas biasanya digambarkan sebagai persegi panjang, atribut sebagai elips, dan hubungan sebagai belah ketupat. ERD membantu memvisualisasikan bagaimana data diatur dan dihubungkan, mempermudah perancangan database secara terstruktur [30], berikut pada tabel 2.4 adalah notasi yang digunakan pada ERD.

Tabel 2.4 Notasi pada *Entity Relationship Diagram (ERD)*

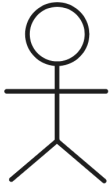
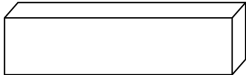
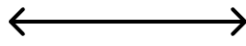
Notasi	Nama	Keterangan
	<i>Association</i>	Menunjukkan hubungan antara dua entitas dalam sistem. Biasanya digunakan untuk merepresentasikan koneksi logis atau fungsional, seperti "memiliki", "mengelola", atau "terdiri dari".
	<i>Entity</i>	Mewakili objek nyata atau abstrak yang memiliki data dan perlu disimpan dalam sistem, seperti pengguna, produk, atau transaksi. Entitas biasanya digambarkan dalam bentuk persegi panjang.
	<i>Attribute</i>	Menjelaskan detail atau properti dari suatu entitas atau relasi. Contohnya seperti nama, ID, tanggal, yang semuanya menyimpan nilai spesifik untuk setiap instansi entitas.
	<i>Relation</i>	Menyatakan hubungan antar entitas, sering kali dilengkapi dengan kardinalitas (1:1, 1:N, N:N) untuk menunjukkan jumlah keterlibatan antar entitas dalam relasi tersebut.

2.2.12 Deployment Diagram

Deployment Diagram digunakan untuk menggambarkan bagaimana perangkat lunak di-deploy pada infrastruktur fisik. Diagram ini menunjukkan node (server, perangkat, atau komponen) dan hubungan antar node tersebut. Setiap node dapat memiliki komponen

perangkat lunak yang berjalan di atasnya, serta koneksi jaringan yang menghubungkan berbagai node. Deployment Diagram membantu dalam memahami arsitektur sistem dan bagaimana komponen-komponen perangkat lunak berinteraksi dalam lingkungan produksi, berikut pada tabel 2.5 adalah notasi yang digunakan pada deployment diagram.

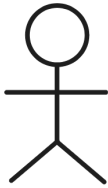
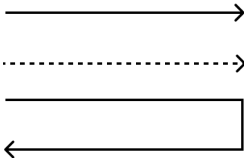
Tabel 2.5 Notasi pada *Deployment Diagram*

Notasi	Nama	Keterangan
	<i>Actor</i>	Mewakili entitas eksternal, biasanya manusia, yang berinteraksi dengan sistem. Contohnya pengguna, admin, atau operator.
	<i>Node</i>	Menunjukkan perangkat keras, layanan, atau komponen logis tempat proses komputasi berjalan, seperti server, container, atau service.
	<i>Package</i>	Mengelompokkan beberapa node atau komponen secara logis untuk menyusun arsitektur sistem secara modular dan terstruktur.
	<i>Cloud</i>	Representasi abstrak dari layanan berbasis cloud seperti GCP, AWS, Azure, atau Internet yang menjadi penghubung antar layanan.
	<i>Artifact</i>	Hasil build atau unit perangkat lunak yang dideploy, seperti aplikasi frontend, backend, skrip konfigurasi, atau file binary.
	<i>Database</i>	Komponen penyimpanan data terstruktur, mewakili sistem manajemen basis data seperti PostgreSQL, MySQL, atau SQLite.
	<i>Storage</i>	Media penyimpanan file atau objek tidak terstruktur, misalnya bucket di Google Cloud Storage, Amazon S3, dan sejenisnya.
   	<i>Arrow</i>	Digunakan untuk menggambarkan hubungan atau komunikasi antar komponen. Panah solid menunjukkan komunikasi langsung (aktif), panah putus-putus untuk akses pasif (misalnya CDN atau asset fetch), dan panah dua arah mewakili interaksi bolak-balik.

2.2.13 Sequence Diagram

Sequence Diagram digunakan untuk menggambarkan interaksi antar objek dalam suatu sistem berdasarkan urutan waktu. Diagram ini menampilkan objek-objek yang terlibat, pesan yang dikirim antar objek, dan urutan eksekusi pesan tersebut. Sequence Diagram membantu dalam memahami bagaimana objek berinteraksi satu sama lain untuk menyelesaikan suatu fungsi atau skenario tertentu, berikut pada tabel 2.6 adalah notasi yang digunakan pada sequence diagram.

Tabel 2.6 Notasi pada *Sequence Diagram*

Notasi	Nama	Keterangan
	<i>Actor</i>	Mewakili entitas eksternal seperti pengguna atau sistem lain yang memicu interaksi dalam proses yang digambarkan.
	<i>Combined Fragment</i>	Menyatakan blok kendali alur seperti percabangan, pengulangan, kondisi opsional, dan paralelisme. Label di pojok kiri atas menunjukkan jenis kontrolnya, seperti <i>opt</i> , <i>alt</i> , <i>loop</i> , <i>par</i> , dan lainnya.
	<i>Lifeline/Participant</i>	Representasi objek atau komponen sistem yang terlibat dalam proses, seperti Frontend, Backend, atau API. Digambarkan sebagai garis vertikal yang menunjukkan eksistensinya sepanjang waktu.
	<i>Activation Bar</i>	Menunjukkan periode di mana sebuah lifeline sedang menjalankan aktivitas atau metode tertentu. Biasanya digambarkan sebagai kotak vertikal tipis.
	<i>Arrow</i>	Menggambarkan komunikasi atau interaksi antar elemen. Panah solid mewakili permintaan (sinkron), panah putus-putus merepresentasikan respons atau hasil (asinkron), dan panah melengkung digunakan untuk pemanggilan metode pada objek itu sendiri (self-message).

	<i>Comment</i>	Menyediakan keterangan tambahan yang tidak direpresentasikan secara eksplisit dalam notasi diagram, guna memperjelas konteks atau penjelasan proses tertentu.
---	-----------------------	--

2.2.14 Teknologi dan Kakas Pengembangan

Dalam pengembangan proyek akhir ini, penulis menggunakan beberapa teknologi dan kakas pengembangan yang mendukung proses pengembangan. Beberapa teknologi dan kakas pengembangan yang digunakan antara lain:

- **VueJS:** VueJS adalah pustaka JavaScript yang digunakan untuk membangun antarmuka pengguna yang interaktif dan responsif. VueJS menggunakan konsep komponen yang dapat digunakan kembali untuk memisahkan tampilan dari logika aplikasi, memungkinkan pengembangan yang lebih efisien dan mudah diatur [31].
- **Python:** Python adalah bahasa pemrograman tingkat tinggi yang digunakan untuk pengembangan aplikasi web, analisis data, dan kecerdasan buatan. Python memiliki sintaks yang sederhana dan mudah dipahami, serta mendukung berbagai pustaka dan kerangka kerja yang mempermudah pengembangan perangkat lunak [32].
- **Flask:** Flask adalah kerangka kerja web berbasis Python yang digunakan untuk membangun aplikasi web dan API. Flask dirancang untuk menjadi ringan dan fleksibel, memungkinkan pengembang untuk memilih komponen yang sesuai dengan kebutuhan proyek mereka. Flask juga mendukung berbagai ekstensi yang mempermudah pengembangan aplikasi web [33].
- **PostgreSQL:** PostgreSQL adalah sistem manajemen basis data relasional (RDBMS) open-source yang digunakan untuk menyimpan dan mengelola data. PostgreSQL mendukung berbagai fitur seperti transaksi, indeks, dan fungsi yang memungkinkan pengembang untuk membangun aplikasi yang aman dan efisien [34].
- **Git:** Git adalah sistem kontrol versi yang digunakan untuk mengelola perubahan kode sumber dalam pengembangan perangkat lunak. Git memungkinkan pengembang untuk bekerja secara kolaboratif, melacak perubahan kode, dan mengelola versi kode dengan mudah [35].
- **GitLab:** GitLab adalah platform pengembangan perangkat lunak berbasis Git yang menyediakan repositori kode, manajemen proyek, dan alat kolaborasi untuk tim pengembang. GitLab memungkinkan pengembang untuk menyimpan kode sumber, melacak masalah, dan berkolaborasi dalam proyek secara efisien [36].

- **Visual Studio Code:** Visual Studio Code adalah editor kode sumber ringan dan kuat yang dikembangkan oleh Microsoft. Visual Studio Code menyediakan berbagai fitur seperti penyorotan sintaks, debugging, dan ekstensi yang mempermudah pengembangan perangkat lunak [37].
- **Google Cloud Run:** Google Cloud Run adalah layanan komputasi tanpa server yang memungkinkan pengembang untuk menjalankan aplikasi kontainer di infrastruktur Google Cloud. Cloud Run secara otomatis mengelola skala dan sumber daya yang dibutuhkan untuk menjalankan aplikasi, memungkinkan pengembang untuk fokus pada pengembangan tanpa perlu khawatir tentang manajemen infrastruktur [38].
- **Hoppscotch:** hoppscotch adalah aplikasi web open-source yang digunakan untuk menguji dan mengelola API. hoppscotch menyediakan antarmuka yang intuitif dan mudah digunakan untuk membuat dan mengirim permintaan HTTP ke API, serta melihat respons yang dihasilkan [39].
- **JSON Web Token (JWT):** JSON Web Token (JWT) adalah standar terbuka yang digunakan untuk membuat token akses yang aman dan dapat diverifikasi. JWT digunakan dalam aplikasi web modern untuk otentikasi dan otorisasi pengguna, serta pertukaran informasi yang aman antara klien dan server [40].
- **SQLAlchemy:** SQLAlchemy adalah ORM (Object-Relational Mapping) yang digunakan untuk menghubungkan aplikasi Backend python yang menggunakan Flask dengan basis data. SQLAlchemy menyediakan API yang intuitif dan deklaratif untuk berinteraksi dengan basis data, memungkinkan pengembang untuk fokus pada logika aplikasi tanpa perlu menulis query SQL secara manual [41].
- **Google Cloud Storage:** Google Cloud Storage adalah server penyimpanan objek (object storage) yang merupakan salah satu lini produk dari google cloud [42].
- **Docker:** Docker adalah platform open-source yang digunakan untuk mengemas, mengirim, dan menjalankan aplikasi dalam kontainer. Docker memungkinkan pengembang untuk membuat lingkungan yang konsisten dan portabel untuk aplikasi, serta mempercepat siklus pengembangan dan penyebaran aplikasi [43].
- **Latex:** Latex adalah sistem penyiapan dokumen yang digunakan untuk membuat dokumen yang berkualitas tinggi, seperti artikel ilmiah, laporan, dan buku. Latex menggunakan sintaks markup untuk mengatur struktur dokumen, memungkinkan pengembang untuk fokus pada konten tanpa perlu khawatir tentang tata letak [44].

2.2.15 Pengujian Sistem

Setelah pengembangan sistem selesai, langkah selanjutnya adalah melakukan pengujian untuk memastikan bahwa sistem berfungsi dengan baik dan sesuai dengan kebutuhan

pengguna. Pengujian sistem adalah proses verifikasi dan validasi yang dilakukan untuk mengevaluasi kualitas sistem dan memastikan bahwa sistem memenuhi spesifikasi yang telah ditentukan. Beberapa jenis pengujian sistem yang umum dilakukan antara lain:

1. **Pengujian Black Box:** Pengujian ini berfokus pada validasi fungsionalitas sistem tanpa memperhatikan struktur internal atau kode sumbernya. Tujuan utama dari pengujian ini adalah untuk memastikan bahwa masukan (input) menghasilkan keluaran (output) yang sesuai dengan spesifikasi [45].
2. **User Acceptance Testing (UAT) :** UAT adalah tahap akhir pengujian yang melibatkan pengguna akhir untuk memastikan bahwa sistem memenuhi kebutuhan bisnis dan dapat digunakan sebagaimana mestinya. Pada UAT, setiap fungsi sistem diuji berdasarkan skenario yang dirancang sesuai dengan kebutuhan pengguna [46]. Tingkat keberhasilan UAT dapat dihitung dengan rumus berikut:

Jawaban	Bobot
Sangat Setuju(SS)	5
Setuju(S)	4
Cukup Setuju(CS)	3
Tidak Setuju(TS)	2
Sangat Tidak Setuju(STS)	1

Tabel 2.7 Bobot Jawaban UAT berdasarkan skala *Likert*

Penentuan skor UAT dihitung menggunakan rumus pada Persamaan 2.1 dan hasil dari Skor tersebut akan dibandingkan dengan pengelompokkan pada tabel 2.7 berdasarkan skala *Likert*.

$$Hasil(\%) = \left(\frac{A}{B \times N} \right) \times 100\% \quad (2.1)$$

Keterangan:

A = total skor yang diperoleh dari semua responden

B = maksimum skor yang dapat diperoleh dari setiap responden

N = total responden

Persentase akhir UAT kemudian dapat dijadikan sebagai indikator kelayakan sesuai kategori yang ditunjukkan pada Tabel 2.8.

No.	Skor	Kategori Kelayakan
1.	< 21%	Sangat Tidak Layak
2.	21-40%	Tidak Layak
3.	41-60%	Cukup Layak
4.	61-80%	Layak
5.	81-100%	Sangat Layak

Tabel 2.8 Kategori Kelayakan UAT

BAB III

METODE PROYEK AKHIR

Metodologi yang diterapkan pada penelitian menjadi pokok bahasan pada bab ini. Alur perencanaan penelitian, pengumpulan dan analisis data, serta hasil penelitian akan dijelaskan secara terperinci. Sebagai tambahan, penjelasan dalam bab ini juga meliputi komponen-komponen lain seperti pendekatan penelitian, teknik pengumpulan data, jenis data yang terkumpul, metode analisis, dan instrumen pendukung yang digunakan.

3.1 Bahan

Bahan berupa data yang digunakan penulis untuk mengerjakan penelitian ini diambil langsung dari sistem hotel kapsul Tamu.id cabang jalan Magelang Yogyakarta yang sudah ada dengan observasi langsung ke lokasi. Data yang diambil berupa data detail cabang, gambar, fasilitas, jenis pod, pod, staff, harga, dan lain-lain yang diambil pada periode bulan Maret 2025 hingga April 2025.

3.2 Peralatan dan Kakas

Peralatan dan kakas yang digunakan dalam pengembangan aplikasi Tamu.id dan proyek akhir ini adalah sebagai berikut:

3.2.1 Perangkat Lunak

1. Visual Studio Code: Editor kode sumber yang digunakan untuk menulis kode program.
2. Flask: Framework Python yang digunakan untuk membangun backend aplikasi.
3. VueJS: Library JavaScript yang digunakan untuk membangun frontend aplikasi.
4. Tailwind CSS: Framework CSS yang digunakan untuk mendesain antarmuka pengguna.
5. PrimeVue: Komponen UI yang digunakan untuk mempercepat pengembangan antarmuka pengguna terutama bagian Super Admin dan Admin.
6. Axios: Library JavaScript yang digunakan untuk melakukan permintaan HTTP dari frontend ke backend.
7. JSON Web Token (JWT): Standar terbuka yang digunakan untuk mengamankan komunikasi antara frontend dan backend.
8. Docker: Platform yang digunakan untuk mengemas aplikasi dan dependensinya ke dalam kontainer, sehingga memudahkan pengembangan, pengujian, dan penyebaran.
9. GitLab: Platform pengembangan perangkat lunak yang digunakan untuk mengelola kode sumber.

10. PostgreSQL: Sistem manajemen basis data relasional yang digunakan untuk menyimpan data aplikasi.
11. Google Cloud Run: serverless platform tempat deploy aplikasi backend.
12. Google Cloud Storage: Layanan penyimpanan objek yang digunakan untuk menyimpan file statis.
13. Google Cloud SQL: Layanan basis data terkelola yang digunakan untuk menyimpan data aplikasi di lingkungan Produksi.
14. Cloudflare Pages: Layanan hosting statis yang digunakan untuk menyajikan aplikasi frontend.
15. Midtrans Core API: Layanan pembayaran yang digunakan untuk memproses transaksi pembayaran.
16. Ngrok: Alat yang digunakan untuk membuat terowongan ke localhost, memungkinkan akses ke aplikasi yang sedang dikembangkan di mesin lokal.
17. Hoppscotch: Alat pengujian API yang digunakan untuk menguji endpoint API.
18. PostgreSQL: Sistem manajemen basis data relasional yang digunakan untuk menyimpan data aplikasi.
19. Latex: Bahasa markup yang digunakan untuk menulis dokumen proyek akhir.
20. Draw.io (VSCode Extension): Ekstensi Visual Studio Code yang digunakan untuk membuat diagram alur.
21. PlantUML: Alat yang digunakan untuk membuat diagram UML, seperti diagram kelas, diagram urutan, dan diagram aktivitas.
22. ClickUp: Alat manajemen proyek yang digunakan untuk mengelola tugas-tugas dalam proyek akhir ini.

3.2.2 Perangkat Keras

Berikut spesifikasi perangkat keras yang digunakan dalam pengembangan aplikasi AtleTiket dan proyek akhir ini:

1. Laptop Thinkpad T480s
 - Prosesor: Intel Core i5-8350U
 - RAM: 16 GB DDR4 2400 MHz
 - Penyimpanan: 256 GB SSD NVMe M.2

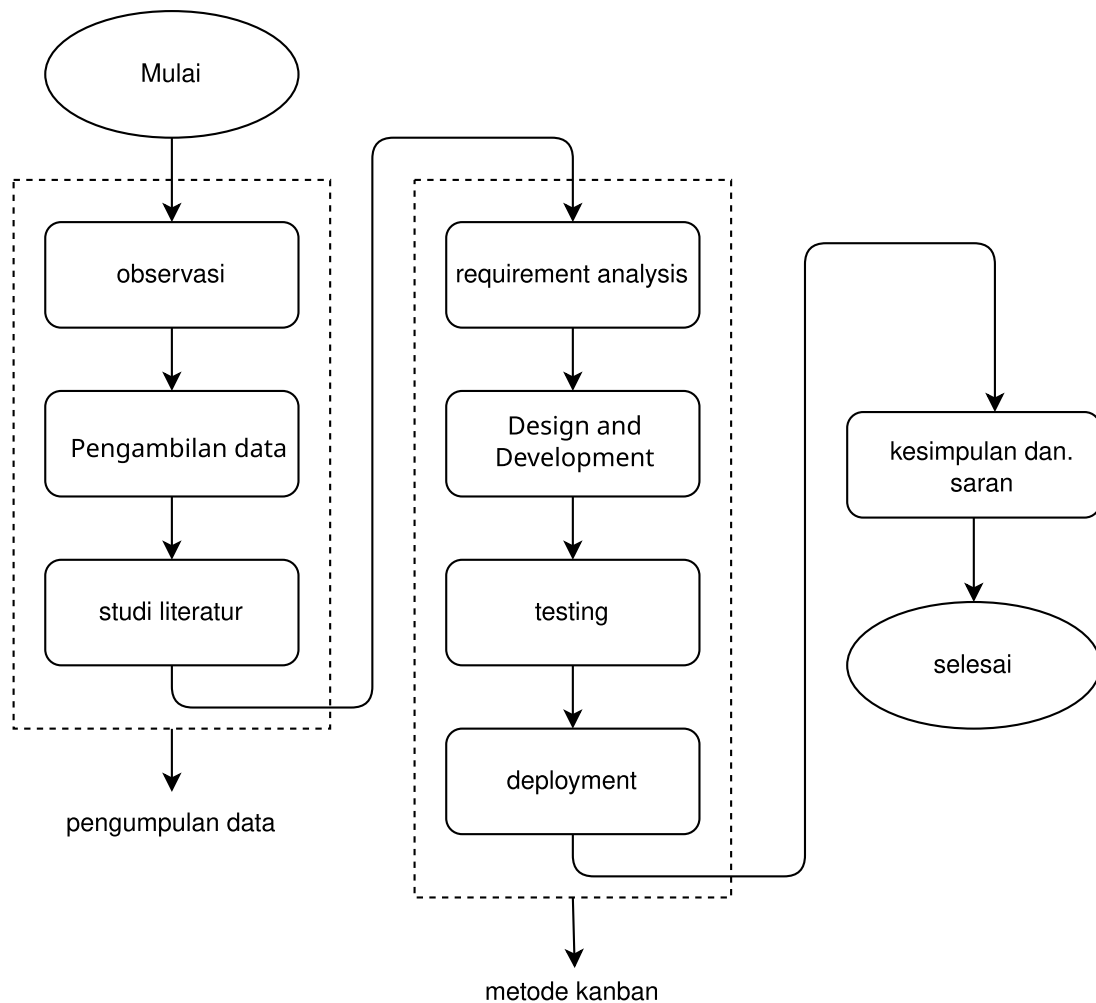
- Kartu Grafis: Intel UHD Graphics
- Sistem Operasi: Ubuntu 24.02 LTS

2. Smartphone Infinix Hot 9

- Prosesor: MediaTek Helio A25
- RAM: 4 GB
- Penyimpanan: 128 GB
- Sistem Operasi: Android 10
- Browser: Google Chrome
- Resolusi Layar: 720 x 1600 piksel
- Ukuran Layar: 6.6 inci

3.3 Tahapan Proyek Akhir

Tahapan dalam proyek akhir ini dibagi menjadi beberapa tahapan yang harus dilalui oleh penulis yang terdiri dari 3 tahapan utama, yaitu tahap pengumpulan data, tahap pengembangan aplikasi dengan metode kanban, dan di akhir yaitu kesimpulan dan saran. Berikut adalah tahapan-tahapan tersebut pada gambar 3.1:



Gambar 3.1 Diagram Tahapan Proyek Akhir

Berikut uraian lebih lengkap mengenai tahapan pada gambar 3.1:

1. Tahap Pengumpulan Data

Tahap ini merupakan tahap awal yang dilakukan oleh penulis untuk mengumpulkan data-data yang dibutuhkan dalam proyek akhir ini, yaitu dengan survei langsung ke hotel kapsul Tamu.id yang sudah ada. Penulis melakukan survei untuk mendapatkan data-data yang dibutuhkan dalam proyek akhir ini, seperti data hotel, fasilitas, jenis pod, dan pod. Penulis juga melakukan wawancara dengan pihak hotel kapsul Tamu.id untuk mendapatkan informasi lebih lanjut mengenai sistem yang sudah ada.

2. Tahap Pengembangan Aplikasi dengan Metode Kanban

Tahap ini merupakan tahap pengembangan aplikasi Tamu.id yang dilakukan oleh penulis dengan menggunakan metode Kanban. Penulis melakukan pengembangan aplikasi dengan menggunakan metode Kanban untuk mempermudah dalam mengelola proyek akhir ini. Penulis juga menggunakan alat bantu seperti ClickUp untuk mengelola proyek akhir ini. Dalam tahap ini, penulis melakukan pengembangan aplikasi dengan

menggunakan bahasa pemrograman Python dan JavaScript. Penulis juga menggunakan framework Flask untuk membangun backend aplikasi dan VueJS untuk membangun frontend aplikasi. Penulis juga menggunakan Tailwind CSS untuk mendesain antarmuka pengguna. Penulis juga menggunakan PostgreSQL sebagai sistem manajemen basis data relasional yang digunakan untuk menyimpan data aplikasi.

3. Tahap Kesimpulan dan Saran

Tahap ini merupakan tahap akhir yang dilakukan oleh penulis untuk menyimpulkan hasil dari proyek akhir ini. Penulis juga akan memberikan saran-saran untuk pengembangan aplikasi Tamu.id ke depannya.

3.4 Analisis Kebutuhan Sistem

Pada tahap ini penulis melakukan proses analisis kebutuhan sesuai dengan hasil wawancara dengan *stakeholder*. Tujuan utama dari analisis kebutuhan ini adalah untuk menentukan kebutuhan sistem yang akan dibangun, baik kebutuhan fungsional maupun non-fungsional. Kebutuhan fungsional adalah kebutuhan yang berkaitan dengan fungsi-fungsi yang harus ada dalam sistem, sedangkan kebutuhan non-fungsional adalah kebutuhan yang berkaitan dengan kualitas sistem seperti keamanan, performa, dan lain-lain.

3.4.1 Analisis Pengguna Sistem

Pada tahap ini penulis melakukan analisis terkait pengguna sistem dan *role* yang ada dalam sistem. Pengguna sistem Tamu.id terdiri dari beberapa *role* sebagai berikut:

1. Admin Pusat (SuperAdmin) Superadmin adalah pengguna yang memiliki hak akses penuh terhadap sistem, termasuk mengelola data cabang, admin untuk cabang, fasilitas, kota, voucher multi cabang, verifikasi identitas pengguna, verifikasi pencairan referral pengguna, dan lain-lain.
2. Admin Cabang (Admin) Admin cabang adalah pengguna yang memiliki hak akses untuk mengelola data pada cabang tertentu, seperti mengelola data fasilitas, voucher cabang, jenis pod cabang, pod cabang, dan melakukan aktifitas operasional pada cabang tersebut.
3. Pengunjung (User) Pengunjung adalah pengguna yang dapat mendaftar, membuat akun, dan melakukan pemesanan tiket untuk mengunjungi cabang tertentu. Pengunjung juga dapat melakukan identitas untuk mengakses lebih banyak fitur seperti mendapatkan penghasilan dari referral, mengelola data diri, dan melakukan pencairan referral.

3.4.2 Analisis Kebutuhan Fungsional

Analisis kebutuhan fungsional dilakukan untuk menentukan fungsi-fungsi yang harus ada dalam sistem. Kebutuhan fungsional yang diperlukan dalam pengembangan aplikasi Tamu.id adalah sebagai berikut:

1. Admin Pusat (SuperAdmin)

- (a) SuperAdmin dapat melihat grafik dan statistik global dari seluruh cabang sebagai garis besar informasi keseluruhan sistem Tamu.id untuk membantu pengambilan keputusan.
- (b) Superadmin dapat melakukan *CRUD (Create, Read, Update, Delete)* pada data master Cabang, Kota, *Online Travel Agent (OTA)*, Staff, fasilitas, dan voucher multi cabang.
- (c) Superadmin dapat memberikan ataupun mengedit hak akses pada admin cabang untuk mengelola satu atau beberapa cabang.
- (d) Superadmin dapat melakukan verifikasi identitas pengguna baik itu menerima ataupun menolak verifikasi identitas pengguna dengan alasan yang jelas serta melihat catatan verifikasi identitas yang telah dilakukan.
- (e) SuperAdmin dapat menolak ataupun menerima permintaan pencairan referral pengguna dengan serta melihat catatan pencairan referral yang telah dilakukan.
- (f) SuperAdmin dapat mengubah data dirinya sendiri seperti nama, email, dan password.

2. Admin Cabang (Admin)

- (a) Admin dapat melihat grafik dan statistik cabang untuk membantu pengambilan keputusan.
- (b) Admin dapat mengelola data cabangnya sendiri seperti mengubah gambar, nama, alamat, dan informasi lainnya.
- (c) Admin dapat mengelola data admin untuk cabang tersebut, termasuk menambah, mengubah, dan menghapus admin cabang.
- (d) Admin dapat mengaktifkan atau menonaktifkan *Online Travel Agent (OTA)* yang bekerja sama dengan cabang tersebut.
- (e) Admin dapat melakukan *CRUD (Create, Read, Update, Delete)* pada data master jenis pod cabang, pod cabang, voucher cabang.
- (f) Admin dapat melihat dan mengelola review dari pengguna yang telah mengunjungi cabang tersebut.
- (g) Admin dapat melakukan *CRUD (Create, Read, Update, Delete)* pada pengaturan harga dinamis untuk jenis pod pada cabang tersebut dengan rentang waktu tertentu.
- (h) Admin dapat menginputkan data Transaksi Reservasi via *Online Travel Agent (OTA)* yang bekerja sama dengan cabang tersebut untuk kemudian nantinya diclaim oleh

pengguna yang bersangkutan.

- (i) Admin dapat memindahkan pengunjung(*User*) yang sedang checkin pada pod ke pod lain yang tersedia jika terjadi masalah pada pod yang sedang digunakan serta melihat catatan pemindahan pod yang telah dilakukan.
- (j) Admin dapat melakukan pembersihan pada pod yang telah digunakan oleh pengunjung untuk memastikan kebersihan dan kenyamanan pod serta melihat catatan pembersihan pod yang telah dilakukan.
- (k) Admin dapat melakukan *monitoring dan controlling* pada pod cabangnya, termasuk melihat status pod dan mengubah status pod jika diperlukan.
- (l) Admin dapat mengubah data dirinya sendiri seperti nama, email, dan password.

3. Pengunjung (*User*)

- (a) Pengunjung dapat mendaftar dan membuat akun untuk mengakses fitur-fitur yang ada di aplikasi Tamu.id via Email atau Google OAuth2.
- (b) Pengunjung dapat melakukan verifikasi identitas untuk mendapatkan akses lebih banyak fitur seperti referral, pencairan referral, dan lain-lain.
- (c) Pengunjung dapat melakukan pencairan referral yang telah didapatkan dari pengguna lain dengan minimal saldo referral yang telah ditentukan.
- (d) Pengunjung dapat melakukan pencarian dengan memilih cabang dan menetapkan tanggal checkin dan checkout untuk melihat ketersediaan pod pada cabang tersebut.
- (e) Pengunjung dapat melihat detail cabang, jenis pod, dan fasilitas yang tersedia pada cabang tersebut.
- (f) Pengunjung dapat melakukan pemesanan tiket dengan memilih jenis pod, tanggal checkin, dan checkout, serta melakukan pembayaran melalui QRIS.
- (g) Pengunjung dapat memasukkan kode voucher yang valid dan juga kode referral yang valid untuk mendapatkan potongan harga pada pemesanan tiket.
- (h) Pengunjung yang telah melakukan reservasi via *Online Travel Agent (OTA)* dapat mengsinkronisasi data reservasi tersebut dengan memasukkan kode reservasi yang diberikan oleh OTA.
- (i) Pengunjung dapat melakukan checkin pada tanggal dan waktu yang telah ditentukan dan memilih pod yang tersedia untuk digunakan sesuai dengan jenis pod yang telah dipesan.

- (j) Pengunjung yang telah melakukan checkin dapat mengakses pod dengan menggunakan QR Code yang dihasilkan oleh sistem.
- (k) Pengunjung dapat melakukan checkout pada pod yang telah digunakan dan sistem akan otomatis mengubah status pod menjadi dalam pembersihan.
- (l) Pengunjung dapat melihat riwayat transaksi dan detail transaksi yang telah dilakukan,
- (m) Pengunjung dapat memberikan ulasan dan rating terhadap cabang yang telah dikunjungi.
- (n) Pengunjung dapat mengelola data diri seperti nama, username, email, nomor telepon, dan password.
- (o) Pengunjung dapat melihat dan mengelola saldo referral yang dimiliki, termasuk melakukan permintaan pencairan saldo referral.
- (p) Pengunjung dapat melihat promo yang sedang berlaku.

3.4.3 Analisis Kebutuhan Non-Fungsional

Pada bagian ini analisis kebutuhan Non Fungsional dilakukan untuk menentukan kebutuhan sistem yang berkaitan dengan kualitas sistem seperti keamanan, performa, dan lain-lain. Kebutuhan non-fungsional yang diperlukan dalam pengembangan aplikasi Tamu.id adalah sebagai berikut:

1. Walaupun pod dalam keadaan offline sementara ataupun dalam interval belum mengambil data kunci valid terbaru, pengguna yang baru checkin harus dapat langsung mengakses pod dengan QR Code yang telah dihasilkan sebelumnya.
2. Sistem dapat mengamankan pesanan pengguna yang masih dalam proses pembayaran sehingga tidak terjadinya *double booking* pada pod yang sama serta dapat otomatis menghapus pesanan yang tidak dibayar dalam waktu satu jam untuk menghindari pemborosan kapasitas pod serta sistem harus dapat otomatis mencheckout pengguna yang tidak melakukan checkout sesuai waktu yang telah ditentukan.
3. Sistem dapat berjalan pada peramban (*browser*) modern seperti Google Chrome, Mozilla Firefox, Microsoft Edge, dan Safari serta memiliki tampilan yang responsif pada perangkat seluler dan desktop.

3.4.4 Pengembangan Sistem

Pengembangan sistem Tamu.id dilakukan secara individual, sehingga penulis memilih metode *Kanban* sebagai pendekatan manajemen tugas yang praktis dan efisien. Papan *Kanban* yang digunakan terdiri dari tiga kolom utama, yaitu *To Do*, *In Progress*, dan *Complete*, yang

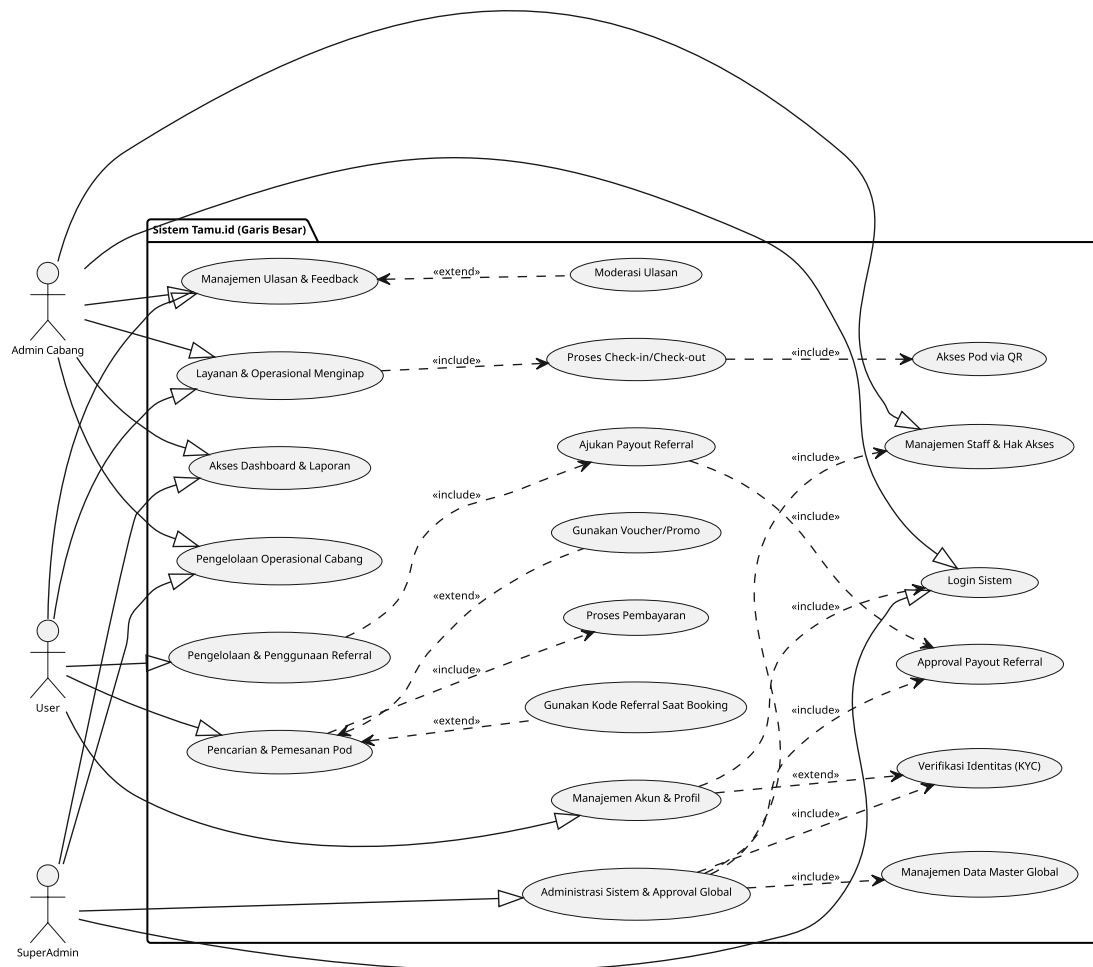
merepresentasikan status dari setiap pekerjaan yang sedang dikerjakan. Pemilihan metode ini didasarkan pada kebutuhan akan fleksibilitas dan kemudahan dalam memantau progres secara mandiri. Setiap aktivitas, mulai dari analisis kebutuhan hingga pengujian, diatur dalam bentuk kartu tugas yang dipindahkan antar kolom sesuai statusnya. Pendekatan ini terbukti efektif dalam membantu penulis menyelesaikan pengembangan secara bertahap dan terstruktur.

3.5 Rancangan Sistem

Pada tahap ini penulis melakukan proses perancangan sistem berdasarkan kebutuhan fungsional dan non-fungsional yang telah dianalisis sebelumnya. Rancangan sistem ini mencakup arsitektur sistem, desain basis data, *Unified Modeling Language (UML)* diagram yang mencakup beberapa diagram, desain antarmuka pengguna, rancangan alur kerja, dan lain-lain yang diperlukan untuk membangun sistem Tamu.id.

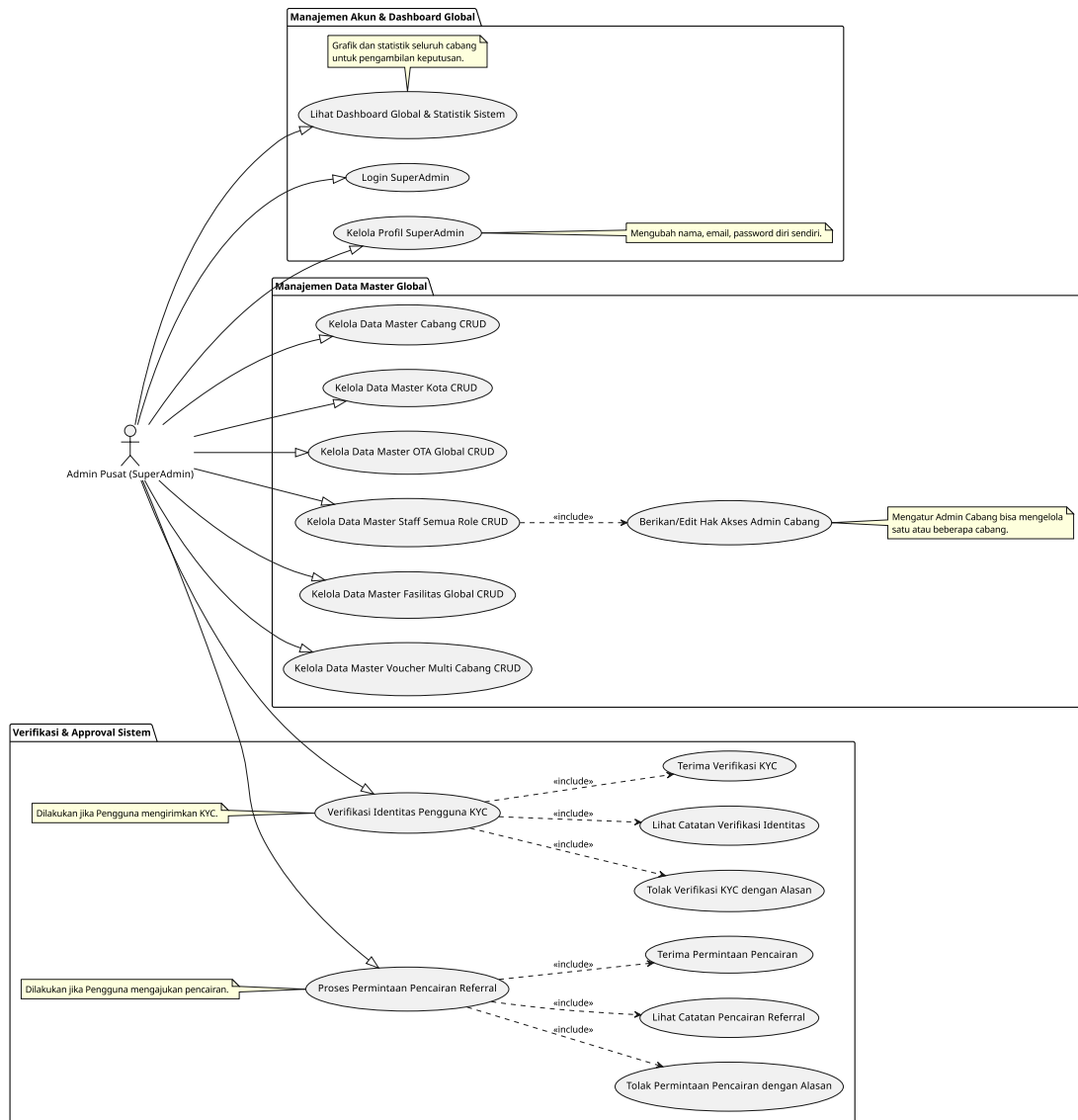
3.5.1 Use Case Diagram

Pada tahap ini *Use Case Diagram* dibuat untuk menggambarkan interaksi antara pengguna sistem dan sistem itu sendiri. Use Case Diagram ini mencakup aktor-aktor yang terlibat, seperti SuperAdmin, Admin Cabang, dan Pengunjung, serta use case yang menggambarkan fungsi-fungsi yang dapat dilakukan oleh masing-masing aktor. *Use Case Diagram* ini akan membantu dalam memahami kebutuhan sistem dan bagaimana pengguna akan berinteraksi dengan sistem sesuai pada gambar 3.2 hingga 3.6.



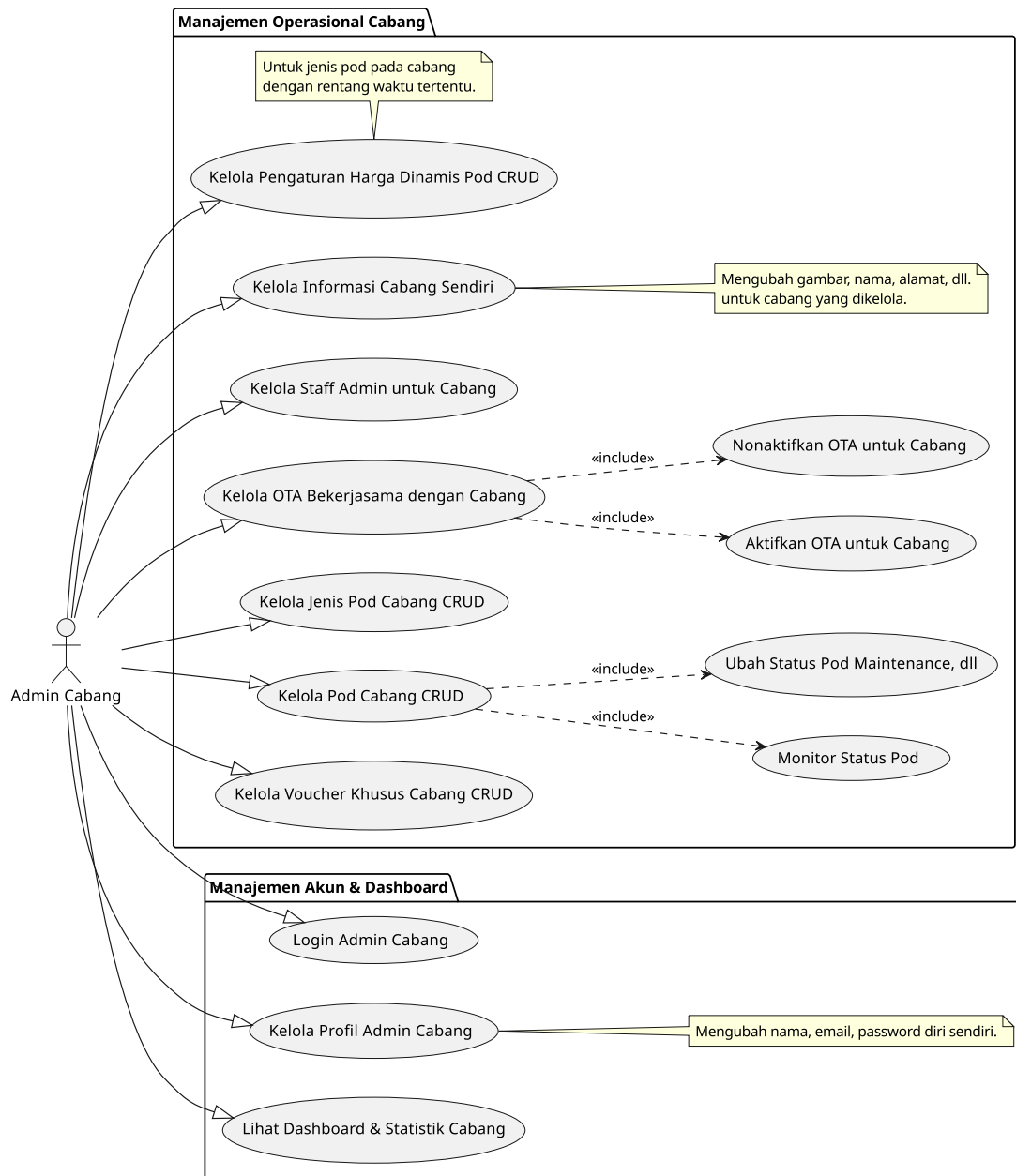
Gambar 3.2 Use Case Diagram Integrasi

Pada gambar 3.2 di atas merupakan gambaran kasar namun keseluruhan dari sistem yang akan dibangun. Terdapat beberapa aktor yang berinteraksi dengan sistem, yaitu SuperAdmin, Admin Cabang, dan User. SuperAdmin bertanggung jawab untuk mengelola seluruh sistem, termasuk pengelolaan Admin Cabang, sedangkan Admin Cabang memiliki tanggung jawab terbatas pada cabang mereka masing-masing. Pengguna atau User dapat melakukan reservasi kamar, memberikan ulasan, dan berpartisipasi dalam program loyalitas.



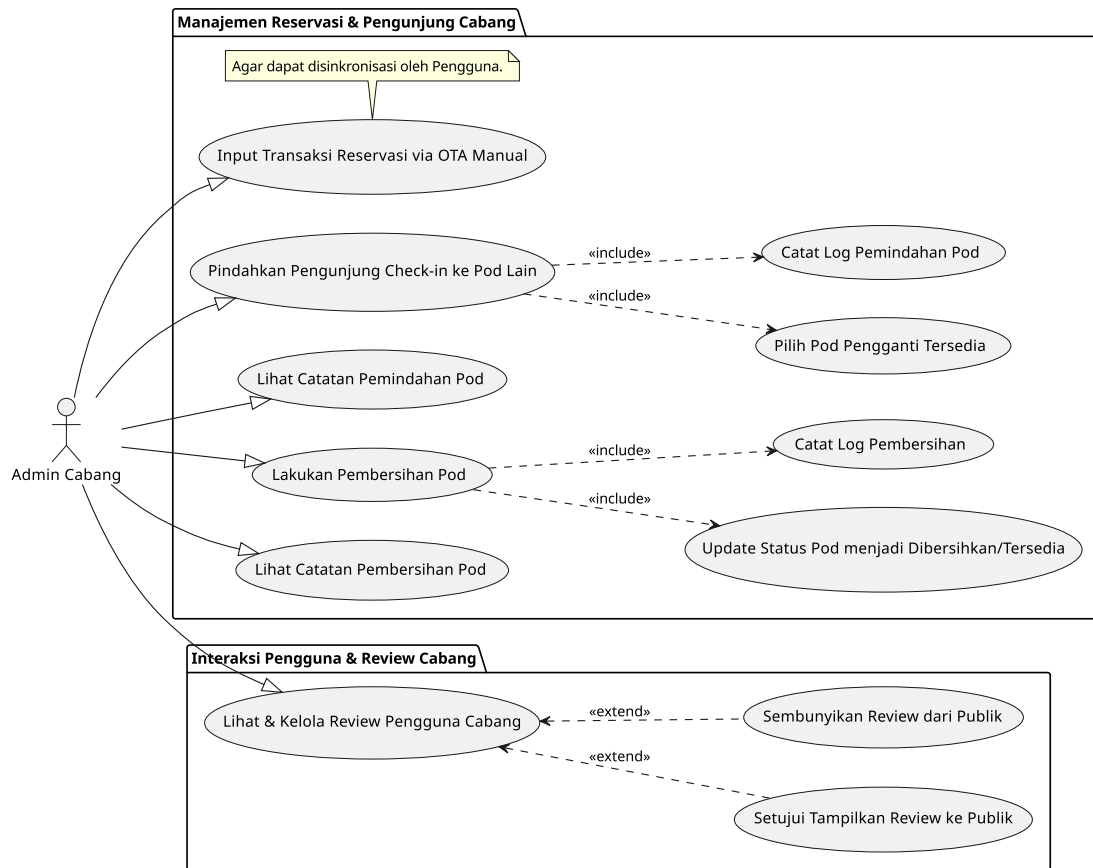
Gambar 3.3 Use Case Diagram SuperAdmin (Admin Pusat)

Pada gambar 3.3, ditampilkan use case diagram untuk SuperAdmin (Admin Pusat). SuperAdmin memiliki akses penuh untuk mengelola seluruh sistem, termasuk pengelolaan Admin Cabang, pengaturan data - data master, dasbor global, serta verifikasi dan sistem persetujuan.



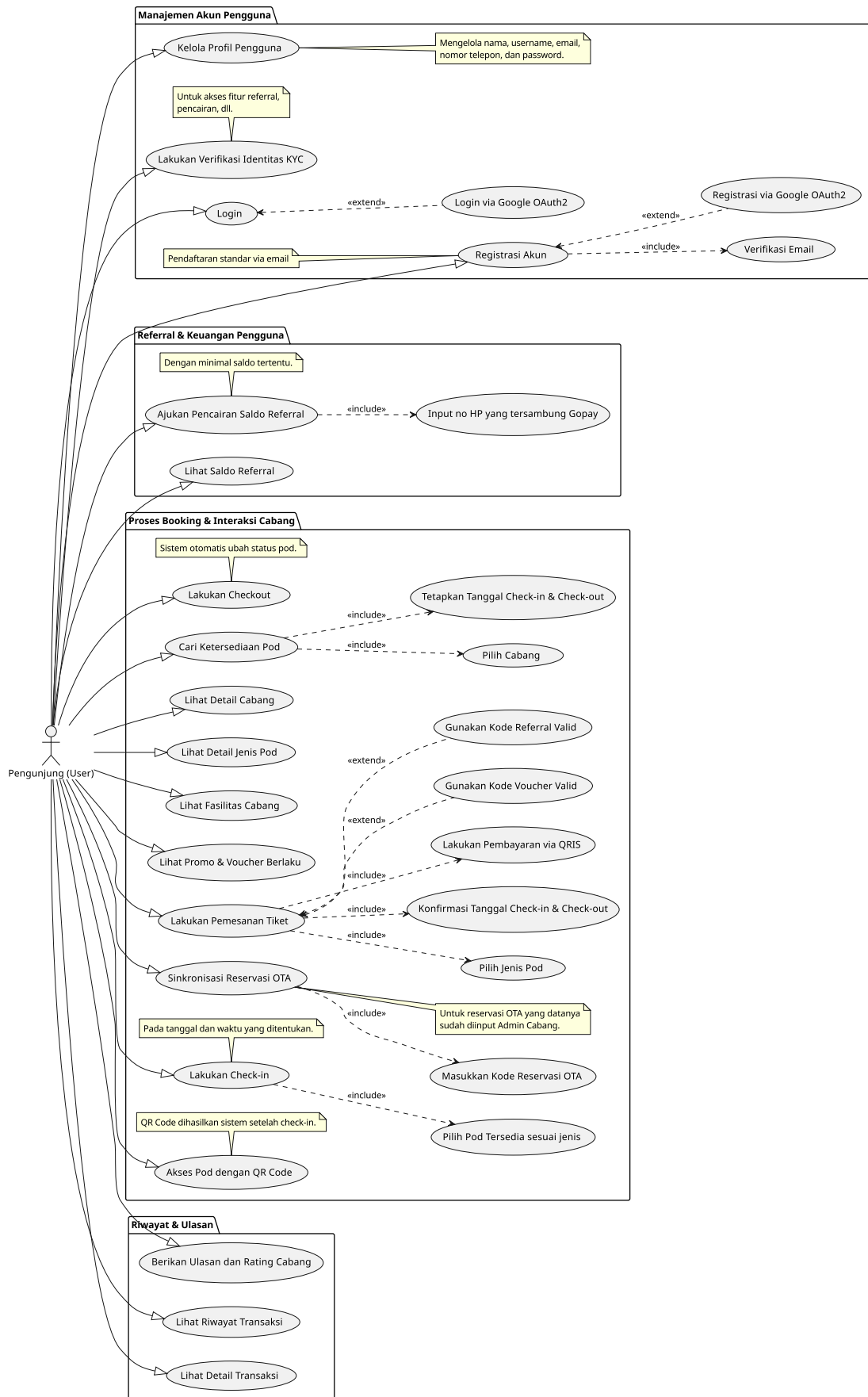
Gambar 3.4 Use Case Diagram Admin (Admin Cabang) 1

Pada gambar 3.4 merupakan case diagram untuk Admin Cabang. Admin Cabang memiliki tanggung jawab untuk mengelola data cabang mereka, termasuk pengelolaan kamar, fasilitas, dan layanan yang tersedia di cabang tersebut. Admin Cabang juga dapat mengelola reservasi yang dilakukan oleh pengguna di cabang mereka.



Gambar 3.5 Use Case Diagram Admin (Admin Cabang) 2

Pada gambar 3.5, ditampilkan use case diagram yang merupakan untuk use case diagram Admin Cabang juga yang dibagi menjadi dua yaitu pada gambar 3.4 dan gambar 3.5 karena jika digabung akan terlalu besar dan tidak terbaca diagramnya, diagram ini mencakup manajemen operasional harian, input data via *Online Travel Agent (OTA)*, dan sistem manajemen *Review* dan *Rating*.



Gambar 3.6 Use Case Diagram User (Pengunjung)

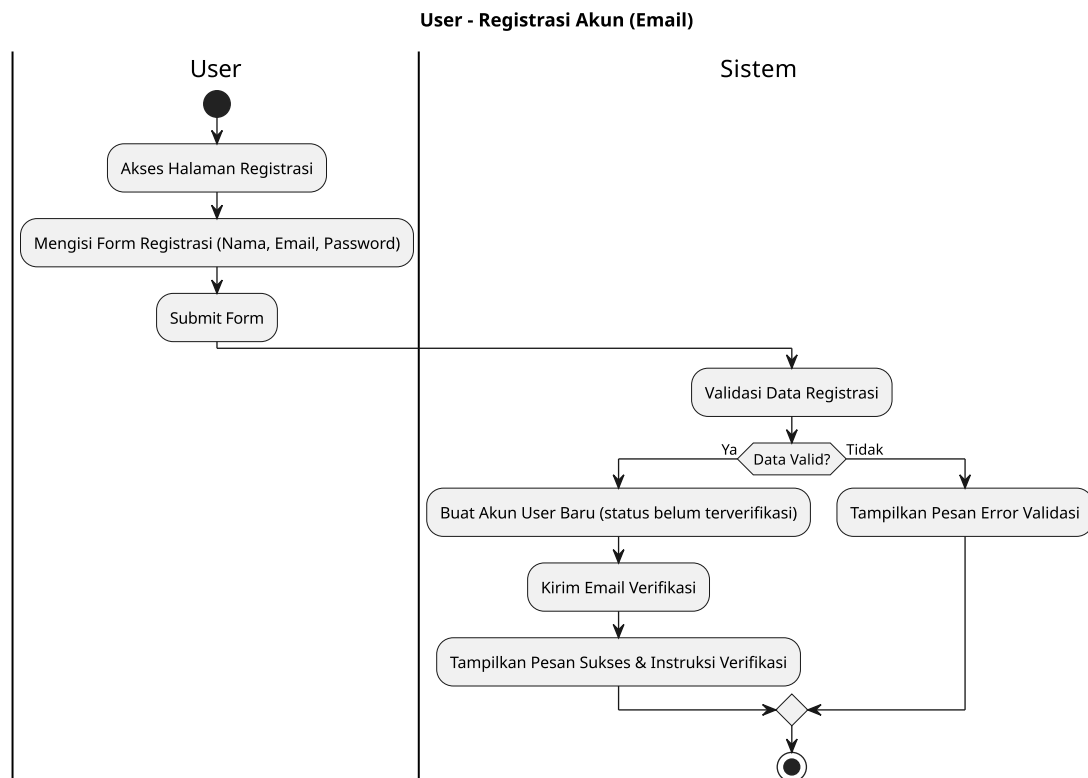
Pada gambar 3.6, ditampilkan use case diagram untuk User (Pengunjung). Pengguna dapat melakukan reservasi kamar, memberikan ulasan, dan berpartisipasi dalam program loyalitas. Diagram ini mencakup fungsi-fungsi yang dapat diakses oleh pengguna, seperti pencarian kamar, pemesanan, dan manajemen akun pengguna.

3.5.2 Activity Diagram

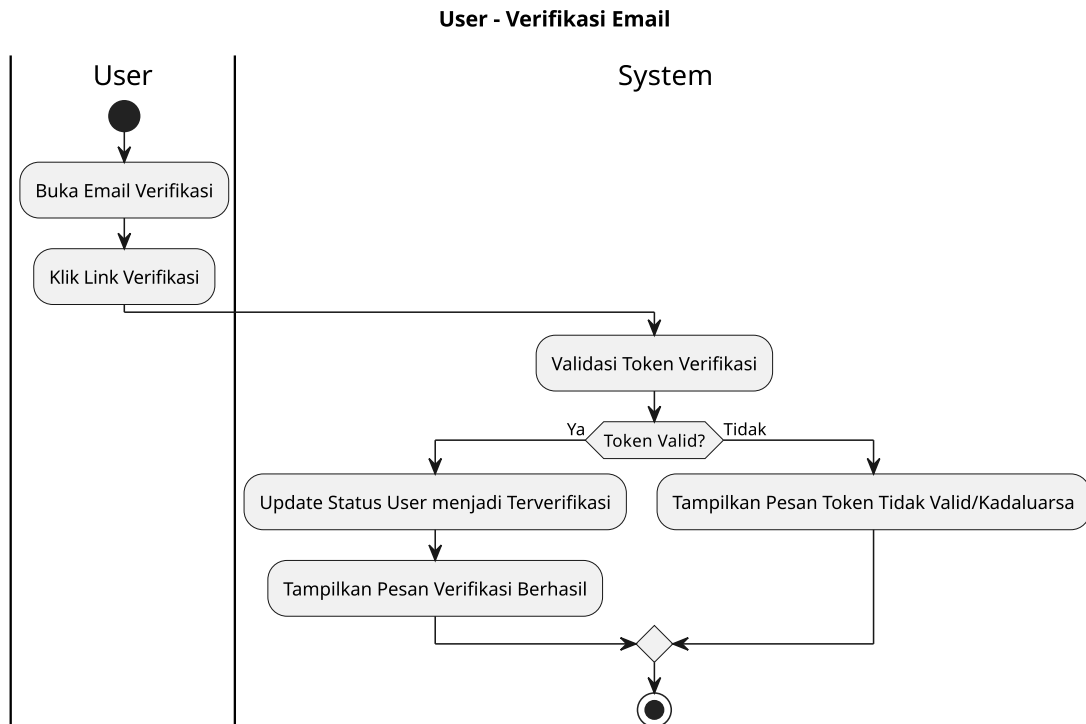
Pada tahap ini, *Activity Diagram* dibuat untuk menggambarkan alur kerja dari berbagai aktivitas yang dilakukan oleh. Berikut adalah beberapa diagram aktivitas yang telah dibuat beserta gambar dan penjelasannya untuk tiap proses yang ada dalam sistem.

A. Diagram Aktivitas Pengguna

Pada gambar 3.7 hingga 3.14, ditampilkan diagram aktivitas yang menggambarkan berbagai proses yang dilakukan oleh pengguna dalam sistem. Diagram ini mencakup proses registrasi, verifikasi email, reservasi kamar, check-in, stay, checkout, verifikasi identitas (KYC), dan payout referral. Setiap diagram memberikan gambaran yang jelas tentang langkah-langkah yang harus diikuti oleh pengguna dalam setiap proses.

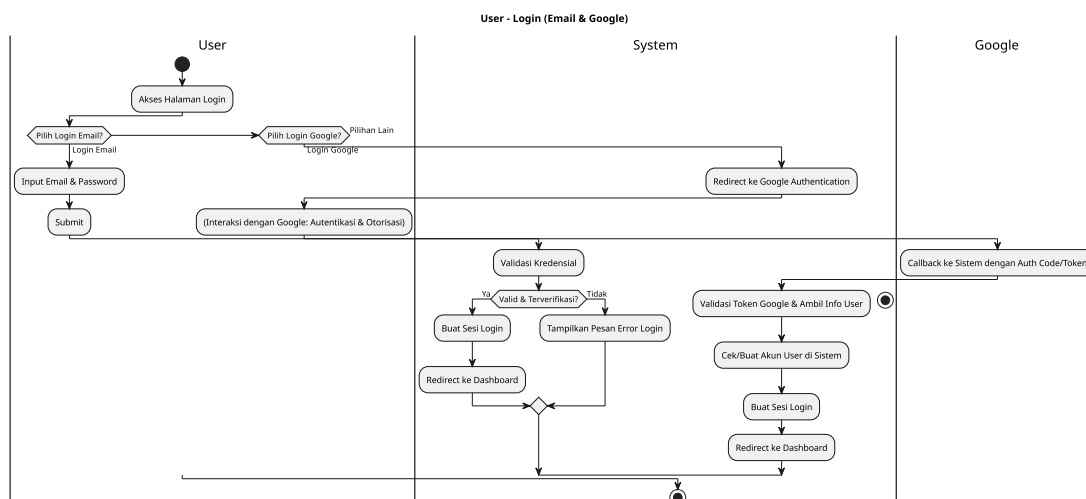


Gambar 3.7 Diagram Aktivitas Pengguna: Registrasi Email



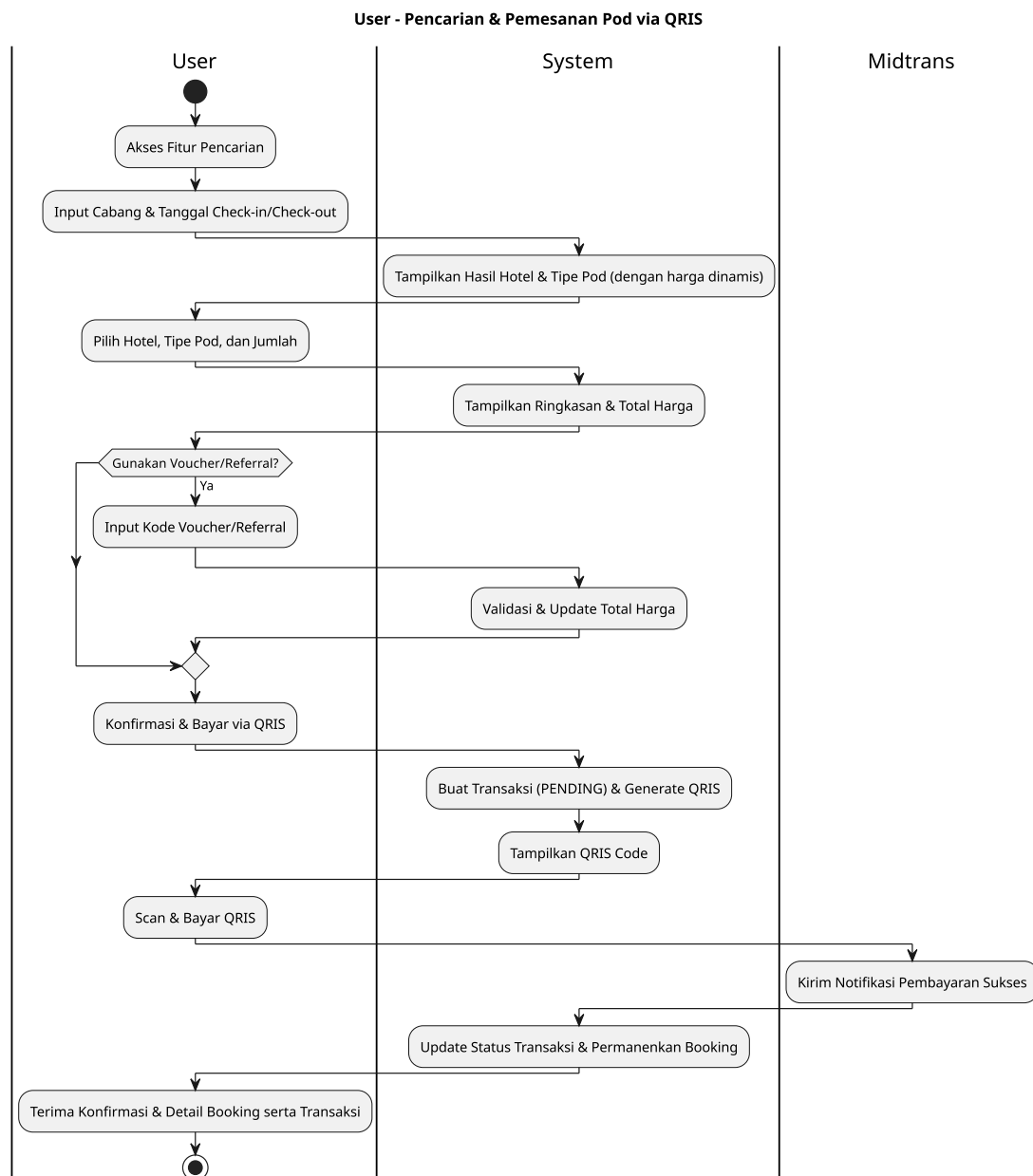
Gambar 3.8 Diagram Aktivitas Pengguna: Verifikasi Email

Pada gambar 3.7 dan 3.8, ditampilkan proses registrasi pengguna dan verifikasi email. Proses ini dimulai dengan pengguna mengisi formulir registrasi, kemudian sistem mengirimkan email verifikasi. Pengguna harus mengklik tautan verifikasi dalam email untuk menyelesaikan proses registrasi.



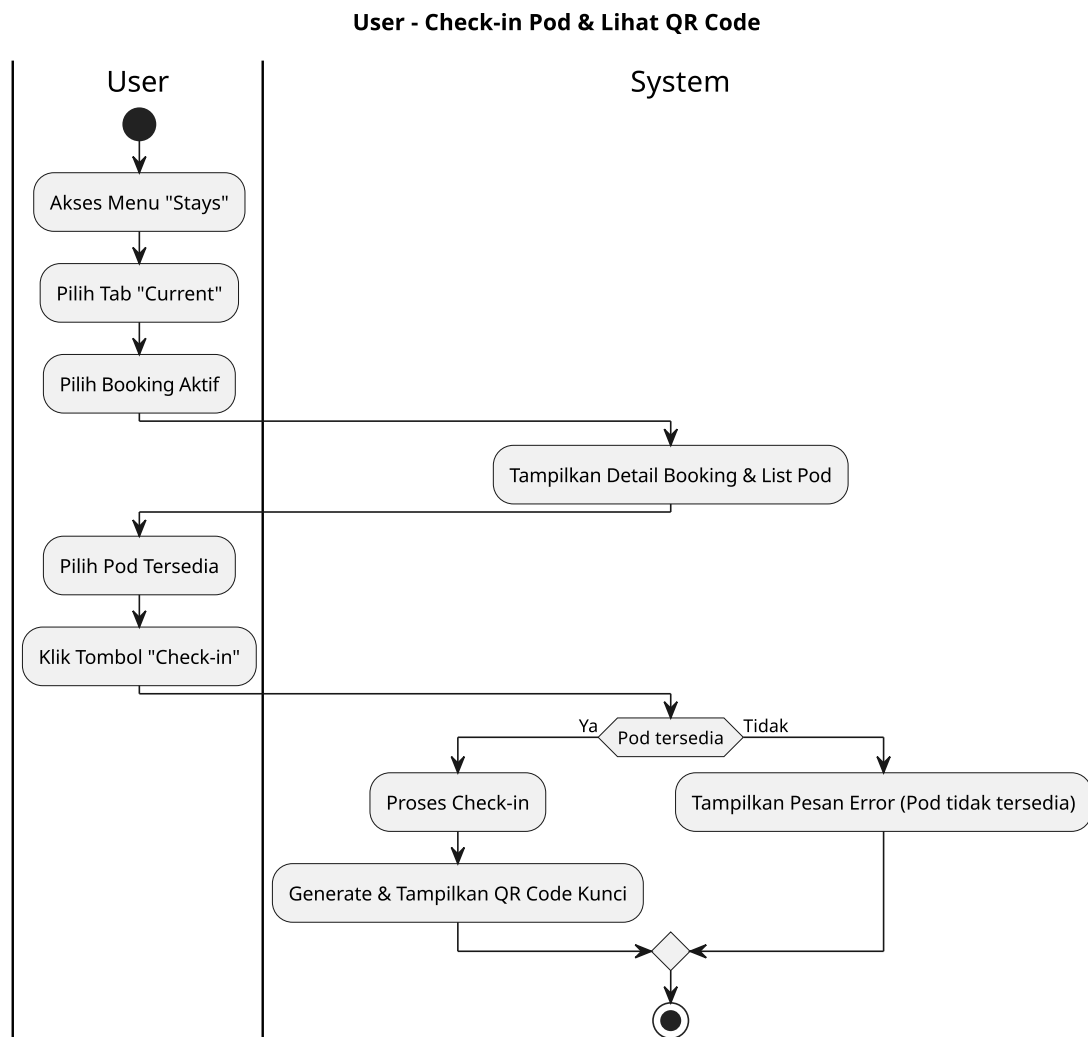
Gambar 3.9 Diagram Aktivitas Pengguna: Registrasi dan Login dengan Google OAuth2

Pada gambar 3.9, ditampilkan proses registrasi dan login menggunakan Google OAuth2. Pengguna dapat memilih untuk mendaftar atau masuk ke sistem menggunakan akun Google mereka, yang memudahkan proses autentikasi.



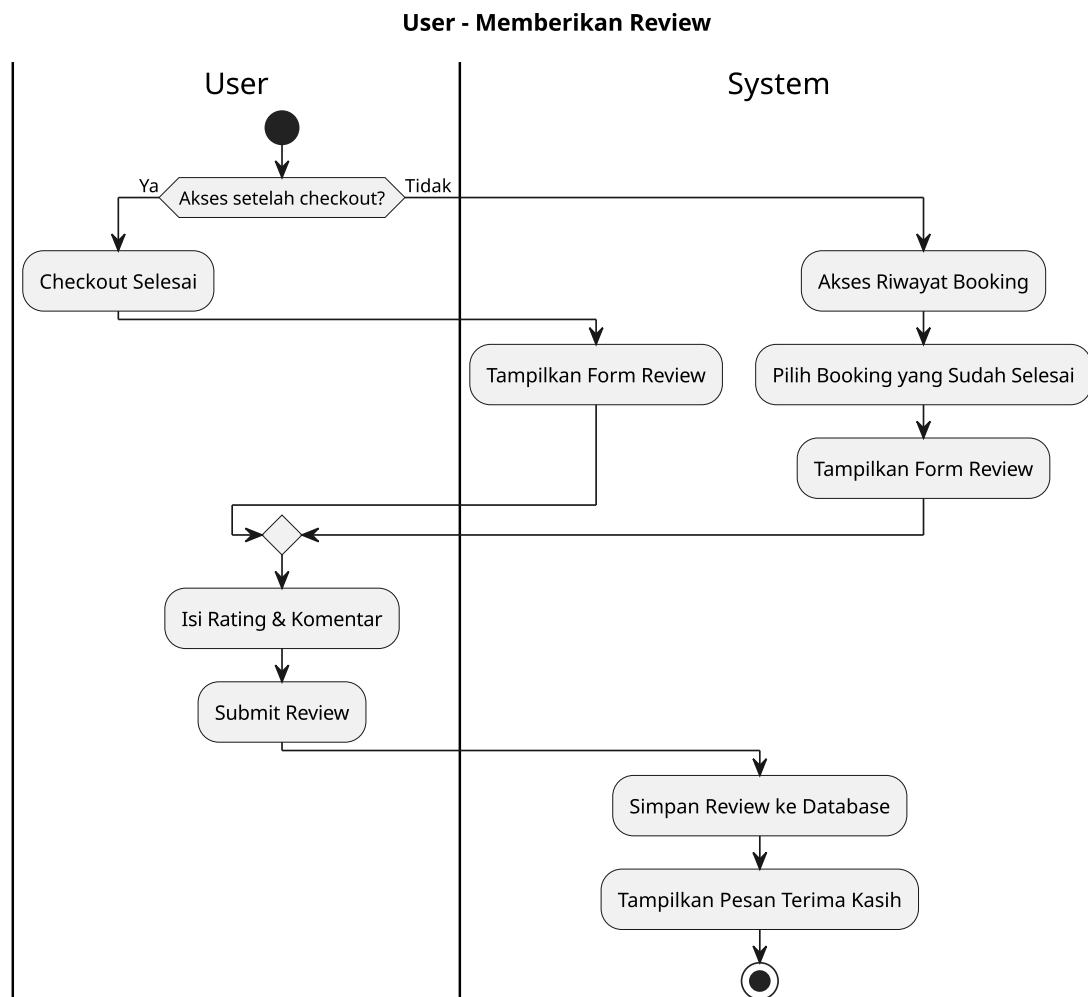
Gambar 3.10 Diagram Aktivitas Pengguna: Reservasi

Pada gambar 3.10, ditampilkan proses reservasi kamar oleh pengguna. Pengguna dapat memilih kamar atau pod yang tersedia, menentukan tanggal check-in dan check-out, serta melakukan pembayaran untuk reservasi tersebut.



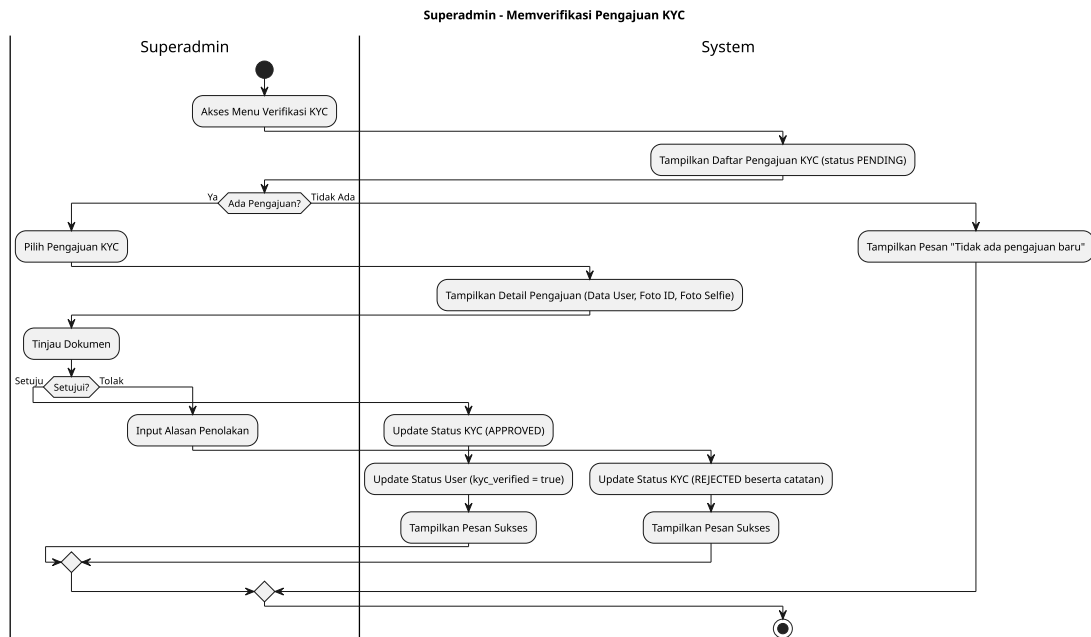
Gambar 3.11 Diagram Aktivitas Pengguna: Checkin dan Stay

Pada gambar 3.11, ditampilkan proses check-in dan stay. Setelah reservasi berhasil, pengguna dapat melakukan check-in ke kamar atau pod yang telah dipesan untuk mendapatkan kunci akses yang berupa kode QR untuk discan pada pintu masuk kamar atau pod agar dapat masuk ke dalamnya.



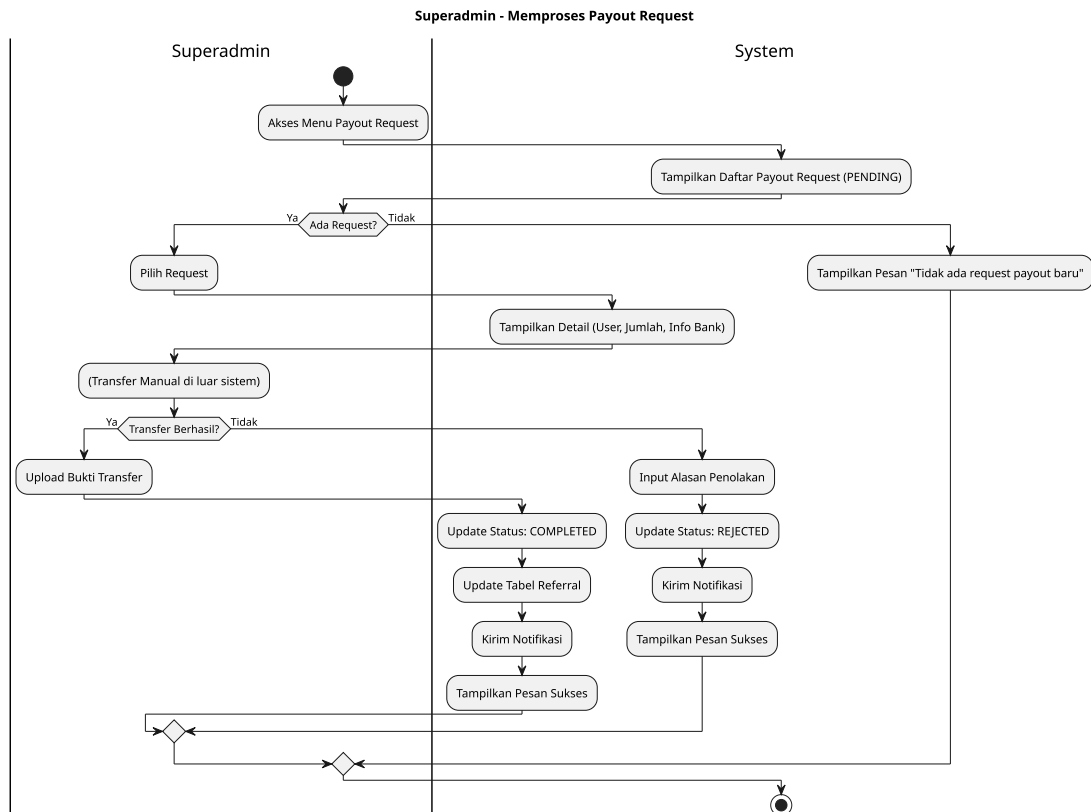
Gambar 3.12 Diagram Aktivitas Pengguna: Checkout dan Review

Pada gambar 3.12, ditampilkan proses checkout dan review. Setelah masa tinggal selesai, pengguna dapat melakukan checkout dari kamar atau pod yang telah dipesan. Pengguna juga dapat memberikan ulasan tentang pengalaman mereka selama menginap.



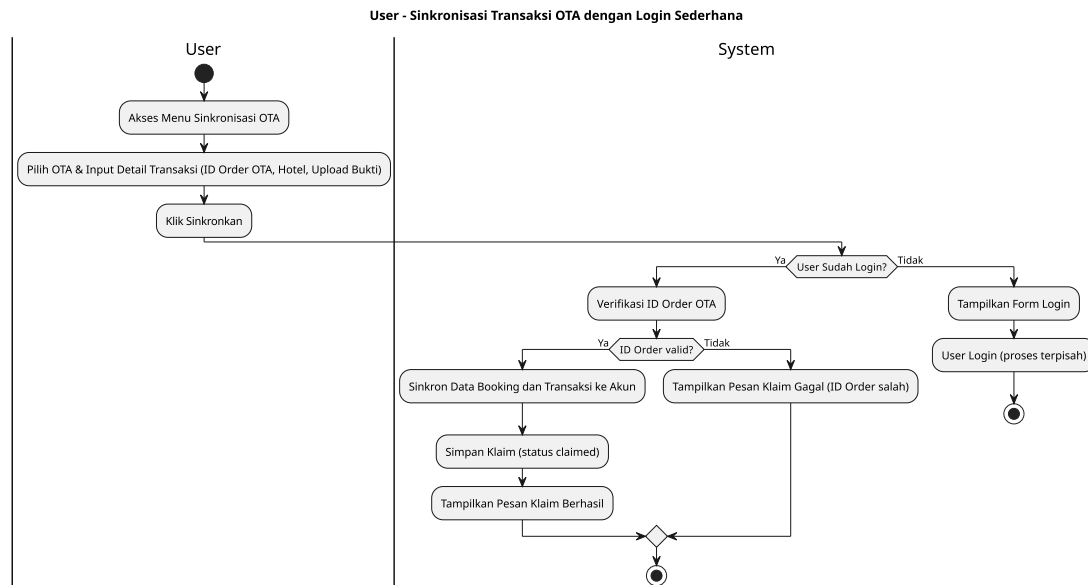
Gambar 3.13 Diagram Aktivitas Pengguna: Verifikasi Identitas (KYC)

Pada gambar 3.13, ditampilkan proses verifikasi identitas (KYC) yang dilakukan oleh pengguna. Proses ini penting untuk memastikan keamanan dan keaslian identitas pengguna dalam sistem. Pengguna diminta untuk mengunggah dokumen identitas yang diperlukan, yang kemudian akan diverifikasi oleh SuperAdmin.



Gambar 3.14 Diagram Aktivitas Pengguna: Payout Referral

Pada gambar 3.14, ditampilkan proses payout referral. Pengguna yang berhasil mereferensikan teman atau orang lain untuk menggunakan layanan dapat menerima payout sebagai imbalan. Proses ini melibatkan verifikasi referral dan pemrosesan pembayaran oleh SuperAdmin.

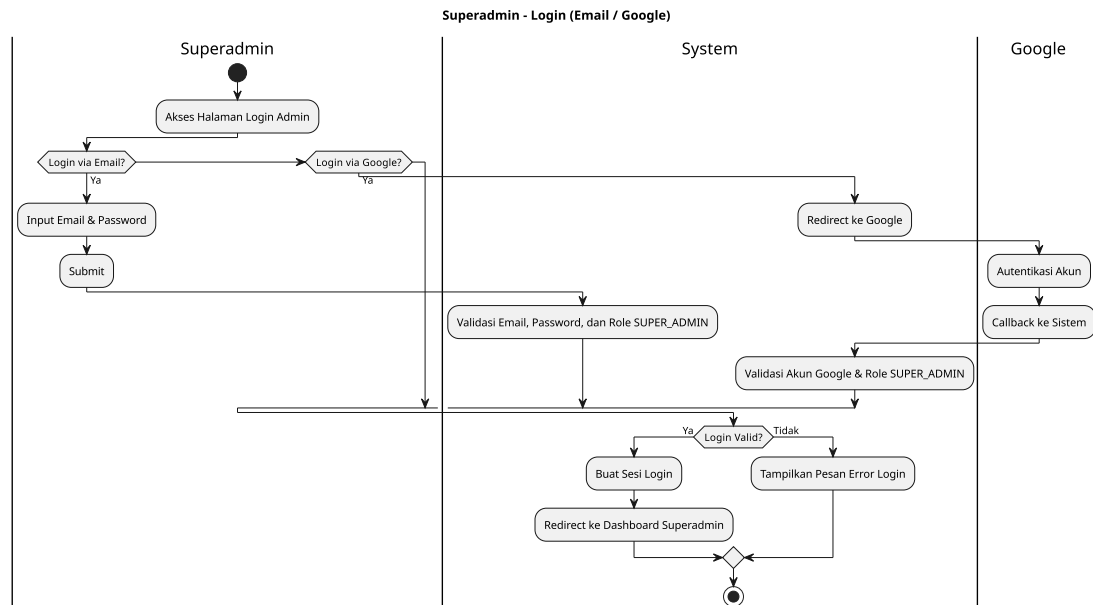


Gambar 3.15 Diagram Aktivitas Pengguna: Sinkronisasi Data Reservasi OTA

Pada gambar 3.15, ditampilkan proses sinkronisasi data reservasi dengan *Online Travel Agent (OTA)*. Proses ini memungkinkan pengguna untuk mengklaim reservasi yang dibuat melalui OTA, sehingga data reservasi dapat terintegrasi dengan sistem utama.

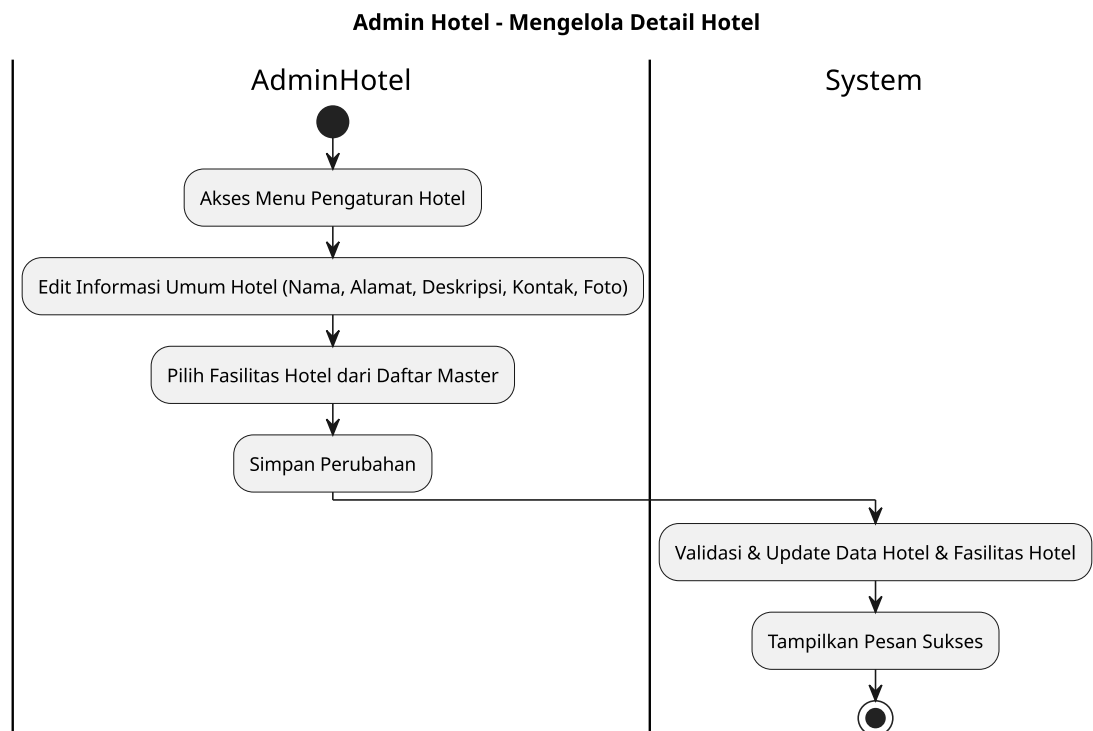
B. Diagram Aktivitas Admin Cabang

Pada gambar 3.16 hingga 3.23, ditampilkan diagram aktivitas yang menggambarkan berbagai proses yang dilakukan oleh Admin Cabang dalam sistem. Diagram ini mencakup proses login, pengelolaan cabang (hotel), jenis pod, pod, moderasi review, pengelolaan voucher cabang, pengelolaan pindah pod, dan pengelolaan pembersihan pod.



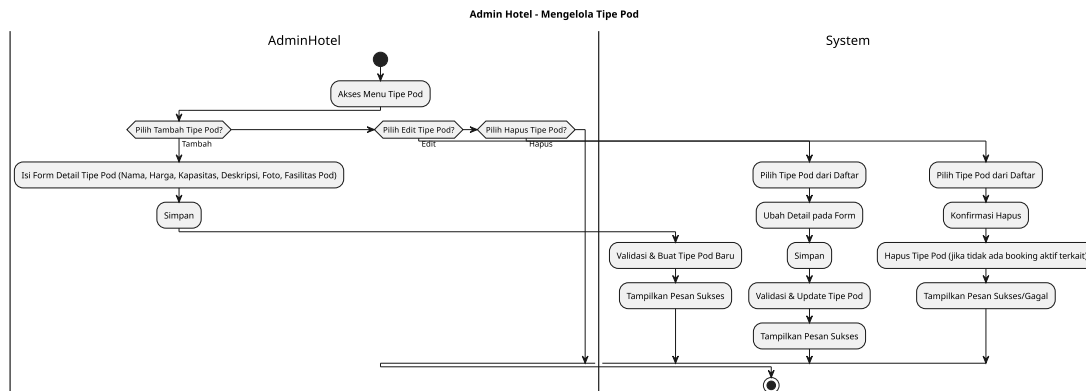
Gambar 3.16 Diagram Aktivitas Admin: Login Email dan Google OAuth2

Pada gambar 3.16, ditampilkan proses login Admin Cabang menggunakan email dan Google OAuth2. Admin Cabang dapat masuk ke sistem untuk mengakses fitur-fitur yang tersedia bagi mereka.



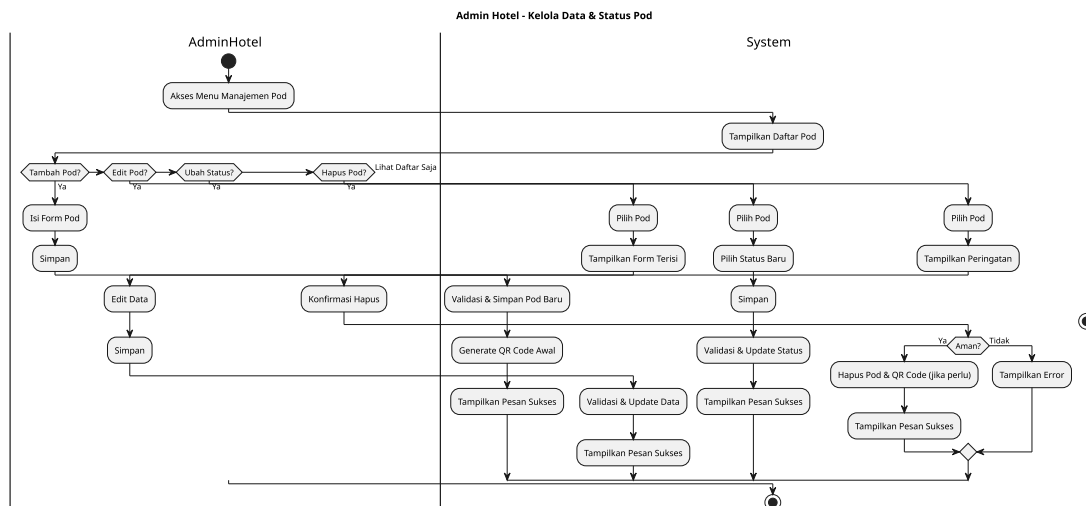
Gambar 3.17 Diagram Aktivitas Admin: Kelola Cabang (Hotel)

Pada gambar 3.17, ditampilkan proses pengelolaan cabang (hotel) oleh Admin Cabang. Admin Cabang dapat menambahkan, mengedit, atau menghapus informasi tentang cabang mereka, termasuk fasilitas yang tersedia.



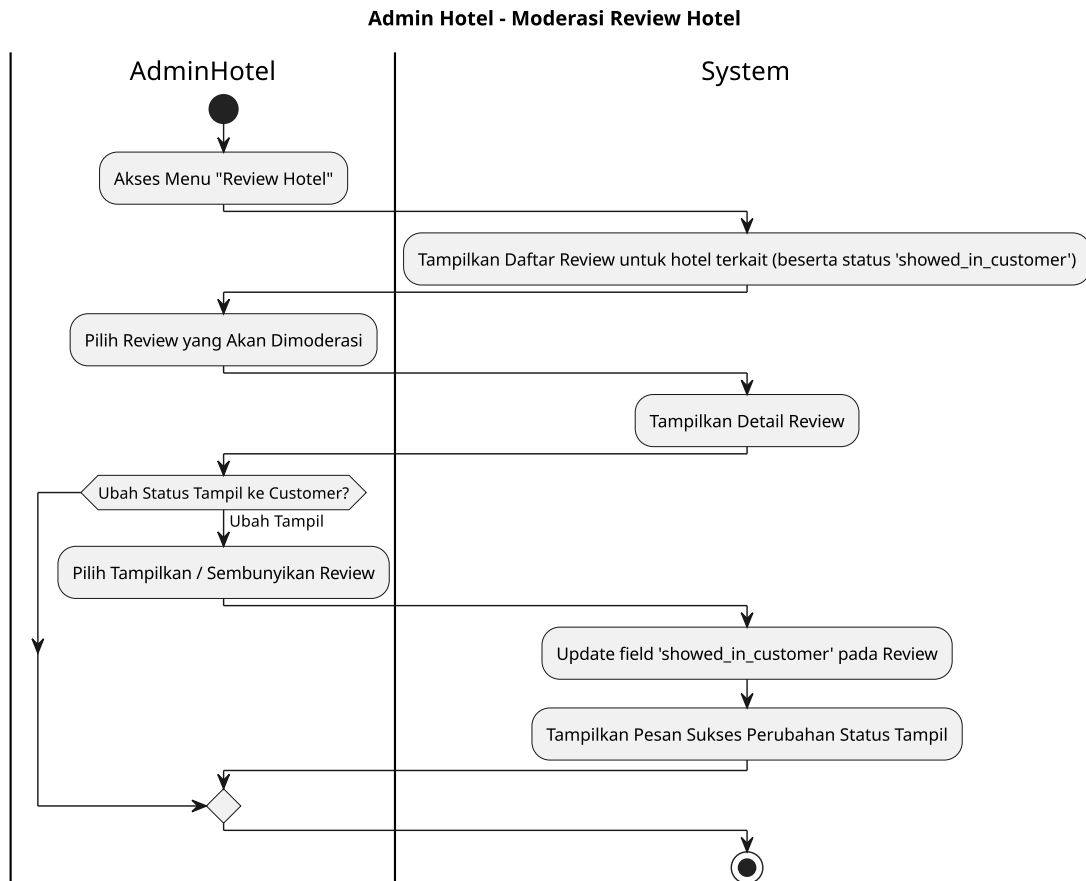
Gambar 3.18 Diagram Aktivitas Admin: Kelola Jenis Pod

Pada gambar 3.18, ditampilkan proses pengelolaan jenis pod oleh Admin Cabang. Admin Cabang dapat menambahkan, mengedit, atau menghapus jenis pod yang tersedia di cabang mereka, termasuk informasi tentang fasilitas dan harga.



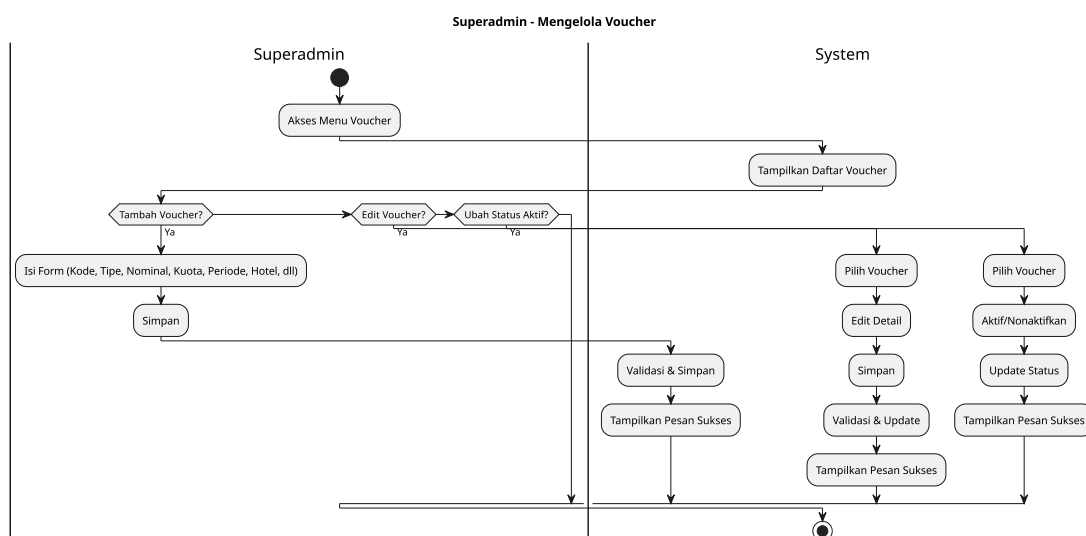
Gambar 3.19 Diagram Aktivitas Admin: Kelola Pod

Setelah menambahkan jenis pod, Admin Cabang dapat mengelola pod yang tersedia di cabang mereka. Pada gambar 3.19, ditampilkan proses pengelolaan pod, termasuk penambahan, pengeditan, dan penghapusan pod yang tersedia untuk reservasi.



Gambar 3.20 Diagram Aktivitas Admin: Moderasi Review

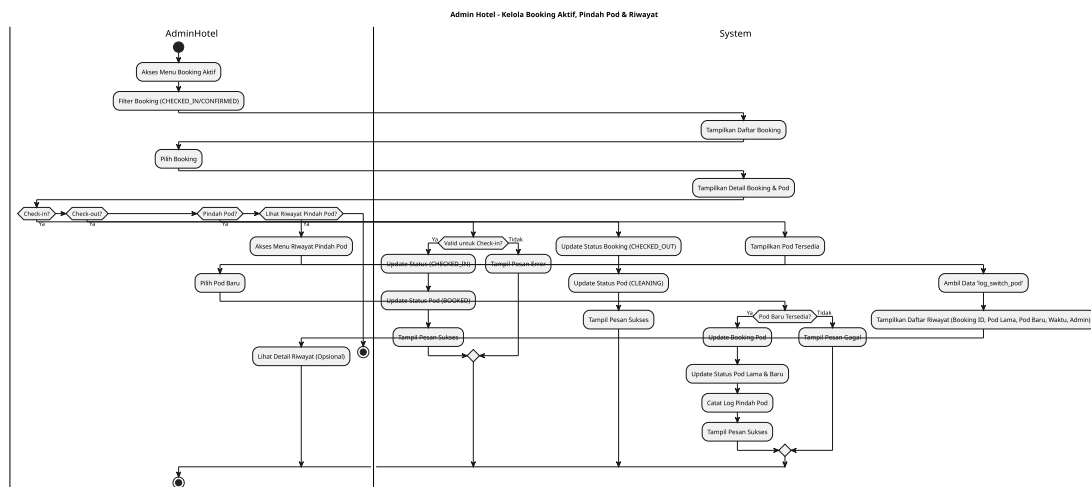
Pada gambar 3.20, ditampilkan proses moderasi review oleh Admin Cabang. Admin Cabang dapat meninjau, menyetujui, atau menolak ulasan yang diberikan oleh pengguna tentang pod atau cabang mereka.



Gambar 3.21 Diagram Aktivitas Admin: Kelola Voucher Cabang

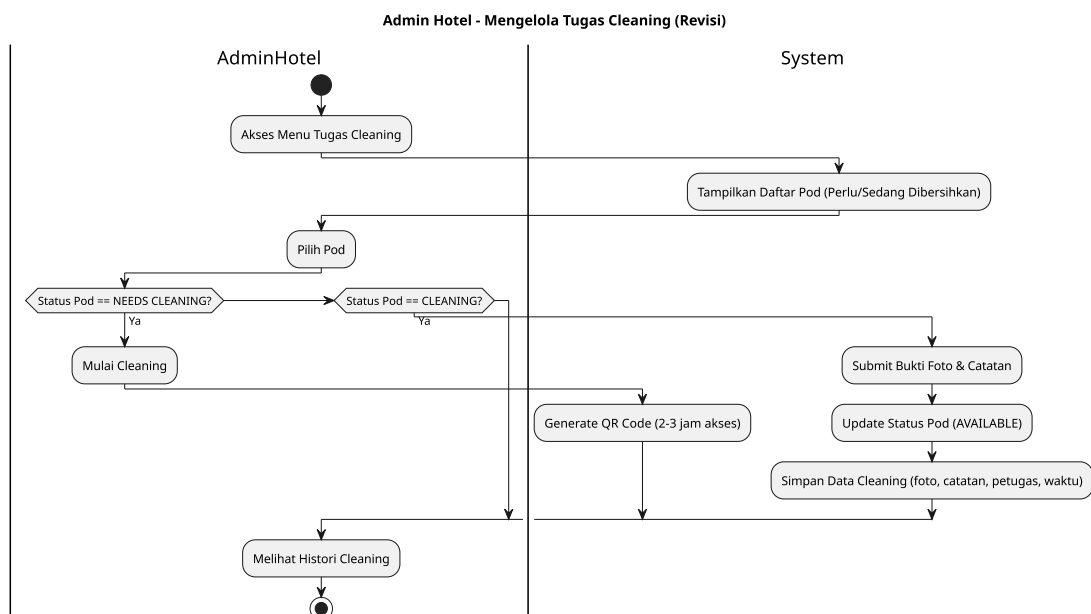
Pada gambar 3.21, ditampilkan proses pengelolaan voucher cabang oleh Admin Cabang.

Admin Cabang dapat membuat, mengedit, atau menghapus voucher yang dapat digunakan oleh pengguna untuk mendapatkan diskon atau penawaran khusus di cabang mereka.



Gambar 3.22 Diagram Aktivitas Admin: Kelola Pindah Pod

Pada gambar 3.22, ditampilkan proses pengelolaan pindah pod oleh Admin Cabang. Admin Cabang dapat mengelola permintaan pindah pod dari pengguna, termasuk memproses permintaan dan memastikan ketersediaan pod yang diminta.

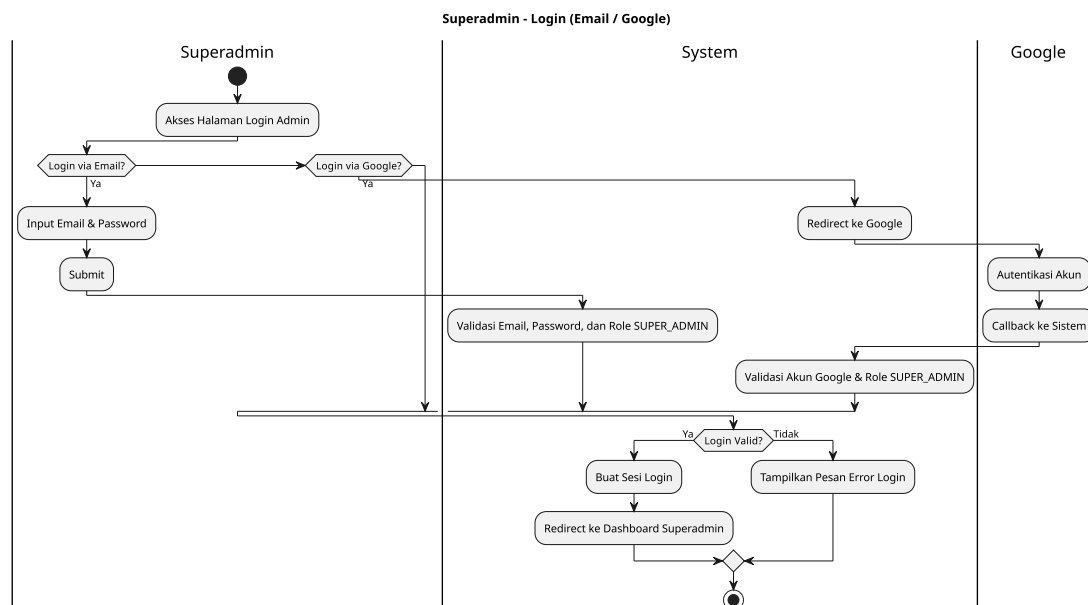


Gambar 3.23 Diagram Aktivitas Admin: Kelola Pembersihan Pod

Pada gambar 3.23, ditampilkan proses pengelolaan pembersihan pod oleh Admin Cabang. Admin Cabang dapat melakukan pembersihan pod setelah pengguna check-out, memastikan bahwa pod siap digunakan oleh pengguna berikutnya.

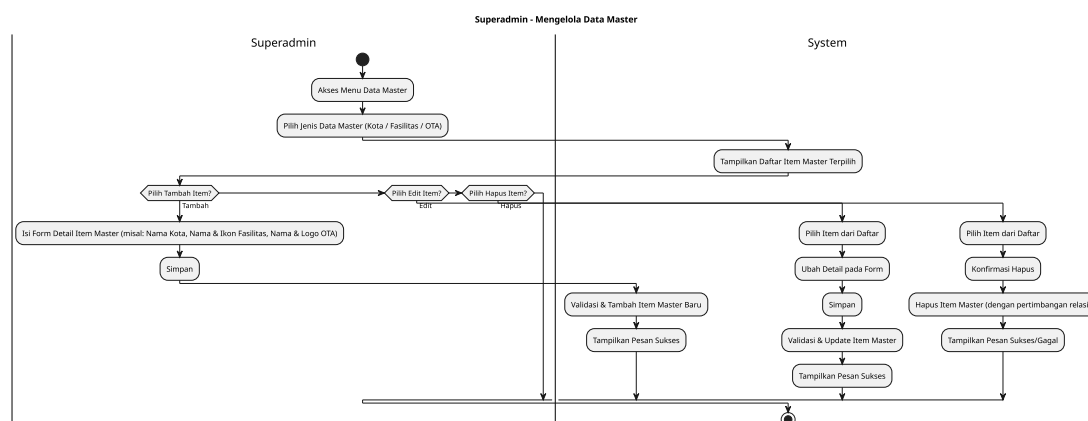
C. Diagram Aktivitas SuperAdmin

Pada gambar 3.24 hingga 3.30, ditampilkan diagram aktivitas yang menggambarkan berbagai proses yang dilakukan oleh SuperAdmin dalam sistem. Diagram ini mencakup proses login, pengelolaan data master, cabang (hotel), staff, verifikasi identitas (KYC), voucher, dan payout request.



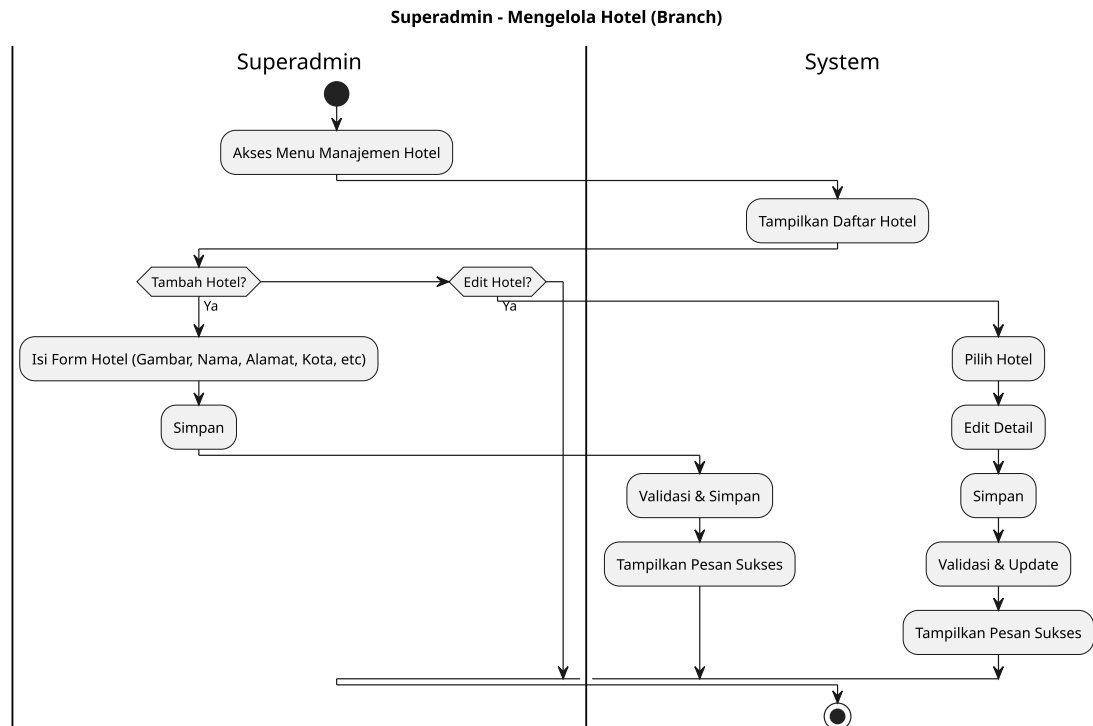
Gambar 3.24 Diagram Aktivitas SuperAdmin: Login Email dan Google OAuth2

Pada gambar 3.24, ditampilkan proses login SuperAdmin menggunakan email dan Google OAuth2. SuperAdmin dapat masuk ke sistem untuk mengakses fitur-fitur yang tersedia bagi mereka.



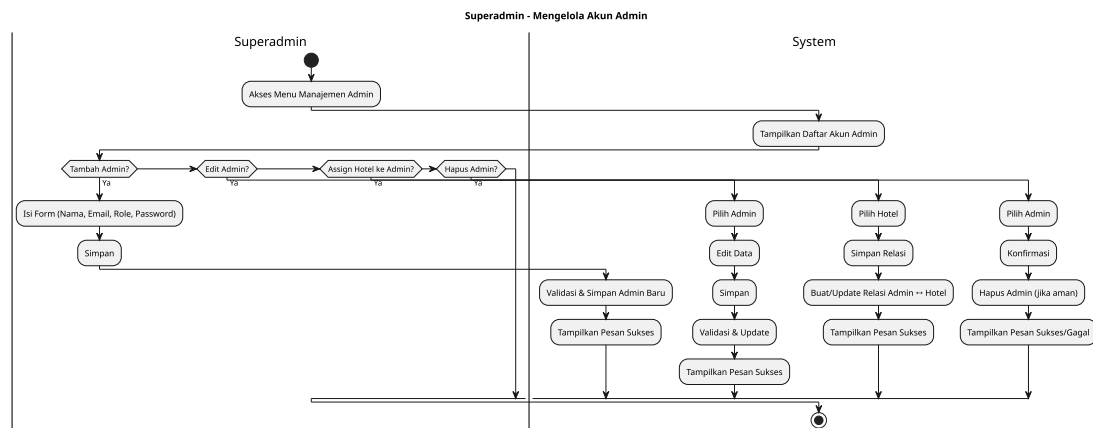
Gambar 3.25 Diagram Aktivitas SuperAdmin: Kelola Data Master

Pada gambar 3.25, ditampilkan proses pengelolaan data master oleh SuperAdmin. SuperAdmin dapat mengelola data master yang mencakup informasi dasar sistem seperti fasilitas, Kota, Cabang, dan Karyawan.



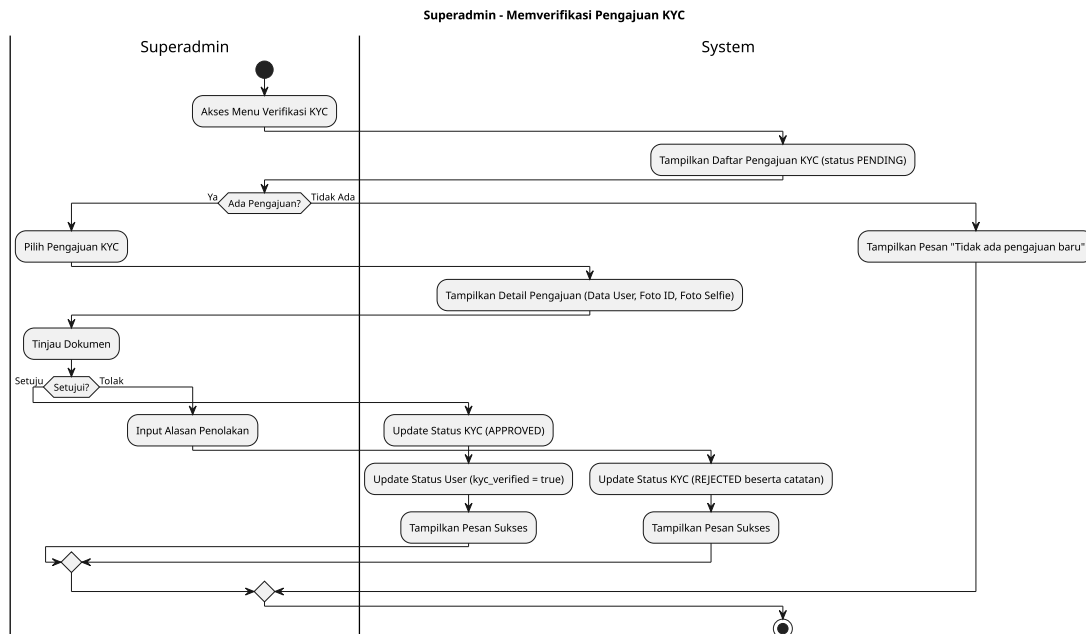
Gambar 3.26 Diagram Aktivitas SuperAdmin: Kelola Data Cabang (Hotel)

Pada gambar 3.26, ditampilkan proses pengelolaan data cabang (hotel) oleh SuperAdmin. SuperAdmin dapat menambahkan, mengedit, atau menghapus informasi tentang cabang yang ada dalam sistem.



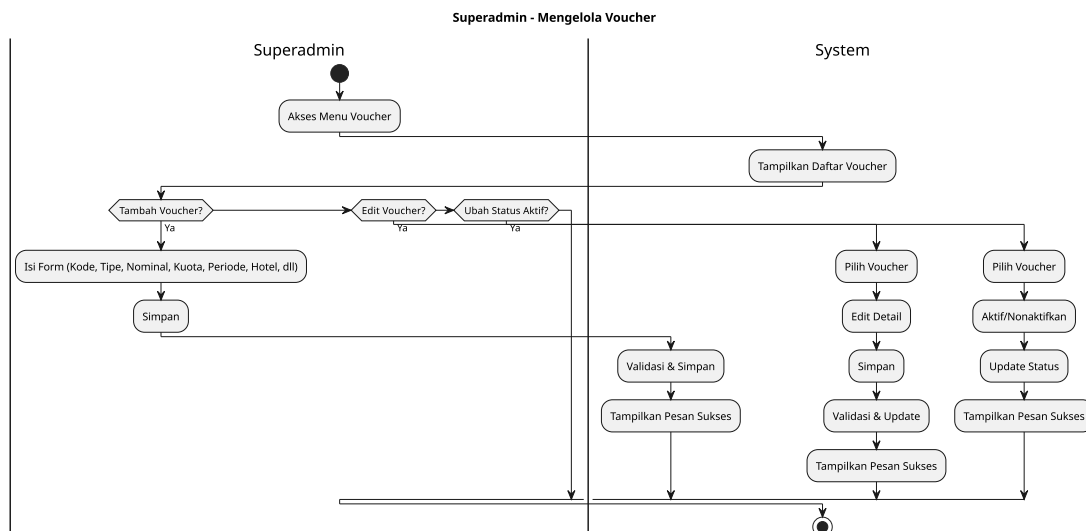
Gambar 3.27 Diagram Aktivitas SuperAdmin: Kelola Data Staff

Pada gambar 3.27, ditampilkan proses pengelolaan data staff oleh SuperAdmin. SuperAdmin dapat menambahkan, mengedit, atau menghapus informasi tentang staff yang bekerja di cabang-cabang yang ada dalam sistem.



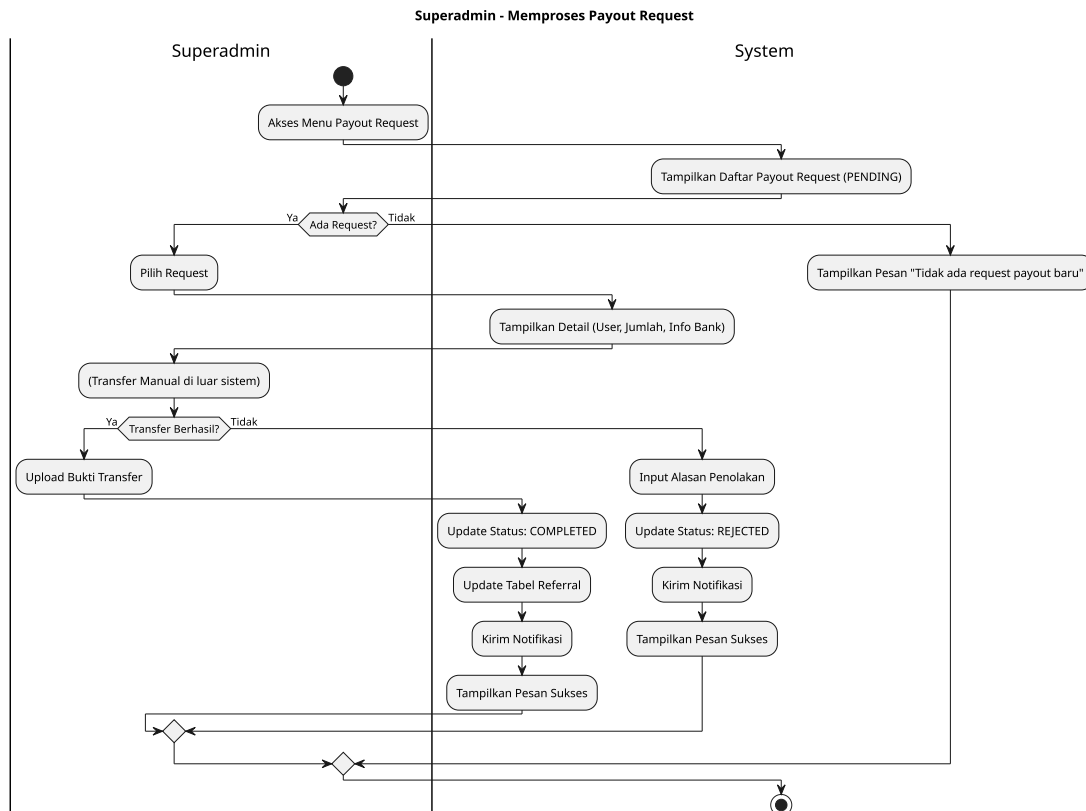
Gambar 3.28 Diagram Aktivitas SuperAdmin: Kelola Data Verifikasi Identitas (KYC)

Pada gambar 3.28, ditampilkan proses pengelolaan data verifikasi identitas (KYC) oleh SuperAdmin. SuperAdmin dapat mengelola proses verifikasi identitas pengguna, termasuk persetujuan atau penolakan dokumen yang diunggah oleh pengguna.



Gambar 3.29 Diagram Aktivitas SuperAdmin: Kelola Data Voucher

Pada gambar 3.29, ditampilkan proses pengelolaan data voucher oleh SuperAdmin. SuperAdmin dapat membuat, mengedit, atau menghapus voucher yang dapat digunakan oleh pengguna untuk mendapatkan diskon atau penawaran khusus baik itu secara global maupun cabang tertentu.

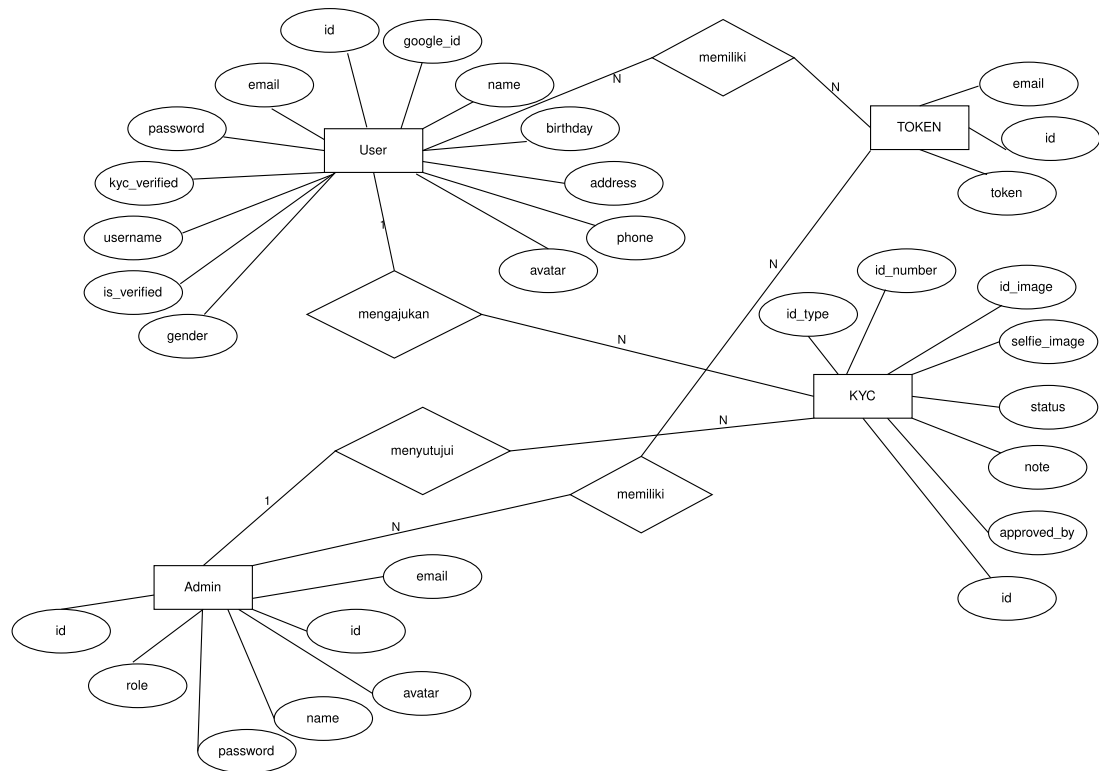


Gambar 3.30 Diagram Aktivitas SuperAdmin: Kelola Payout Request

Pada gambar 3.30, ditampilkan proses pengelolaan payout request oleh SuperAdmin. SuperAdmin dapat memproses permintaan payout dari pengguna yang telah mengajukan payout referral. Proses ini melibatkan verifikasi dan persetujuan payout sebelum dana dikirim ke pengguna.

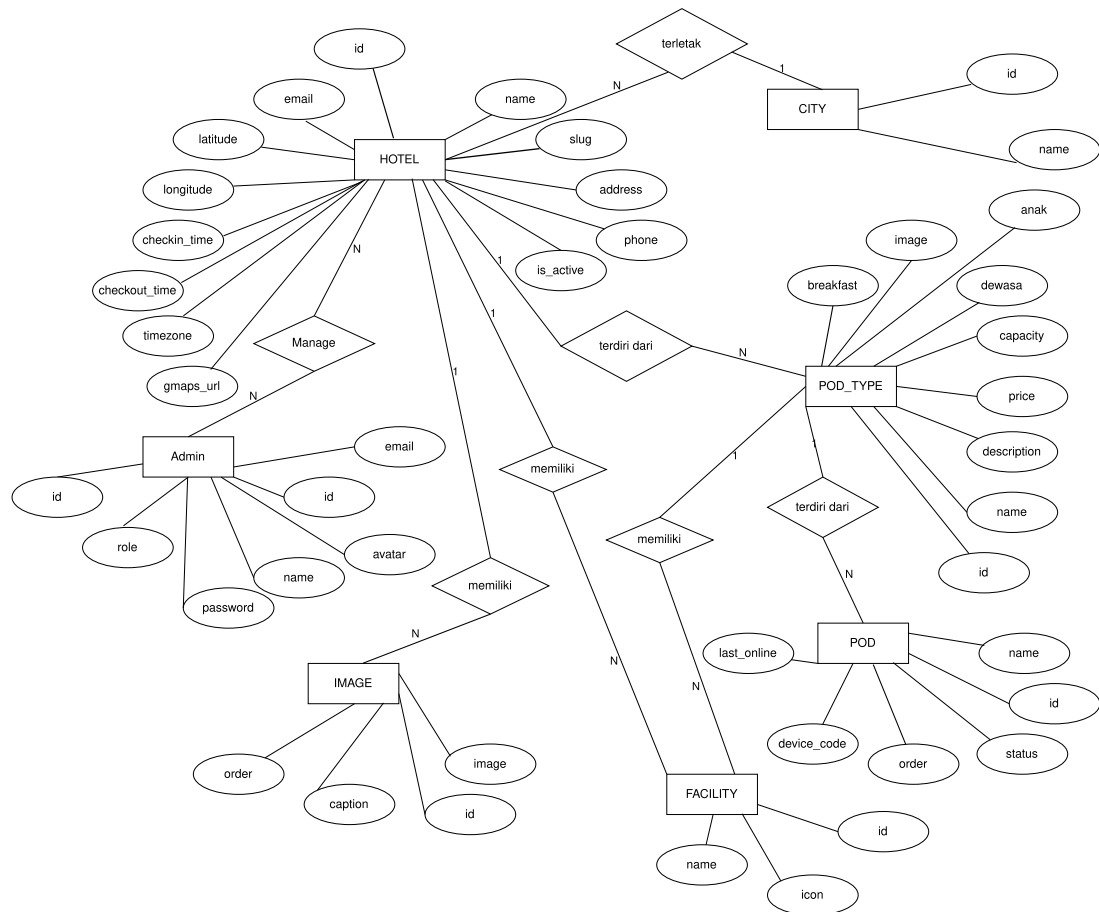
3.5.3 Entity Relationship Diagram

Pada tahap ini, *Entity Relationship Diagram* (ERD) dibuat untuk menggambarkan struktur basis data yang akan digunakan dalam sistem. ERD ini mencakup entitas-entitas utama, atribut-atributnya, serta hubungan antar entitas. ERD ini akan membantu dalam merancang basis data yang efisien dan sesuai dengan kebutuhan sistem.



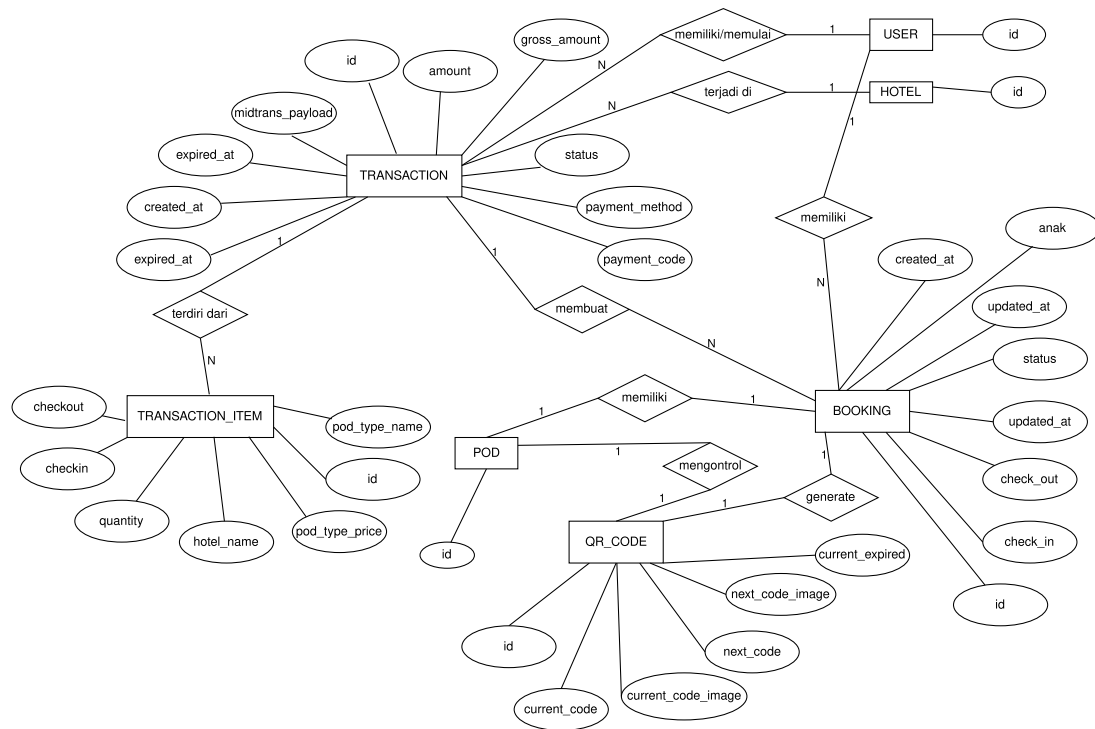
Gambar 3.31 ERD User, Verikasi Identitas, Admin, dan Token Verifikasi Global

Pada gambar 3.31, ditampilkan ERD yang mencakup entitas-entitas utama seperti User, Verifikasi Identitas (KYC), Admin, dan Token Verifikasi Global. Entitas User menyimpan informasi pengguna, sedangkan Verifikasi Identitas menyimpan data terkait verifikasi identitas pengguna. Entitas Admin menyimpan informasi tentang admin yang mengelola sistem, dan Token Verifikasi Global digunakan untuk mengelola proses verifikasi dan otentikasi pengguna dengan strategi *One Time Token*.



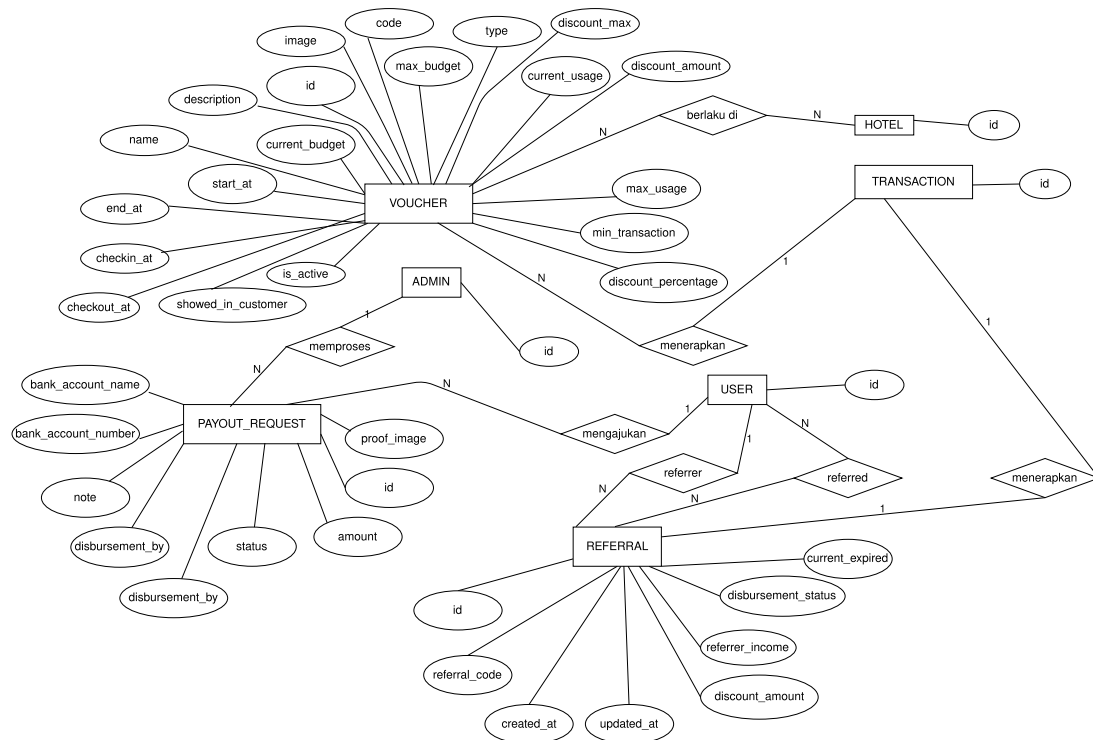
Gambar 3.32 ERD Kota, Cabang(Hotel), Fasilitas, Jenis Pod, Pod, Fasilitas, Staff (Admin), dan Gambar

Pada gambar 3.32, ditampilkan ERD yang mencakup entitas Kota, Cabang (Hotel), Fasilitas, Jenis Pod, Pod, Fasilitas, Staff (Admin), dan Gambar. Entitas Kota menyimpan informasi tentang lokasi cabang, sedangkan Cabang menyimpan data terkait hotel yang dikelola. Fasilitas dan Jenis Pod menyimpan informasi tentang fasilitas yang tersedia di setiap cabang dan jenis pod yang ditawarkan. Entitas Pod menyimpan data terkait pod yang tersedia, dan Staff (Admin) menyimpan informasi tentang admin yang mengelola cabang tersebut. Gambar digunakan untuk menyimpan representasi visual dari entitas lainnya.



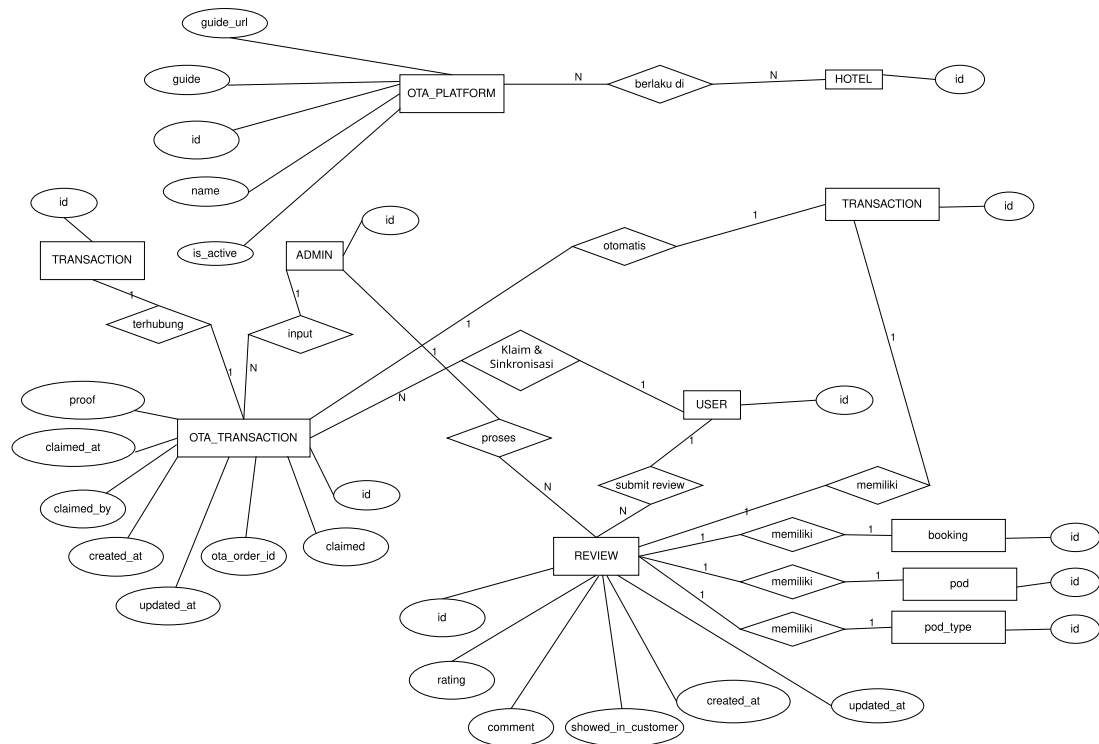
Gambar 3.33 ERD Transaksi, Item Transaksi, Booking, dan QR Code

Pada gambar 3.33, ditampilkan ERD yang mencakup entitas Transaksi, Item Transaksi, Booking, dan QR Code. Entitas Transaksi menyimpan informasi tentang transaksi yang dilakukan oleh pengguna, sedangkan Item Transaksi menyimpan data terkait item yang dibeli dalam transaksi tersebut. Entitas Booking menyimpan informasi tentang reservasi yang dilakukan oleh pengguna, dan QR Code digunakan untuk mengelola kode unik yang terkait dengan setiap transaksi atau reservasi.



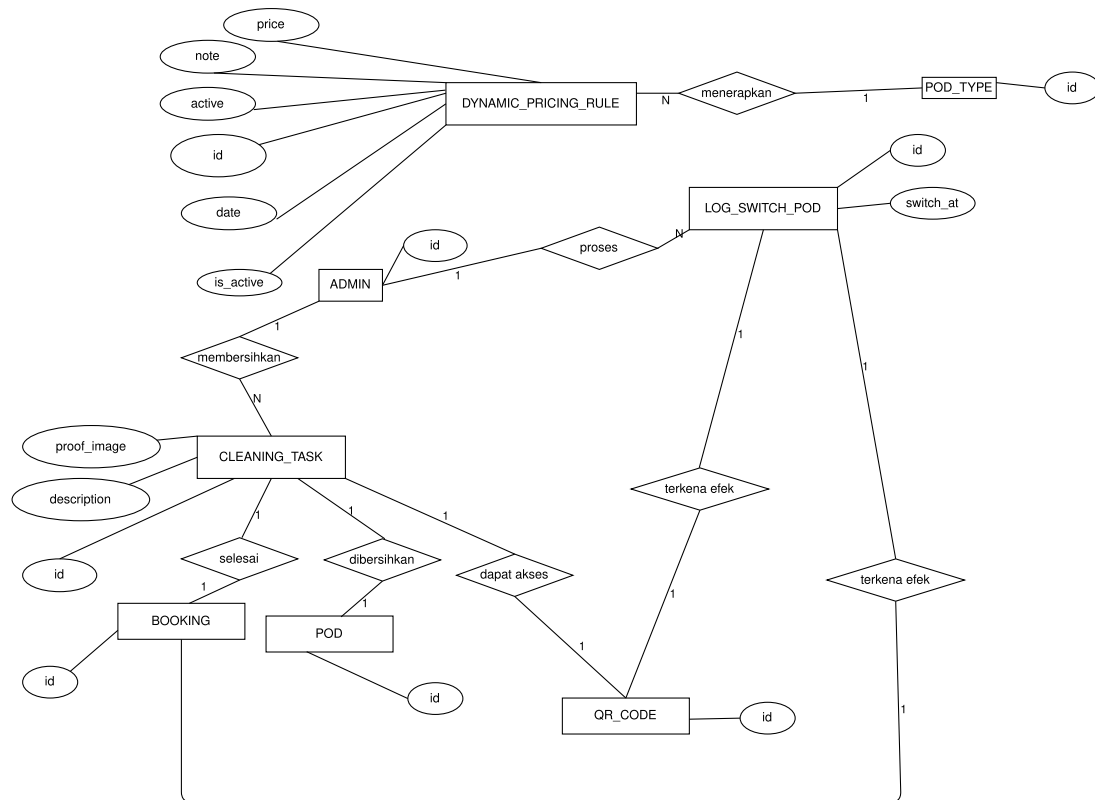
Gambar 3.34 ERD Voucher, Referral, dan Payout Request

Pada gambar 3.34, ditampilkan ERD yang mencakup entitas Voucher, Referral, dan Payout Request. Entitas Voucher menyimpan informasi tentang voucher yang digunakan oleh pengguna, sedangkan Referral menyimpan data terkait program referral yang memungkinkan pengguna untuk mengajak orang lain bergabung. Entitas Payout Request digunakan untuk mengelola permintaan pembayaran yang diajukan oleh pengguna yang berpartisipasi dalam program referral.



Gambar 3.35 ERD OTA Platform, OTA Transaction, dan Review

Pada gambar 3.35, ditampilkan ERD yang mencakup entitas OTA Platform, OTA Transaction, dan Review. Entitas OTA Platform menyimpan informasi tentang platform *Online Travel Agent* yang digunakan untuk integrasi, sedangkan OTA Transaction menyimpan data terkait transaksi yang dilakukan melalui platform tersebut. Entitas Review digunakan untuk mengelola ulasan dan penilaian yang diberikan oleh pengguna terhadap layanan yang mereka terima.

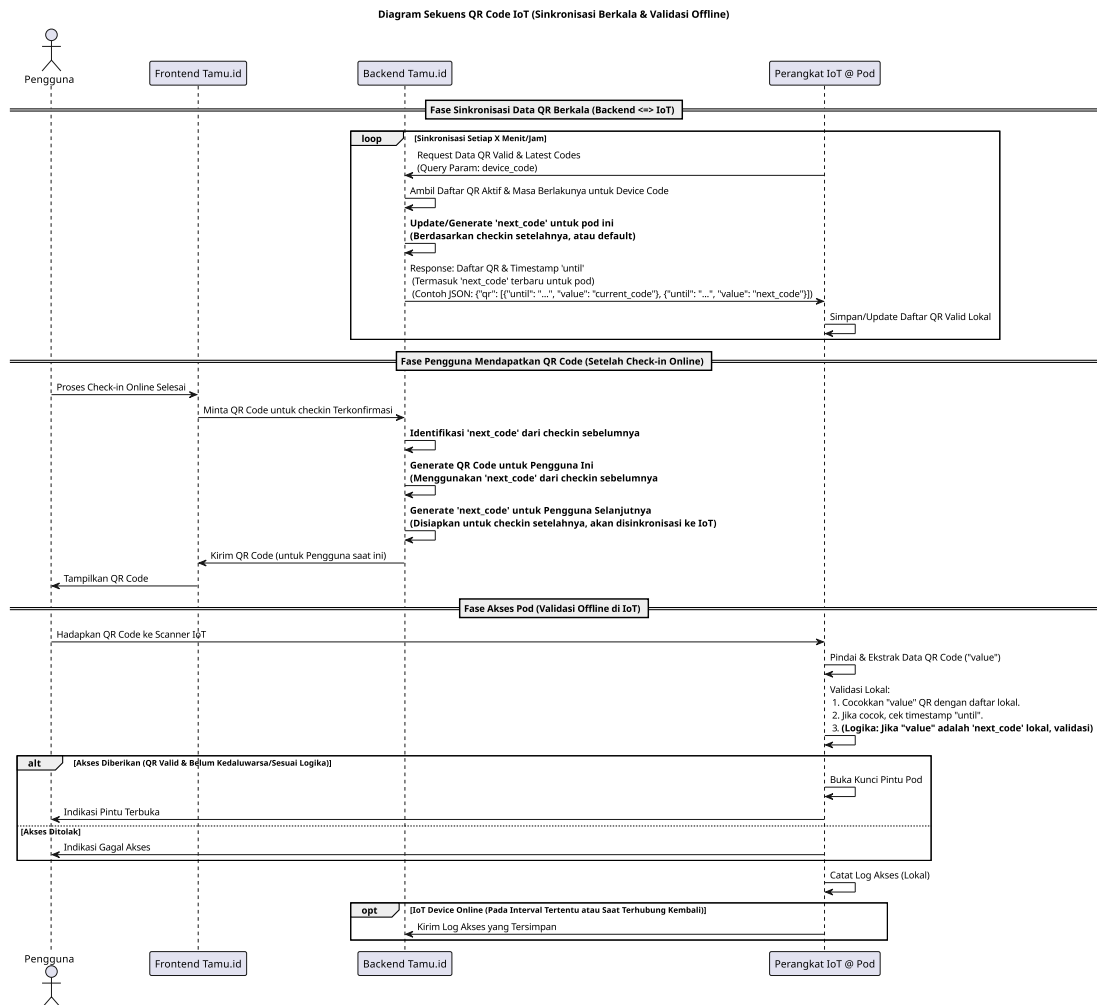


Gambar 3.36 ERD Dyanmic Pricing, Pod Cleaning, dan Switch Pod

Pada gambar 3.36, ditampilkan ERD yang mencakup entitas Dynamic Pricing, Pod Cleaning, dan Switch Pod. Entitas Dynamic Pricing menyimpan informasi tentang penetapan harga dinamis berdasarkan permintaan dan ketersediaan, sedangkan Pod Cleaning menyimpan data terkait proses pembersihan pod. Entitas Switch Pod digunakan untuk mengelola proses pindah pod yang dilakukan oleh admin cabang.

3.5.4 Ilustrasi Implementasi *Offline Fault Tolerance* pada rotasi Kunci Pod

Salah satu fitur penting dalam sistem Tamu.id adalah kemampuan untuk menangani situasi di mana koneksi internet tidak tersedia atau terputus, terutama saat pengguna melakukan checkin pada pod. Untuk mengatasi masalah ini, sistem dirancang dengan fitur *offline fault tolerance* yang memungkinkan pengguna tetap dapat mengakses pod meskipun koneksi internet tidak tersedia pada tenggang waktu tertentu di suatu cabang, dengan struktur data dan mekanisme seperti ini juga yang membuat ketika user melakukan checkin dan mendapatkan QR Code, QR Code tersebut langsung dapat digunakan untuk membuka pintu pod tanpa delay menunggu fetching data dari pod ke *backend*.

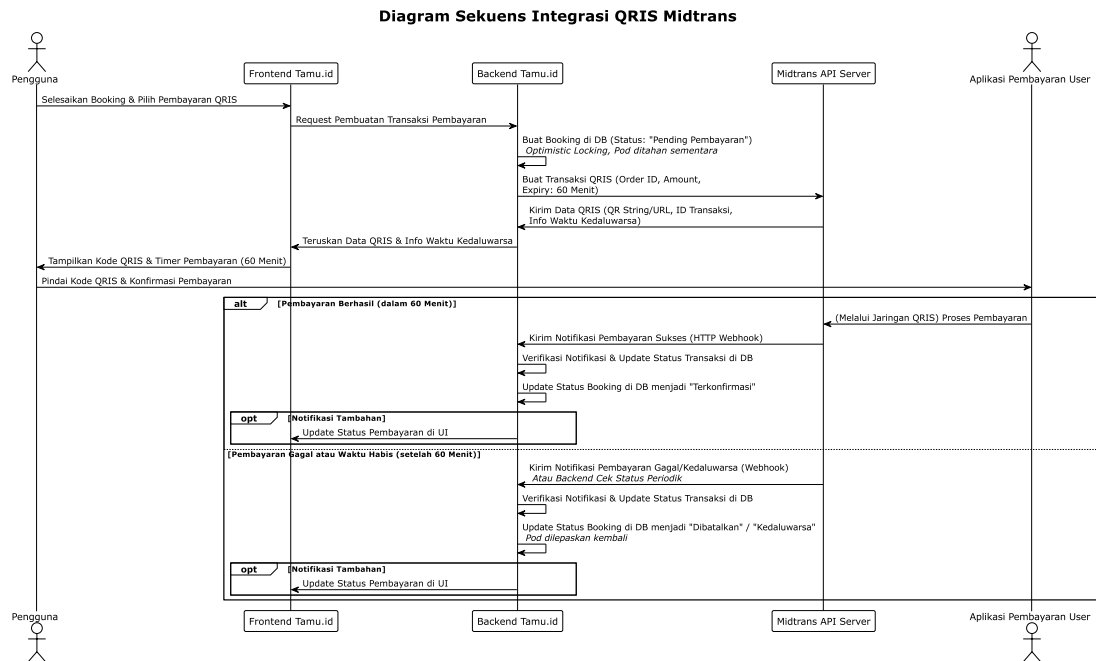


Gambar 3.37 Diagram Sekuens Implementasi *Offline Fault Tolerance* pada Rotasi Kunci Pod

Ilustrasi implementasi *offline fault tolerance* pada rotasi kunci pod dilakukan untuk memastikan bahwa sistem tetap dapat berfungsi dengan baik meskipun terjadi gangguan pada koneksi internet atau perangkat IoT. Hal ini penting untuk menjaga pengalaman pengguna yang baik dan memastikan bahwa pengguna dapat mengakses pod yang telah mereka pesan tanpa masalah.

3.5.5 Ilustrasi Integrasi *WebHook Payment Gateway*

Ilustrasi integrasi *WebHook Payment Gateway* dilakukan untuk memastikan bahwa sistem dapat menerima notifikasi pembayaran secara real-time dari penyedia layanan pembayaran. Hal ini penting untuk memastikan bahwa status pembayaran pengguna dapat diperbarui secara otomatis dan pengguna dapat melanjutkan proses checkin setelah pembayaran berhasil dilakukan.

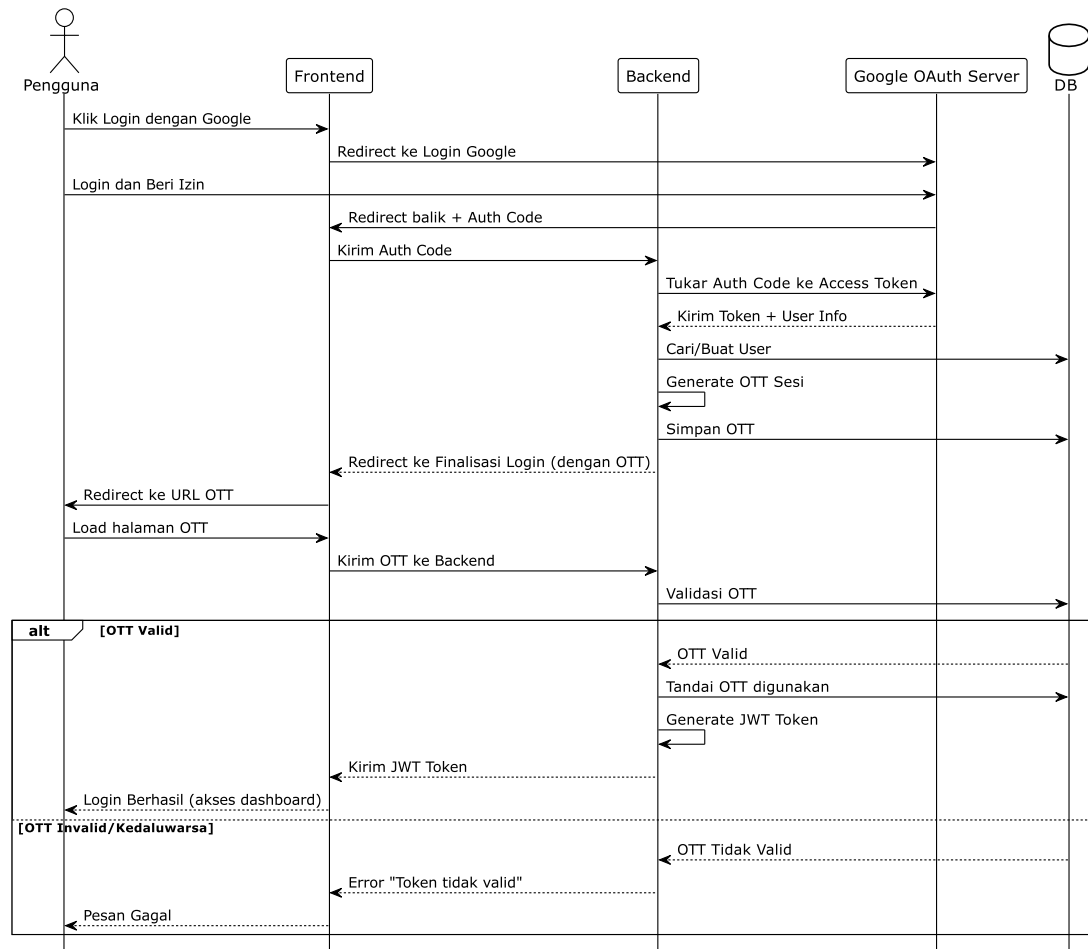


Gambar 3.38 Diagram Sekuensial Integrasi *WebHook Payment Gateway*

3.5.6 Ilustrasi Integrasi *Passwordless* dengan OAuth2 Google

Ilustrasi integrasi *passwordless* dengan OAuth2 Google dilakukan untuk memberikan kemudahan bagi pengguna dalam melakukan pendaftaran dan masuk ke dalam sistem. Dengan menggunakan OAuth2 Google, pengguna dapat masuk ke dalam sistem Tamu.id tanpa perlu mengingat password, sehingga meningkatkan pengalaman pengguna dan keamanan sistem.

Diagram Sekuens Login Google dengan (OTT)

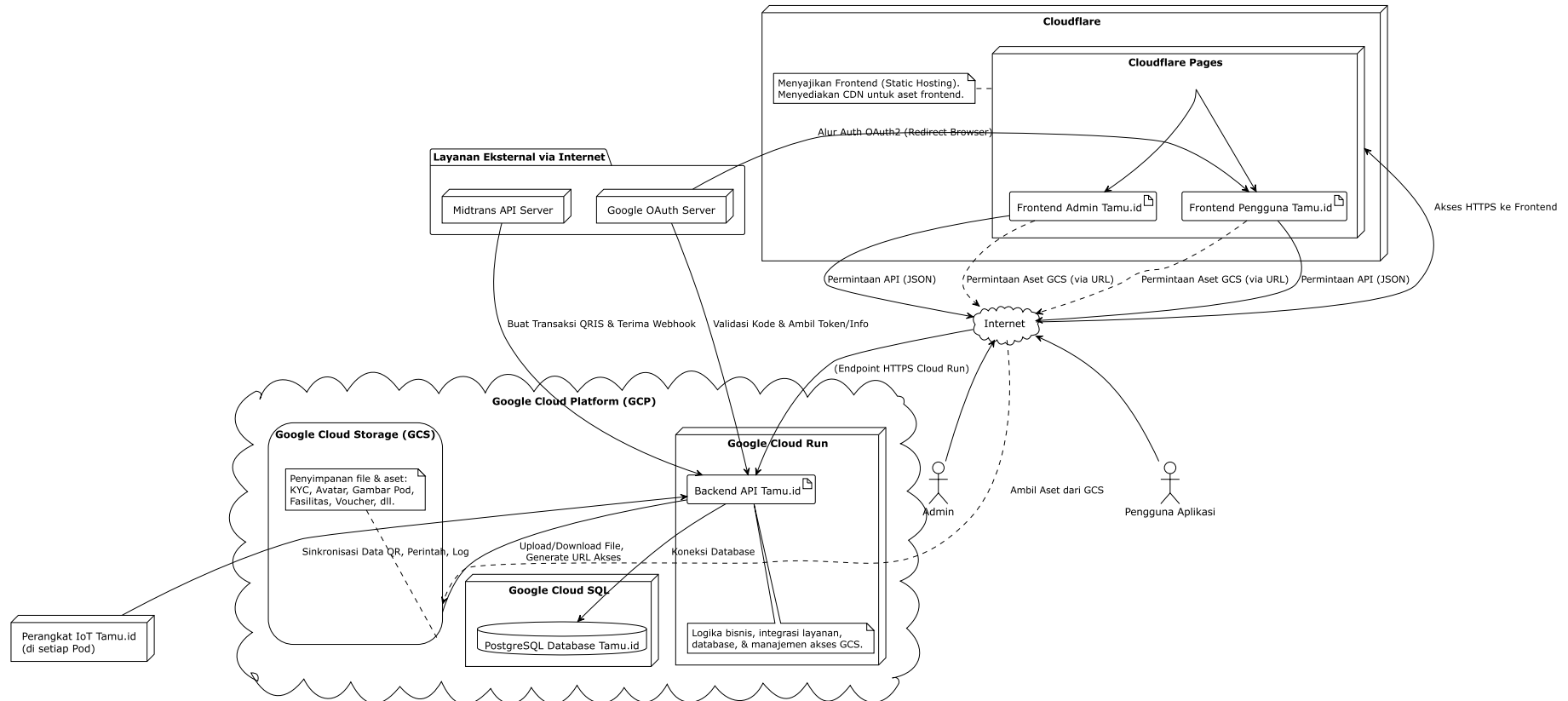


Gambar 3.39 Diagram Sekuensial Integrasi *Passwordless* dengan OAuth2 Google

3.5.7 Peluncuran Aplikasi (*Deployment*)

Peluncuran aplikasi Tamu.id dilakukan setelah semua tahap pengembangan, pengujian, dan perbaikan bug selesai. Aplikasi ini diluncurkan ke lingkungan produksi dengan menggunakan layanan cloud seperti Google Cloud Platform (GCP) untuk memastikan ketersediaan, skalabilitas, dan keamanan sistem. Proses peluncuran meliputi konfigurasi server, pengaturan basis data, dan pengaturan keamanan untuk memastikan bahwa aplikasi dapat berjalan dengan baik di lingkungan produksi. Pada gambar 3.40 dapat dilihat arsitektur peluncuran aplikasi Tamu.id yang mencakup berbagai komponen seperti server, basis data, dan layanan cloud yang digunakan untuk mendukung operasional aplikasi.

Diagram Arsitektur Deploy Aplikasi Tamu.id



Gambar 3.40 Diagram Arsitektur Peluncuran Aplikasi Tamu.id

3.5.8 Pengujian Sistem

Pengujian sistem dilakukan untuk memastikan bahwa sistem Tamu.id berfungsi sesuai dengan kebutuhan yang telah ditentukan. Pengujian ini mencakup beberapa jenis pengujian, yaitu pengujian fungsional, pengujian keamanan, pengujian performa, dan pengujian penerimaan pengguna.

A. Pengujian Black Box

Pengujian black box dilakukan untuk menguji fungsionalitas sistem tanpa memperhatikan struktur internalnya. Pengujian ini dilakukan dengan menggunakan skenario pengujian yang telah disusun berdasarkan kebutuhan fungsional yang telah dianalisis sebelumnya. Pengujian ini mencakup pengujian pada setiap fitur yang ada dalam sistem, seperti pendaftaran pengguna, pemesanan tiket, checkin, checkout, dan lain-lain, pengujian dilakukan untuk tiga peran pengguna yaitu pada tabel 3.1 untuk pengguna, tabel 3.2 untuk admin cabang, dan tabel 3.3 untuk superadmin. Pengujian ini dilakukan untuk memastikan bahwa setiap fitur berfungsi sesuai dengan yang diharapkan dan tidak ada bug yang ditemukan.

Tabel 3.1 Skenario Pengujian Black Box (User)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Registrasi Akun			
Registrasi berhasil dengan email baru	1. Buka halaman registrasi. 2. Isi nama lengkap. 3. Masukkan email yang belum terdaftar (format valid). 4. Masukkan kata sandi kuat. 5. Konfirmasi kata sandi dengan benar. 6. (Jika ada) Centang persetujuan Syarat & Ketentuan. 7. Klik tombol "Daftar".	- Pesan sukses registrasi ditampilkan. - Pengguna diarahkan ke halaman login atau dashboard. - Email konfirmasi terkirim.	

Registrasi gagal karena email sudah terdaftar	1. Buka halaman registrasi. 2. Isi nama lengkap. 3. Masukkan email yang sudah terdaftar. 4. Masukkan kata sandi. 5. Konfirmasi kata sandi. 6. Klik tombol "Daftar".	- Pesan error "Email sudah terdaftar" ditampilkan. - Pengguna tetap di halaman registrasi.	
Registrasi gagal karena format email salah	1. Buka halaman registrasi. 2. Masukkan email dengan format salah (misal: user@.com, user.com). 3. Isi field lainnya. 4. Klik "Daftar".	- Pesan error "Format email tidak valid" ditampilkan.	
Registrasi gagal karena konfirmasi kata sandi tidak cocok	1. Buka halaman registrasi. 2. Isi nama lengkap, email baru, kata sandi. 3. Masukkan konfirmasi kata sandi yang berbeda. 4. Klik tombol "Daftar".	- Pesan error "Konfirmasi kata sandi tidak cocok" ditampilkan.	
Registrasi menggunakan akun Google (sukses, pengguna baru)	1. Buka halaman registrasi/login. 2. Klik tombol "Daftar/Masuk dengan Google". 3. Pilih akun Google yang valid dan belum terdaftar di sistem. 4. Izinkan akses jika diminta.	- Akun berhasil terbuat/terhubung. - Pengguna otomatis berhasil masuk.	
Fitur Login Pengguna			

Login berhasil dengan email dan kata sandi valid	1. Buka halaman login. 2. Masukkan email terdaftar. 3. Masukkan kata sandi yang benar. 4. Klik tombol "Masuk".	- Login berhasil. - Pengguna berhasil masuk.	
Login gagal (email tidak terdaftar)	1. Buka halaman login. 2. Masukkan email yang tidak terdaftar. 3. Masukkan kata sandi. 4. Klik tombol "Masuk".	- Pesan error "Email atau kata sandi salah" ditampilkan.	
Login gagal (kata sandi salah)	1. Buka halaman login. 2. Masukkan email terdaftar. 3. Masukkan kata sandi yang salah. 4. Klik tombol "Masuk".	- Pesan error "Email atau kata sandi salah" ditampilkan.	
Login gagal (akun belum diverifikasi email, jika ada)	1. Registrasi dengan email lalu jangan verifikasi email. 2. Coba login dengan email dan kata sandi tersebut.	- Pesan error "Akun belum diverifikasi. Silakan cek email Anda." atau sejenisnya.	
Login menggunakan akun Google (sukses, akun sudah ada)	1. Buka halaman login. 2. Klik "Masuk dengan Google". 3. Pilih akun Google yang sudah terdaftar/terhubung sebelumnya.	- Login berhasil. - Pengguna diarahkan ke dashboard.	
Fitur Lupa Kata Sandi			

Proses lupa kata sandi berhasil	1. Di halaman login, klik "Lupa Kata Sandi?". 2. Masukkan email terdaftar. 3. Klik "Kirim Instruksi". 4. Buka email, klik link reset. 5. Masukkan kata sandi baru dan konfirmasinya. 6. Klik "Reset Kata Sandi".	- Pesan sukses "Kata sandi berhasil direset". - Dapat login dengan kata sandi baru.	
Lupa kata sandi dengan email tidak terdaftar	1. Di halaman login, klik "Lupa Kata Sandi?". 2. Masukkan email yang tidak terdaftar. 3. Klik "Kirim Instruksi".	- Pesan error "Email tidak ditemukan" atau instruksi tidak terkirim.	
Fitur Kelola Profil Pengguna			
Mengubah semua data profil yang diizinkan (sukses)	1. Login. 2. Ke halaman "Profil Saya" lalu "Edit Profil". 3. Ubah nama, nomor telepon, alamat, gender, tanggal lahir. 4. Klik "Simpan".	- Pesan sukses "Profil berhasil diperbarui". - Semua perubahan tercermin dengan benar.	
Mengubah data profil dengan format salah (misal: telepon)	1. Login. 2. Ke "Edit Profil". 3. Masukkan nomor telepon dengan format salah (misal: huruf). 4. Klik "Simpan".	- Pesan error "Format nomor telepon tidak valid". - Data tidak tersimpan.	
Fitur Verifikasi Identitas (KYC)			

Mengajukan verifikasi KYC (KTP) dengan data valid	1. Login. 2. Ke halaman KYC. 3. Pilih jenis ID "KTP". 4. Masukkan nomor KTP valid. 5. Unggah foto KTP jelas. 6. Unggah foto selfie jelas. 7. Klik "Ajukan".	- Pengajuan KYC berhasil dikirim. - Status KYC menjadi "Menunggu Verifikasi".	
Mengajukan KYC (Passport) dengan data valid	1. Login. 2. Ke halaman KYC. 3. Pilih jenis ID "Passport". 4. Masukkan nomor Passport valid. 5. Unggah foto Passport jelas. 6. Unggah foto selfie jelas. 7. Klik "Ajukan".	- Pengajuan KYC berhasil dikirim. - Status KYC menjadi "Menunggu Verifikasi".	
Mengajukan KYC dengan file gambar terlalu besar/format salah	1. Login. 2. Ke halaman KYC. 3. Isi data. 4. Unggah file gambar dengan ukuran < batas maksimal atau format bukan JPG/PNG. 5. Klik "Ajukan".	- Pesan error "Ukuran file terlalu besar" atau "Format file tidak didukung".	
Melihat status KYC (Pending, Approved, Rejected)	1. Login. 2. Navigasi ke halaman KYC atau Profil.	- Status KYC pengguna ditampilkan dengan benar sesuai kondisi aktual.	
Fitur Pencarian dan Reservasi			
Mencari pod (tanggal check-out sebelum check-in)	1. Login. 2. Pilih Cabang. 3. Pilih tanggal check-in. 4. Pilih tanggal check-out lebih awal dari check-in. 5. Klik "Cari Pod".	- Pesan error "Tanggal check-out tidak valid".	

Melakukan pemesanan pod dengan voucher kadaluarsa	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Di halaman pembayaran, masukkan kode voucher yang sudah kadaluarsa. 4. Klik "Terapkan Voucher".	- Pesan error "Voucher tidak valid atau sudah kadaluarsa".	
Pemesanan pod dengan jumlah melebihi kapasitas tipe pod	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Coba pesan jumlah tamu yang melebihi kapasitas maksimal dewasa/anak tipe pod tersebut.	- Pesan error "Jumlah tamu melebihi kapasitas" atau input dibatasi.	
Pemesanan pod, pembayaran berhasil (QRIS)	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Lanjutkan ke pembayaran. 4. Pilih metode QRIS, pindai QR. 5. Konfirmasi pembayaran di aplikasi e-wallet.	- Pembayaran berhasil. - Tampilan menunggu dibayar berubah menjadi terbayar - Transaksi tercatat.	
Pemesanan pod, menggunakan kode referral teman (valid, aturan disesuaikan)	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Di halaman pembayaran, masukkan kode referral valid milik teman. 4. Klik "Terapkan". 5. Selesaikan pembayaran.	- Diskon referral berhasil diterapkan. - Harga total berkurang. - Pemesanan berhasil.	
Fitur Sinkronisasi OTA			

Sinkronisasi reservasi OTA dengan kode valid	1. Login. 2. Ke halaman "Sinkronisasi OTA" atau "Klaim Reservasi". 3. Pilih OTA. 4. Masukkan kode booking OTA / nomor order OTA yang valid (sudah diinput Admin Cabang). 5. Klik "Sinkronisasi".	- Reservasi OTA berhasil disinkronkan/diklaim. - Muncul di daftar booking pengguna.	
Sinkronisasi reservasi OTA dengan kode tidak ditemukan/salah	1. Login. 2. Ke halaman "Sinkronisasi OTA". 3. Masukkan kode booking OTA yang salah. 4. Klik "Sinkronisasi".	- Pesan error "Reservasi tidak ditemukan" atau "Kode booking salah".	
Fitur Operasional Menginap			
Check-in berhasil	1. Login. 2. Buka halaman stays lalu klik tab current 3. Coba klik tombol "Check-in" 4. Pilih pod yang diinginkan 5. klik checkin	- berhasil check-in dan mendapatkan kunci pod yang diinginkan berupa QRCode	
Check-in gagal (di luar jam operasional atau sebelum waktunya check-in)	1. Login. 2. Buka halaman stays. 3. Coba klik tombol "Check-in" sebelum jam check-in yang diizinkan atau setelah batas waktu check-in.	- Tombol "Check-in" tidak ada untuk booking yang statusnya masih upcoming	

Check-out	1. Login. 2. Buka detail booking aktif yang sudah check-in. 3. Klik tombol "Check-out" 4. muncul modal rating (user wajib mengisi rating untuk isi review opsional) 5. klik kirim	- Check-out berhasil. - Status booking "Completed". rating dan review juga berhasil dikirim	
Fitur Referral			
Melihat riwayat pendapatan referral	1. Login. 2. Ke halaman Referral. 3. Pilih tab "Riwayat Pendapatan".	- Menampilkan daftar transaksi yang menghasilkan pendapatan referral dengan statusnya.	
Mengajukan pencairan referral (KYC belum disetujui)	1. Login, KYC masih "Pending" atau "Rejected". 2. Ke halaman Referral. 3. Coba klik "Cairkan Dana".	- Tombol "Cairkan Dana" tidak aktif atau pesan error "Verifikasi KYC diperlukan untuk pencairan".	
Fitur Ulasan			
Memberikan ulasan tanpa rating bintang	1. Login, setelah checkout. 2. Ke Riwayat Booking, pilih tab oldest booking. 3. Klik "Beri Ulasan". 4. Hanya tulis komentar, tidak memilih bintang. 5. Coba Kirim.	- tombol kirim review tidak dapat di klik	
Mencoba memberi ulasan untuk booking yang belum selesai/dibatalkan	1. Login. 2. Cari booking yang statusnya "Pending Payment" atau "Cancelled". 3. Coba cari opsi "Beri Ulasan".	- Opsi "Beri Ulasan" tidak tersedia untuk booking tersebut.	

Tabel 3.2 Skenario Pengujian Black Box (Admin)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Login dan Profil Admin Cabang			
Login Admin Cabang berhasil	1. Buka halaman login admin. 2. Masukkan email Admin Cabang valid. 3. Masukkan kata sandi benar. 4. Klik "Masuk".	- Login berhasil. - Diarahkan ke halaman pilih cabang yang ingin dikelola Cabang.	
Login Admin Cabang gagal (kata sandi salah)	1. Buka halaman login admin. 2. Masukkan email Admin Cabang valid. 3. Masukkan kata sandi salah. 4. Klik "Masuk".	- Pesan error "Email atau kata sandi salah". - Tetap di halaman login.	
Mengubah profil Admin Cabang (misal: nama)	1. Login sebagai Admin Cabang. 2. Navigasi ke "Profil Saya". 3. Klik "Edit Profil". 4. Ubah nama. 5. Klik "Simpan".	- Nama Admin Cabang berhasil diperbarui.	
Fitur Pengelolaan Operasional Cabang			
Mengedit semua informasi detail cabang yang diizinkan	1. Login. 2. Ke "Pengaturan data Cabang". 3. Ubah nama, alamat, telepon, email, deskripsi, jam check-in/out, gambar utama. 4. Klik "Simpan".	- Semua perubahan berhasil disimpan dan ditampilkan dengan benar.	

Menambah tipe pod baru dengan semua field valid	1. Login. 2. Ke "Menu jenis pod" 3. Klik "Tambah Baru". 4. Isi nama, deskripsi, harga, kapasitas (dewasa, anak), gambar, fasilitas terkait, status breakfast. 5. Klik "Simpan".	- Tipe pod baru berhasil ditambahkan dengan semua detailnya.	
Menghapus tipe pod yang masih memiliki pod aktif	1. Login. 2. Ke "Manajemen Pod" - "Tipe Pod". 3. Pilih tipe pod yang memiliki pod terdaftar di bawahnya. 4. Klik "Hapus". 5. Konfirmasi.	- Pesan error "Tidak dapat menghapus tipe pod karena masih ada pod yang terhubung" atau sejenisnya. - Tipe pod tidak terhapus.	
Menambah pod baru dengan device code IoT unik	1. Login. 2. Ke "Manajemen Pod" - "pilih jenis pod yang pod nya ingin ditambahkan". 3. Klik "Tambah Pod". 4. Isi nama/nomor pod, pilih tipe pod, masukkan device code IoT yang unik. 5. Klik "Simpan".	- Pod baru berhasil ditambahkan. - Status awal "Tersedia".	
Menambah pod baru dengan device code IoT yang sudah ada	1. Login. 2. Ke "Manajemen Pod" - "Daftar Pod". 3. Klik "Tambah Pod". 4. Masukkan device code IoT yang sudah digunakan pod lain. 5. Klik "Simpan".	- Pesan error "Device code sudah digunakan". - Penambahan pod gagal.	

Mengubah status pod menjadi "Perlu Dibersihkan" secara manual	1. Login. 2. Ke "Manajemen Pod" - "Daftar Pod". 3. Pilih pod yang "Tersedia", klik "Edit". 4. Ubah status ke "Perlu Dibersihkan". 5. Klik "Simpan".	- Status pod berhasil diubah.	
Membuat voucher diskon nominal untuk cabang	1. Login. 2. Ke "Manajemen Voucher Cabang". 3. Klik "Tambah Voucher". 4. Isi kode, nama, deskripsi, tipe "NOMINAL", jumlah diskon, min transaksi, periode berlaku, maks penggunaan. 5. Klik "Simpan".	- Voucher cabang berhasil dibuat.	
Membuat voucher diskon persentase dengan batas maks diskon	1. Login. 2. Ke "Manajemen Voucher Cabang". 3. Klik "Tambah Voucher". 4. Isi kode, nama, tipe "PERSENTASE", nilai persentase, diskon maksimal, min transaksi, periode, kuota. 5. Klik "Simpan".	- Voucher cabang berhasil dibuat.	

Menetapkan harga dinamis untuk akhir pekan	1. Login. 2. Ke "Harga Dinamis". 3. Pilih tipe pod. 4. Pilih rentang tanggal mencakup beberapa akhir pekan. 5. Masukkan harga lebih tinggi. 6. Klik "Simpan".	- Harga dinamis untuk akhir pekan berhasil disimpan.	
Menetapkan harga dinamis yang tumpang tindih dengan aturan lain	1. Login. 2. Ke "Harga Dinamis". 3. Buat aturan harga dinamis baru yang tanggalnya tumpang tindih dengan aturan yang sudah ada untuk tipe pod yang sama.	- Sistem memberikan peringatan bahwa aturan harga ada yang tumpang tindih dan aturan harga baru tidak berhasil disimman	
Fitur Manajemen Reservasi dan Tamu Cabang			
Input manual transaksi OTA dengan semua field lengkap	1. Login. 2. Ke "Transaksi OTA". 3. Klik "Tambah". 4. Isi Jenis OTA, ID Order OTA, nama tamu jika ada, etanggal check-in/out, tipe pod, jumlah tamu, dan harga di OTA 5. Klik "Simpan".	- Transaksi OTA berhasil dicatat.	
Input manual transaksi OTA dengan field wajib kosong	1. Login. 2. Ke "Transaksi OTA". 3. Klik "Tambah". 4. Kosongkan salah satu field wajib (misal: ID Order OTA). 5. Coba klik "Simpan".	- Tombol simpan tidak bisa diklik karena masih ada field wajib yang kosong	

Memindahkan tamu ke pod lain karena ada masalah di pod asal	1. Login. 2. Cari booking tamu aktif. 3. Pilih "Pindah Pod", berikan alasan (misal: AC rusak). 4. Pilih pod lain yang kosong, tipe sama. 5. Konfirmasi.	- Tamu berhasil dipindahkan. - Pod asal mungkin statusnya berubah jadi "Maintenance".	
Mencatat pembersihan pod dengan unggah foto bukti	1. Login. 2. Ke "Log Kebersihan" atau daftar pod. 3. Pilih pod "Perlu Dibersihkan". 4. Klik "Sudah Dibersihkan". 5. Unggah foto bukti kebersihan. 6. (Jika ada) Tambahkan catatan. 7. Simpan.	- Status pod "Tersedia". - Foto bukti dan catatan tersimpan di log.	
Fitur Manajemen Ulasan Cabang			
Menyetujui ulasan pengguna yang konstruktif	1. Login. 2. Ke "Manajemen Ulasan". 3. Pilih ulasan "Pending" yang isinya baik. 4. Klik "Setujui/Tampilkan".	- Ulasan tampil di halaman publik.	
Menolak ulasan pengguna yang mengandung spam/kata kasar	1. Login. 2. Ke "Manajemen Ulasan". 3. Pilih ulasan yang berisi spam. 4. Klik "Tolak/Sembunyikan".	- Ulasan tidak tampil dan statusnya diperbarui.	

Tabel 3.3 Skenario Pengujian Black Box (SuperAdmin)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Login & Dashboard Global SuperAdmin			
Login SuperAdmin berhasil	1. Buka halaman login admin. 2. Masukkan email SuperAdmin. 3. Masukkan kata sandi yang benar. 4. Klik "Masuk".	- Login berhasil. - Diarahkan ke dashboard SuperAdmin.	
Melihat statistik global di dashboard SuperAdmin	1. Login sebagai SuperAdmin. 2. Amati informasi di dashboard (total cabang, total pengguna, total pendapatan global, dll.).	- Menampilkan data statistik agregat dari semua cabang yang akurat.	
Fitur Manajemen Data Master Global			
Menambah data cabang (hotel) baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Cabang/Hotel". 3. Klik "Tambah Cabang Baru". 4. Isi semua field yang diperlukan (nama, alamat, kota, kontak, slug unik, dll.). 5. Klik "Simpan".	- Cabang baru berhasil ditambahkan ke sistem. - Cabang muncul di daftar cabang.	

Mengedit data cabang (hotel) yang sudah ada	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Cabang/Hotel". 3. Pilih cabang, klik "Edit". 4. Ubah salah satu field (misal: email kontak). 5. Klik "Simpan".	- Data cabang berhasil diperbarui.	
Menambah data kota baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Kota". 3. Klik "Tambah Kota". 4. Masukkan nama kota. 5. Klik "Simpan".	- Kota baru berhasil ditambahkan. - Kota dapat dipilih saat menambah/mengedit cabang.	
Menambah akun Admin Cabang baru dan mengassign ke cabang	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Staff". 3. Klik "Tambah Staff". 4. Isi detail staff (nama, email, password), pilih peran "Admin Cabang". 5. Assign ke satu atau beberapa cabang. 6. Klik "Simpan".	- Akun Admin Cabang baru berhasil dibuat. - Admin Cabang tersebut dapat login dan mengelola cabang yang di-assign.	
Menghapus akun staff (Admin Cabang)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Staff". 3. Cari akun Admin Cabang. 4. Klik opsi "Hapus".	- Akun Admin Cabang berhasil dihapus. - Admin Cabang tersebut tidak dapat login.	

Menambah fasilitas global baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Fasilitas". 3. Klik "Tambah Fasilitas". 4. Masukkan nama fasilitas dan unggah ikon. 5. Klik "Simpan".	- Fasilitas baru berhasil ditambahkan. - Fasilitas dapat di-assign ke hotel atau tipe pod.	
Membuat voucher global untuk semua cabang	1. Login. 2. Ke "Manajemen Voucher". 3. Klik "Tambah Voucher". 4. Isi detail (kode, nama, tipe, nilai, periode, kuota). 5. Pilih opsi "Berlaku di semua cabang". 6. Klik "Simpan".	- Voucher global berhasil dibuat. - Dapat digunakan pengguna di semua cabang.	
Fitur Verifikasi & Approval Sistem			
Menyetujui permintaan verifikasi KYC pengguna yang valid	1. Login sebagai SuperAdmin. 2. Navigasi ke "Verifikasi KYC". 3. Pilih permintaan KYC yang berstatus "Pending" dengan data dan foto valid. 4. Klik "Setujui".	- Status KYC pengguna berubah menjadi "Approved". - Pengguna mendapatkan pembaharuan. - Pengguna dapat mengakses fitur yang memerlukan KYC.	
Menolak permintaan verifikasi KYC pengguna (misal: foto buram)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Verifikasi KYC". 3. Pilih permintaan KYC yang fotonya buram. 4. Klik "Tolak". 5. Masukkan alasan penolakan (misal: "Foto tidak jelas").	- Status KYC pengguna berubah menjadi "Rejected". - Pengguna mendapatkan pembaharuan beserta alasan penolakan.	

Menyetujui permintaan pencairan saldo referral yang valid	1. Login sebagai SuperAdmin. 2. Navigasi ke "Permintaan Payout Referral". 3. Pilih permintaan yang valid (saldo pengguna cukup, data bank benar). 4. Klik "Setujui". 5. Proses transfer dana dan unggah buktinya.	- Status permintaan payout berubah menjadi "Approved". - Pengguna mendapatkan pembaharuan.	
Menolak permintaan pencairan saldo referral (saldo kurang)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Permintaan Payout Referral". 3. Pilih permintaan dimana saldo aktual pengguna kurang dari yang diajukan. 4. Klik "Tolak". 5. Masukkan alasan.	- Status permintaan payout berubah menjadi "Rejected". - Pengguna mendapatkan pembaharuan.	

B. User Acceptance Testing

Pengujian penerimaan pengguna dilakukan untuk memastikan bahwa sistem memenuhi kebutuhan dan harapan pengguna. Pengujian ini melibatkan pengguna akhir yang akan menggunakan sistem Tamu.id untuk melakukan berbagai aktivitas seperti pemesanan tiket, checkin, checkout, dan lain-lain. Tujuan dari pengujian ini adalah untuk memastikan bahwa sistem mudah digunakan, intuitif, dan memenuhi kebutuhan pengguna. Penilaian dilakukan berdasarkan lima parameter dalam skala *Likert* yaitu Sangat Tidak Setuju(STS), Tidak Setuju (TS), Netral (N), Setuju (S), dan Sangat Setuju (SS), pengujian ini dilakukan untuk tiga peran pengguna yaitu pada tabel 3.4 untuk pengguna, tabel 3.5 untuk admin cabang, dan tabel 4.7 untuk superadmin. Pengujian ini dilakukan untuk memastikan bahwa sistem mudah digunakan, intuitif, dan memenuhi kebutuhan pengguna.

Tabel 3.4 Kuesioner *User Acceptance Test* - Perspektif *User*

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses pendaftaran akun dan login ke aplikasi Tamu.id mudah dipahami dan dilakukan?					
2.	Apakah tampilan antarmuka aplikasi Tamu.id menarik dan mudah digunakan secara umum?					
3.	Apakah informasi mengenai cabang dan tipe pod (harga, fasilitas, ketersediaan) disajikan dengan jelas?					
4.	Apakah alur proses pencarian dan pemesanan pod mudah diikuti dan efisien?					
5.	Apakah proses pembayaran saat melakukan pemesanan mudah dan terasa aman?					
6.	Apakah proses check-in dan check-out melalui aplikasi berjalan dengan lancar?					
7.	Apakah penggunaan QR Code untuk akses pod (jika ada fitur ini) mudah dan praktis?					
8.	Apakah saya mudah menemukan riwayat pemesanan dan detail transaksi saya?					
9.	Apakah fitur untuk memberikan ulasan dan rating setelah menginap mudah digunakan?					
10.	Secara keseluruhan, apakah Anda puas dengan fungsionalitas dan kemudahan penggunaan aplikasi Tamu.id?					
Total Bobot						
Skor						
Total Skor						

Tabel 3.5 Kuesioner UAT - Perspektif Admin Cabang

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses login ke dashboard Admin Cabang mudah dan aman?					

2.	Apakah tampilan dashboard Admin Cabang informatif untuk memantau operasional harian cabang?					
3.	Apakah pengelolaan informasi umum cabang (seperti detail kontak, foto, atau deskripsi) mudah dilakukan?					
4.	Apakah fitur untuk menambah dan mengelola tipe pod serta unit pod individual di cabang saya mudah digunakan?					
5.	Apakah proses untuk mengatur harga pod (termasuk harga dinamis jika digunakan) cukup jelas dan mudah diterapkan?					
6.	Apakah pengelolaan data reservasi tamu (melihat daftar booking, detail, dan status) efisien?					
7.	Apakah fitur untuk menginput atau mengelola data reservasi yang berasal dari Online Travel Agent (OTA) mudah dioperasikan?					
8.	Apakah proses untuk mengelola status kebersihan dan ketersediaan pod (misalnya, dari "Perlu Dibersihkan" menjadi "Tersedia") sederhana?					
9.	Apakah fitur untuk melihat dan menanggapi (memoderasi) ulasan dari pengguna untuk cabang saya mudah digunakan?					
10.	Secara keseluruhan, apakah dashboard Admin Cabang ini efektif membantu saya dalam mengelola operasional cabang sehari-hari?					
Total Bobot						
Skor						
Total Skor						

Tabel 3.6 Kuesioner UAT - Perspektif SuperAdmin

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses login ke dashboard SuperAdmin mudah dan aman?					
2.	Apakah tampilan dashboard global SuperAdmin menyajikan informasi ringkasan sistem secara efektif dan mudah dipahami?					
3.	Apakah pengelolaan data master sistem (seperti daftar Cabang/Hotel, Kota, OTA global) mudah dilakukan?					
4.	Apakah fitur untuk menambah, mengedit, dan mengelola akun staf (Admin Cabang, SuperAdmin lain) serta hak aksesnya intuitif?					
5.	Apakah proses verifikasi identitas pengguna (KYC) dari sisi SuperAdmin berjalan dengan alur yang jelas dan efisien?					
6.	Apakah proses approval atau penolakan permintaan pencairan saldo referral pengguna mudah dikelola dan dilacak?					
7.	Apakah fitur untuk membuat dan mengelola voucher yang berlaku secara global atau untuk banyak cabang mudah digunakan?					
8.	Apakah sistem SuperAdmin menyediakan alat yang memadai untuk memantau aktivitas penting dan kesehatan sistem secara keseluruhan?					
9.	Apakah laporan global atau data analitik yang dihasilkan sistem (jika ada) membantu dalam analisis dan pengambilan keputusan strategis?					
10.	Secara keseluruhan, apakah antarmuka SuperAdmin efektif dalam membantu saya menjalankan tugas administrasi sistem dan pengawasan global?					
Total Bobot						

Skor					
Total Skor					

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil dan Pembahasan

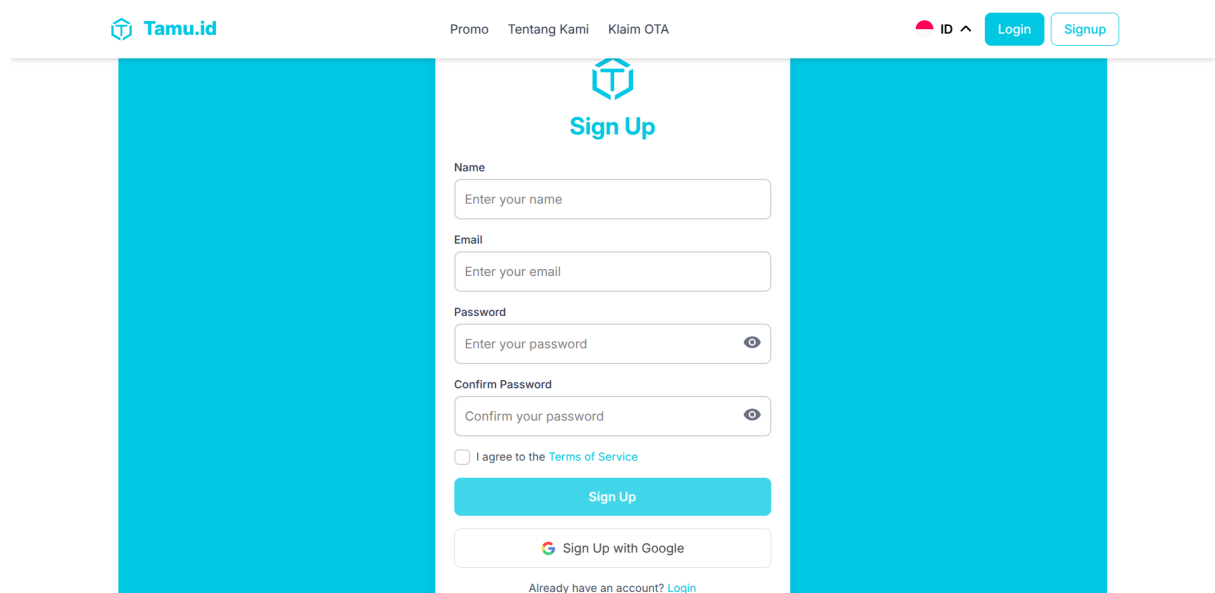
Dari hasil analisis yang telah dilakukan maka selanjutnya adalah mengimplementasikan sistem yang telah dirancang, berikut adalah hasil implementasi sistem yang telah dibuat.

4.1.1 Halaman User

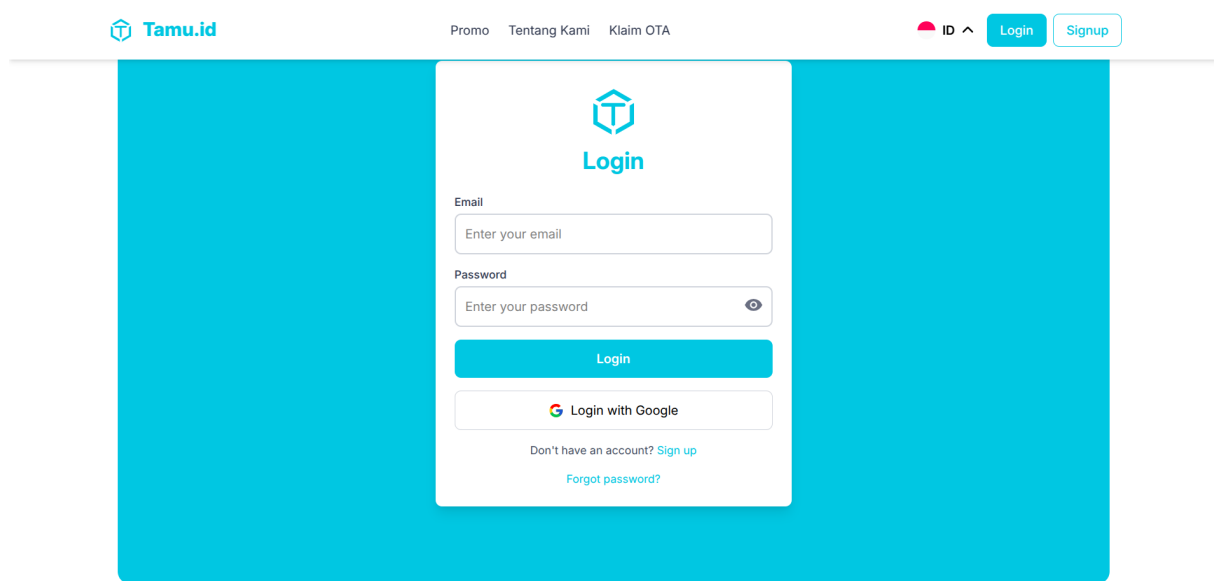
Halaman User adalah antarmuka utama yang dirancang untuk pengguna akhir aplikasi Tamu.id. Melalui halaman ini, pengguna dapat melakukan seluruh alur perjalanan pelanggan, mulai dari pendaftaran akun, pencarian dan reservasi pod, proses menginap, hingga memanfaatkan program loyalitas. Setiap fitur dirancang dengan fokus pada kemudahan penggunaan dan pengalaman pengguna yang seamless dan intuitif.

A. Registrasi, Login dan kelola Data User

Proses autentikasi dan pengelolaan akun merupakan langkah fundamental bagi pengguna untuk dapat berinteraksi secara penuh dengan layanan Tamu.id. Sistem menyediakan alur pendaftaran dan login yang fleksibel serta memberikan kontrol penuh kepada pengguna untuk mengelola data pribadi mereka, demi menciptakan lingkungan yang aman dan terpersonalisasi.



Gambar 4.1 Tampilan Halaman Pendaftaran User



Gambar 4.2 Tampilan Halaman Login User

Pada halaman pendaftaran (Gambar 4.1) dan login (Gambar 4.2), pengguna dapat membuat akun atau masuk menggunakan email dan kata sandi atau memanfaatkan integrasi Google OAuth. Fitur login dengan Google ini sangat penting untuk menyederhanakan proses autentikasi. Seperti yang ditunjukkan pada implementasi backend di Gambar 4.3, sistem menerima token dari Google, memverifikasinya, lalu secara aman membuat atau mengambil data pengguna dan menghasilkan token sesi JWT. Implementasi ini tidak hanya memberikan kemudahan bagi pengguna tetapi juga meningkatkan keamanan dengan mengandalkan protokol autentikasi yang sudah teruji.

```

def google_login():
    try:
        GOOGLE_CLIENT_ID = os.getenv("GOOGLE_CLIENT_ID")
        GOOGLE_REDIRECT_URI = os.getenv("GOOGLE_USER_REDIRECT_URI")
        google_auth_url = (
            "https://accounts.google.com/o/oauth2/auth"
            "?response_type=code"
            f"&client_id={GOOGLE_CLIENT_ID}"
            f"&redirect_uri={GOOGLE_REDIRECT_URI}"
            "&scope=openid%20email%20profile"
        )
        return redirect(google_auth_url)
    except Exception as e:
        logging.error(f"Error in google_login: {e}")
        return jsonify({"message": "Internal server error"}), 500

def google_callback():
    try:
        code = request.args.get("code")
        if not code:
            return jsonify({"message": "Code not provided"}), 400
        # change code with access token
        token_url = "https://oauth2.googleapis.com/token"
        GOOGLE_CLIENT_ID = os.getenv("GOOGLE_CLIENT_ID")
        GOOGLE_CLIENT_SECRET = os.getenv("GOOGLE_CLIENT_SECRET")
        GOOGLE_REDIRECT_URI = os.getenv("GOOGLE_USER_REDIRECT_URI")
        token_data = {
            "code": code,
            "client_id": GOOGLE_CLIENT_ID,
            "client_secret": GOOGLE_CLIENT_SECRET,
            "redirect_uri": GOOGLE_REDIRECT_URI,
            "grant_type": "authorization_code"
        }
        token_response = requests.post(token_url, data=token_data)
        token_data = token_response.json()
        access_token = token_data.get("access_token")
        if not access_token:
            return jsonify({"message": "Invalid code"}), 400

        # get user info
        user_info_res = requests.get(
            "https://www.googleapis.com/oauth2/v1/userinfo",
            headers={"Authorization": f"Bearer {access_token}"},
        )
        user_info = user_info_res.json()
        email = user_info.get("email")

        user = User.query.filter_by(email=email).first()
        token = str(uuid.uuid4())
        new_token = Token(token=token, email=email)
        db.session.add(new_token)
        db.session.commit()
        return redirect(f"{os.getenv('FRONTEND')}/sso?token={token}")
    except Exception as e:
        logging.error(f"Error in google_callback: {e}")
        return jsonify({"message": "Internal server error"}), 500

```

Gambar 4.3 Implementasi Integrasi *Oauth2 Google* pada Sistem Autentikasi

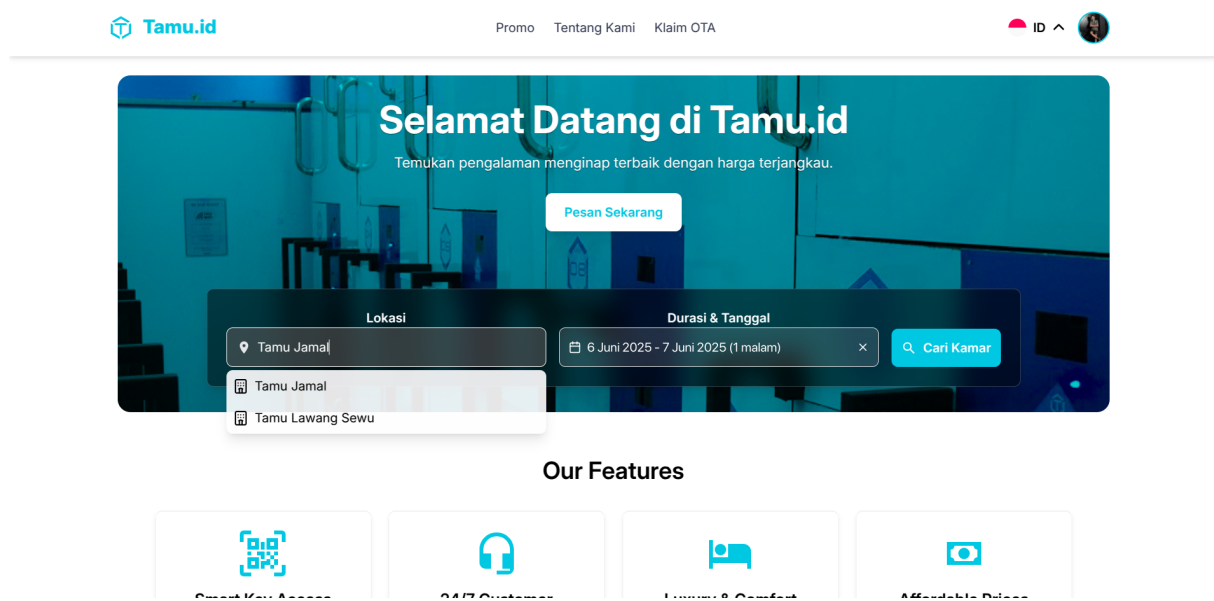
The screenshot displays the user profile management interface of the Tamu.id application. At the top, the 'Tamu.id' logo is on the left, a green notification banner reads 'Profil berhasil diperbarui!', and a user profile icon with 'ID' is on the right. The main content is divided into two panels. The left panel, titled 'Edit Profile', features a circular profile picture of a woman and a 'Simpan Perubahan' button at the bottom. It contains several input fields: 'Nama' (filled with 'Aliif Arief'), 'Username' (filled with 'alfa'), 'Email (can't be changed)' (filled with 'aliif.ariief.m@gmail.com'), 'No. Telepon' (filled with '085172054903'), 'Gender' (with 'Laki-laki' selected and 'Perempuan' as an option), 'Tanggal Lahir' (filled with '09/04/2003'), and 'Alamat' (filled with 'Villa Mutiara Gading'). The right panel, titled 'Update Password', includes three password input fields: 'Password Lama', 'Password Baru', and 'Konfirmasi Password Baru', each with a visibility toggle icon. A 'Simpan Password' button is located at the bottom of this panel.

Gambar 4.4 Tampilan Halaman Profil User

Setelah berhasil login, pengguna dapat mengakses halaman profil mereka (Gambar 4.4) untuk mengelola data pribadi. Halaman ini memberikan pengguna wewenang untuk melihat dan memperbarui informasi seperti nama, foto profil, dan detail kontak lainnya. Fitur pengelolaan profil ini sangat penting untuk menjaga akurasi data pengguna dan memberikan rasa kontrol atas informasi pribadi mereka di dalam platform.

B. Landing Page dan Proses Reservasi User

Proses pencarian dan reservasi merupakan jantung dari aplikasi Tamu.id. Alur ini dirancang untuk menjadi lancar dan intuitif, memandu pengguna dari pencarian awal hingga pembayaran yang aman, dengan menyertakan mekanisme teknis untuk menangani potensi konflik data.



Gambar 4.5 Tampilan Halaman Landing Page User

Landing page (Gambar 4.5) menjadi titik awal proses reservasi, di mana pengguna dapat langsung melakukan pencarian pod berdasarkan kota dan tanggal. Setelah mendapatkan hasil pencarian yang relevan, pengguna dapat memilih jenis pod yang diinginkan dan melanjutkan ke proses pembayaran. Proses ini dirancang sebagai alur multi-langkah yang terstruktur untuk memastikan kejelasan dan keamanan transaksi.

Pada tahap akhir sebelum pembayaran, sistem melakukan proses krusial di sisi backend seperti yang ditunjukkan pada Gambar 4.6. Kode ini bertanggung jawab untuk membuat permintaan transaksi ke *payment gateway* Midtrans. Lebih dari itu, di dalam implementasi ini tertanam logika bisnis yang vital, termasuk mekanisme *optimistic locking*. Sebelum menghasilkan kode QRIS, sistem melakukan validasi akhir terhadap ketersediaan pod untuk mencegah kondisi *race condition* atau *double-booking* yang mungkin terjadi jika beberapa pengguna mencoba memesan pod yang sama secara bersamaan.

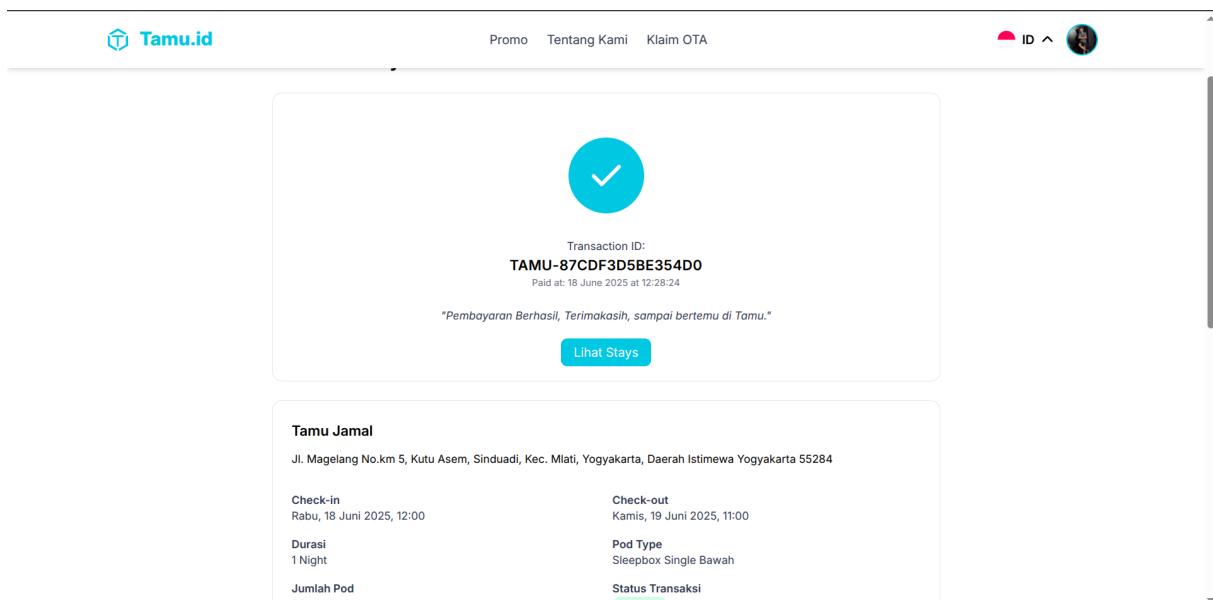
```

payload = {
    "payment_type": payment_type,
    "transaction_details": {
        "order_id": transaction.id,
        "gross_amount": transaction.gross_amount - referral_discount - voucher_discount
    },
    "item_details": item_details,
    "customer_details": {
        "first_name": user.name,
        "email": user.email,
        "phone": user.phone
    },
    "qris": { "acquirer": "gopay" },
    "custom_expiry": {
        "order_time": datetime.fromtimestamp(transaction.created_at,
tz=timedelta(hours=7)).strftime('%Y-%m-%d %H:%M:%S %z'),
        "expiry_duration": transaction.expired_time,
        "unit": "minute"
    }
}
headers = {
    "Content-Type": "application/json",
    "Accept": "application/json",
    "Authorization": f"Basic {auth_key}",
    "X-Override-Notification": webhook_url
}
response = requests.post(f"{midtrans_url}/charge", json=payload, headers=headers)
response_data = response.json()

```

Gambar 4.6 Implementasi Integrasi API Midtrans dan *Optimistic Locking*

Setelah pembayaran berhasil, yang dikonfirmasi melalui *webhook* dari Midtrans, pengguna akan langsung melihat halaman konfirmasi keberhasilan reservasi (Gambar 4.7), memberikan kepastian transaksi secara *real-time*.

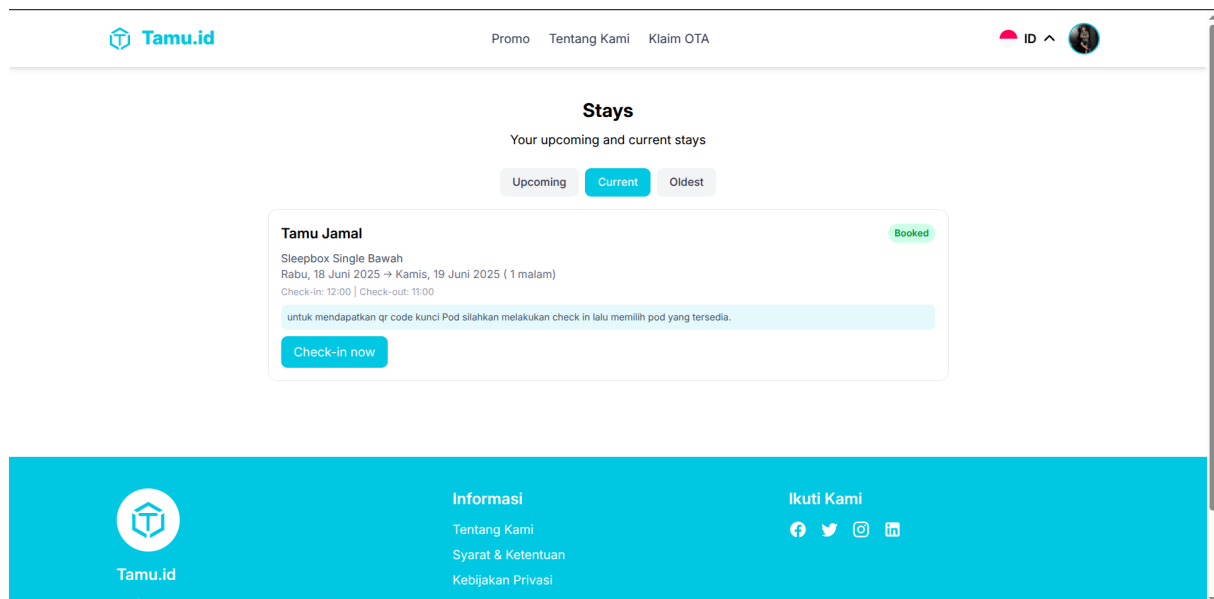


Gambar 4.7 Tampilan Proses Pembayaran Berhasil

C. Proses Menginap User

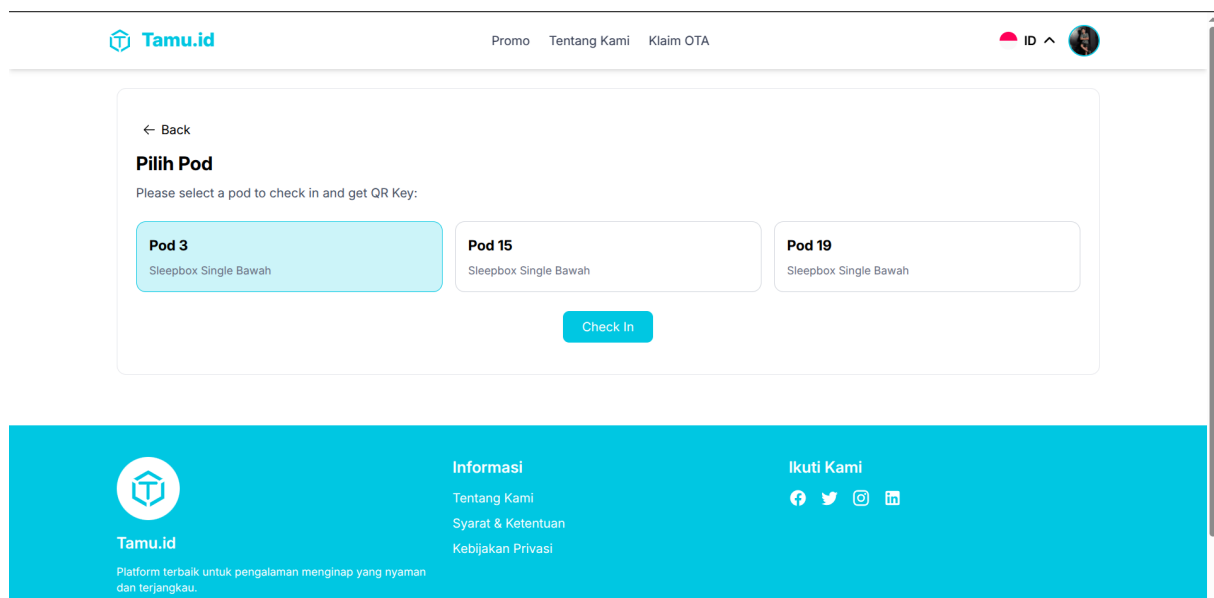
Setelah reservasi berhasil, aplikasi akan memandu pengguna melalui proses menginap yang sepenuhnya digital. Dari proses *check-in*, pemilihan pod, hingga mendapatkan kunci

digital, semuanya dirancang untuk memberikan pengalaman yang modern, mandiri, dan aman kepada pengguna.

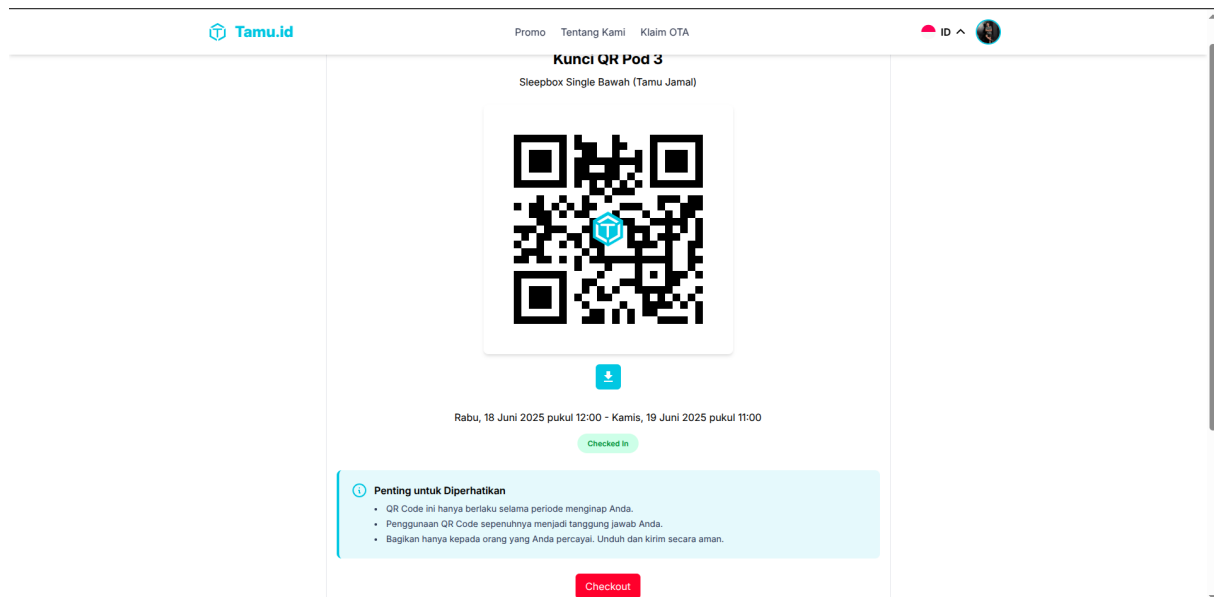


Gambar 4.8 Tampilan List Reservasi User

Pada halaman daftar reservasi (Gambar 4.8), pengguna dapat memulai proses *check-in* saat waktunya tiba. Setelah itu, pengguna akan memilih pod spesifik yang tersedia (Gambar 4.9), sebuah fitur yang memberikan fleksibilitas lebih. Puncak dari proses ini adalah penerbitan kunci pod digital.



Gambar 4.9 Tampilan Proses Pilih Pod User

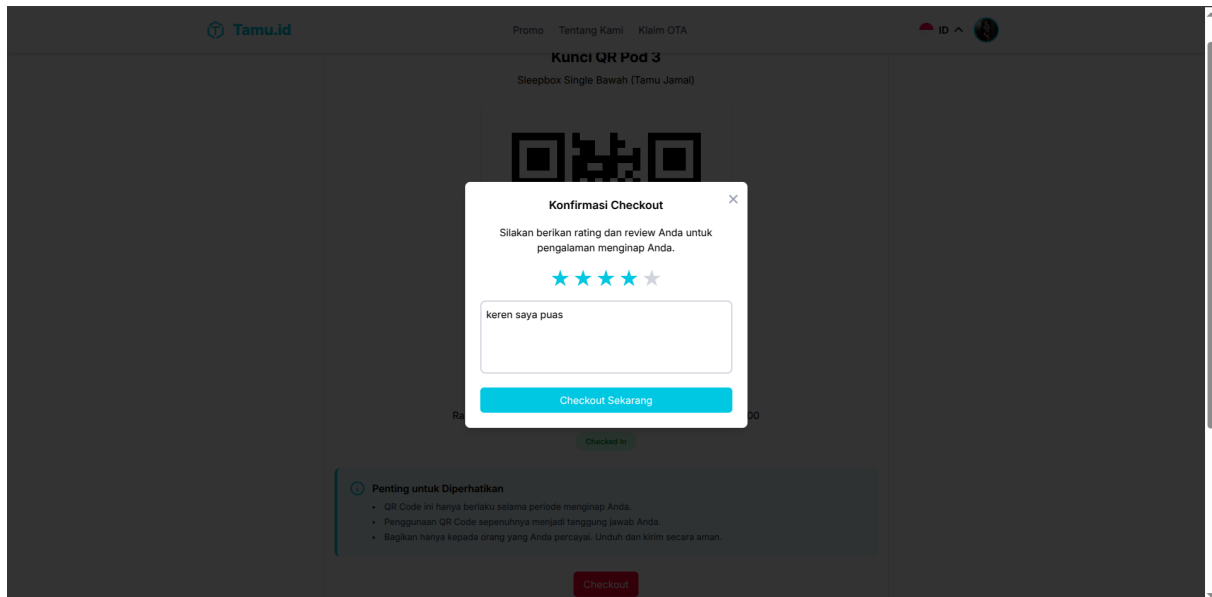


Gambar 4.10 Tampilan Kunci Pod User

Kunci pod (Gambar 4.10) bukanlah kode QR statis, melainkan kunci dinamis yang keamanannya dijaga ketat. Gambar 4.11 menunjukkan implementasi backend dari sistem regenerasi kunci ini. Sistem ini menggunakan struktur data khusus yang dimodifikasi menyerupai *linked-list* yang dipadukan dengan *Unix timestamp*. Setiap kali kunci diminta, sistem akan menghasilkan nilai baru yang valid hanya untuk periode waktu tertentu, sehingga kunci sebelumnya menjadi tidak valid. Mekanisme rotasi kunci ini secara signifikan meningkatkan keamanan dengan mempersulit duplikasi atau penyalahgunaan akses, di akhir masa menginap, pengguna melakukan *checkout* dan diundang untuk memberikan ulasan (Gambar 4.12), melengkapi seluruh siklus pengalaman menginap.

```
hotel = Hotel.query.filter_by(id=pod.pod_type.hotel_id).first()
hotel_timezone = hotel.timezone if hotel and hotel.timezone else "Asia/Jakarta"
checkout_datetime = datetime.combine(booking.check_out, hotel.checkout_time)
hotel_tz = pytz.timezone(hotel_timezone)
localized_checkout = hotel_tz.localize(checkout_datetime)
current_expired = int(localized_checkout.timestamp())
qrcode = QRCode.query.filter_by(pod_id=pod.id).first()
next_code = qr_key_generator()
next_code_image = generate_qr(next_code)
qrcode.current_code=qrcode.next_code
qrcode.current_code_image=qrcode.next_code_image
qrcode.next_code=next_code
qrcode.next_code_image=next_code_image
qrcode.current_expired = current_expired
qrcode.booking_id = booking.id
booking.status = BookingStatusEnum.CHECKED_IN
pod.status = PodStatusEnum.OCCUPIED
```

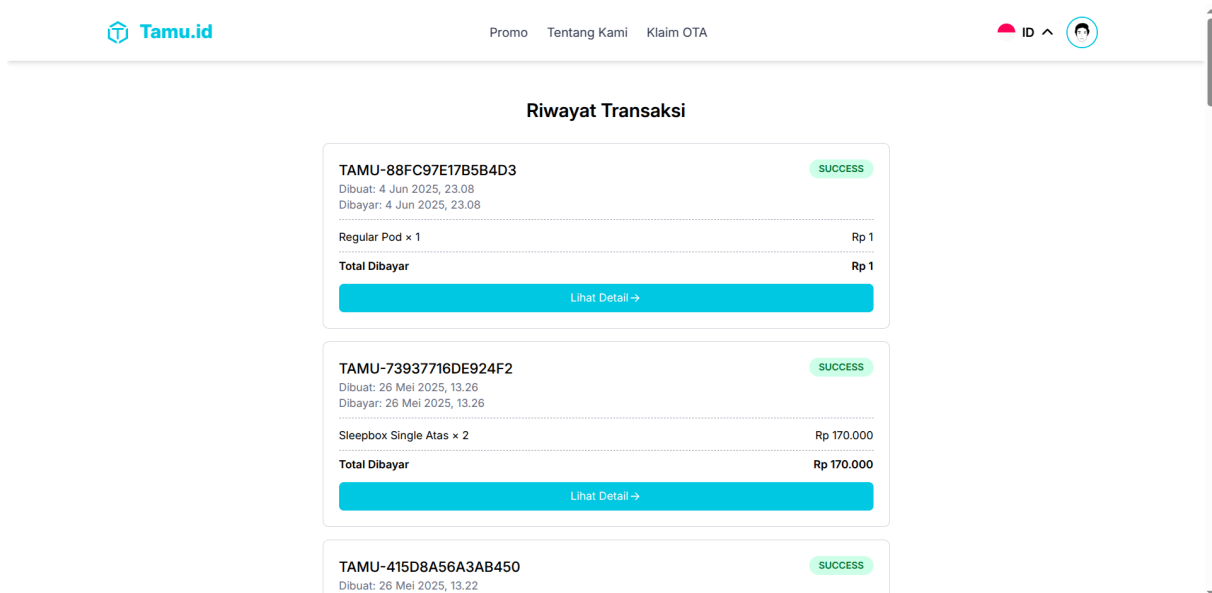
Gambar 4.11 Implementasi Rotasi dan Regenerasi Kunci Pod



Gambar 4.12 Tampilan Checkout dan Memberikan Ulasan User

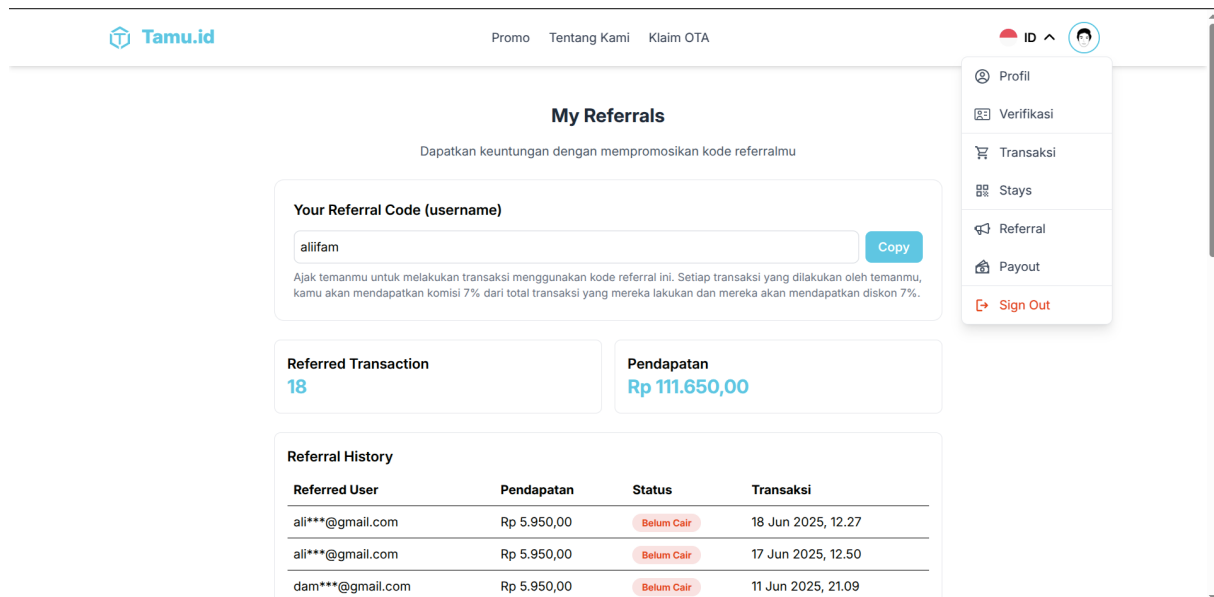
D. Transaksi dan Sistem Loyalitas User

Untuk memberikan transparansi finansial dan mendorong keterlibatan pengguna, Tamu.id menyediakan riwayat transaksi yang detail serta program loyalitas berbasis referral. Fitur ini mewajibkan verifikasi identitas untuk menjaga keamanan dan mencegah penyalahgunaan.

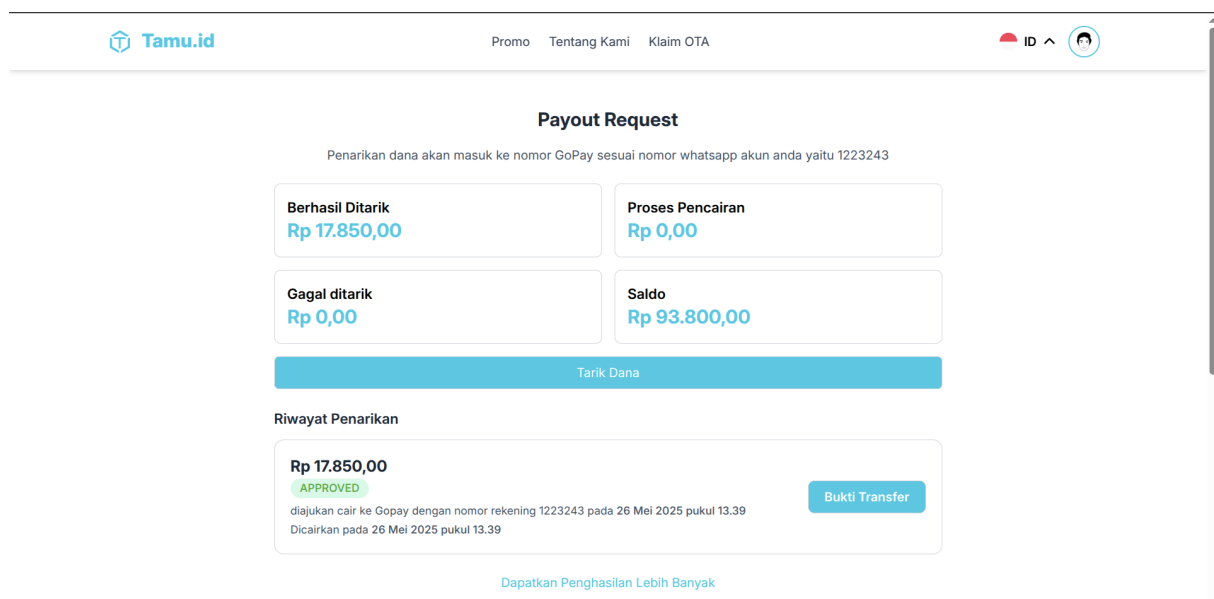


Gambar 4.13 Tampilan list Transaksi User

Pengguna dapat mengakses riwayat lengkap semua transaksi yang pernah dilakukan melalui halaman daftar transaksi (Gambar 4.13). Halaman ini menyajikan ringkasan setiap reservasi, dan pengguna dapat melihat rinciannya untuk memastikan transparansi penuh. Fitur ini penting untuk membangun kepercayaan pengguna terhadap platform.



Gambar 4.14 Tampilan Halaman Referral User



Gambar 4.15 Tampilan Halaman Payout User

Verifikasi Identitas APPROVED

Verifikasi identitasmu lebih awal untuk checkin lebih cepat

Yeay Selamat! Pengajuan verifikasi identitas anda telah disetujui pada 15 Mei 2025 pukul 00:52

Foto Kartu Identitas



Foto Selfie



ID Type

KTP
v

ID Number

3216010409030003

Gambar 4.16 Tampilan Halaman Verifikasi Identitas User

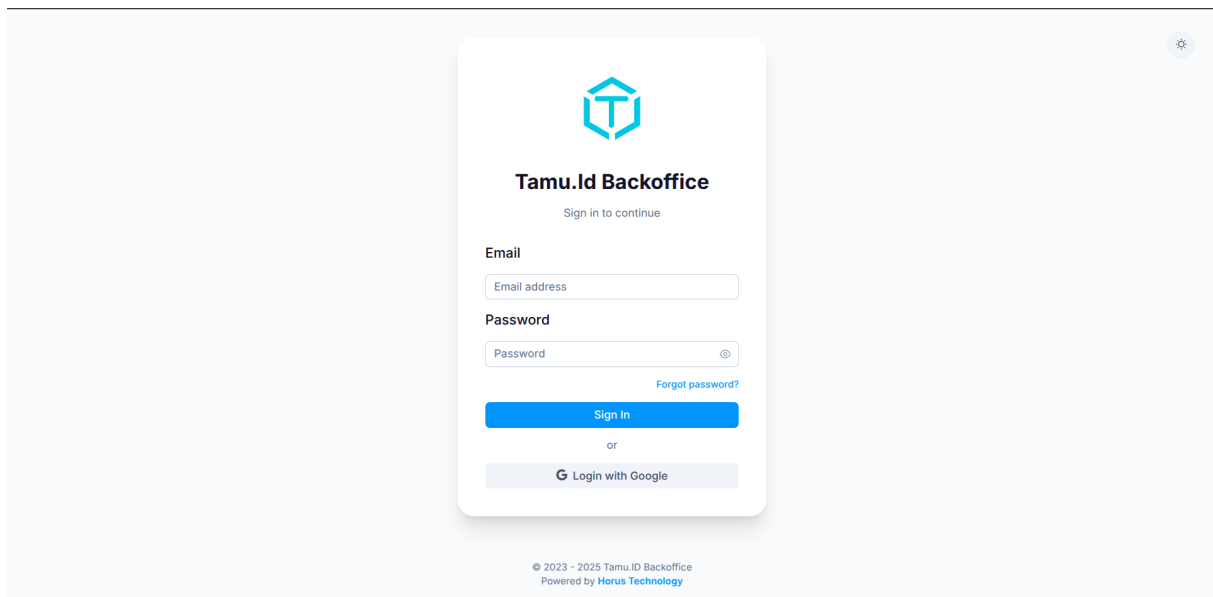
Sistem loyalitas memungkinkan pengguna untuk mendapatkan komisi dengan membagikan kode referral mereka (Gambar 4.14). Namun, untuk dapat menarik komisi yang terkumpul melalui halaman *payout* (Gambar 4.15), pengguna diwajibkan untuk melakukan verifikasi identitas (KYC) terlebih dahulu (Gambar 4.16). Persyaratan KYC ini merupakan langkah keamanan penting untuk memastikan bahwa semua peserta program adalah pengguna yang sah dan untuk mencegah aktivitas penipuan. Dengan demikian, program loyalitas tidak hanya mendorong akuisisi pengguna baru tetapi juga menjaga integritas ekosistem.

4.1.2 Halaman Admin

Halaman Admin atau Admin Cabang merupakan antarmuka yang dirancang khusus bagi staf administrasi untuk mengelola seluruh data dan operasional pada tingkat cabang secara mandiri. Halaman ini memfasilitasi berbagai tugas, mulai dari autentikasi, manajemen sumber daya spesifik cabang, hingga pengawasan aktivitas harian. Dengan adanya peran Admin Cabang, manajemen operasional dapat didesentralisasi, memungkinkan setiap cabang beroperasi secara lebih lincah dan responsif. Berikut adalah implementasi dari halaman Admin yang telah dikembangkan.

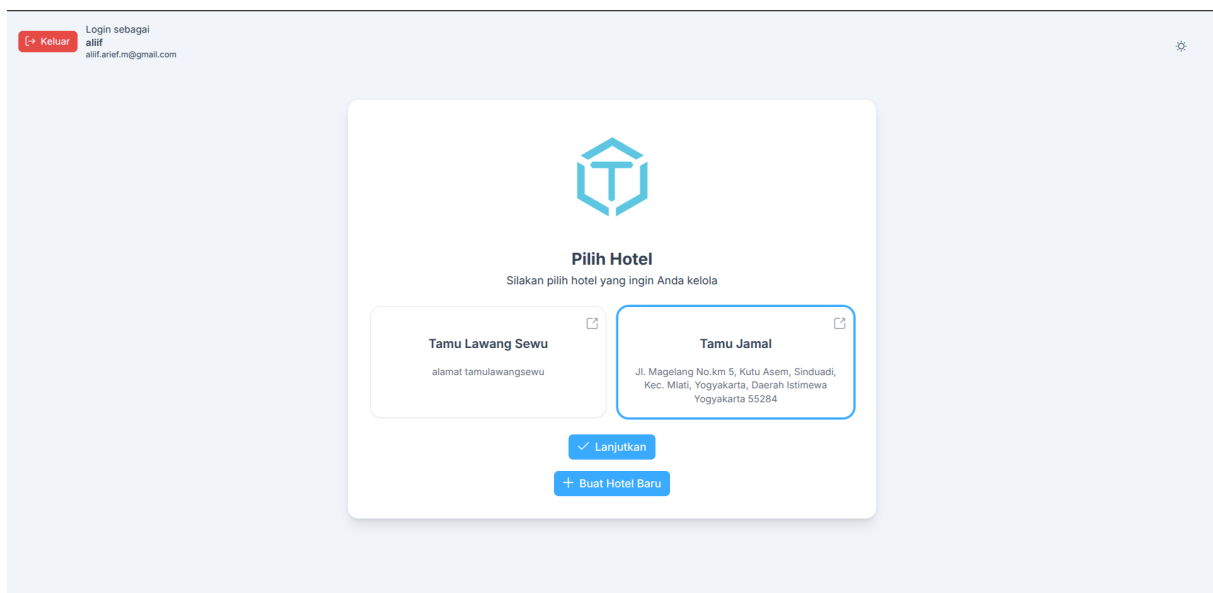
A. Login, Pilih Cabang, Dashboard, dan Pengaturan Akun

Alur kerja awal bagi Admin dirancang untuk memastikan keamanan akses dan ketepatan dalam pengelolaan cabang. Proses ini dimulai dari autentikasi, pemilihan cabang yang akan dikelola, hingga penyajian data performa melalui dasbor. Setiap langkah ini penting untuk memberikan Admin pandangan yang relevan dan data yang akurat sesuai dengan lingkup tanggung jawabnya.



Gambar 4.17 Tampilan Halaman Login Admin

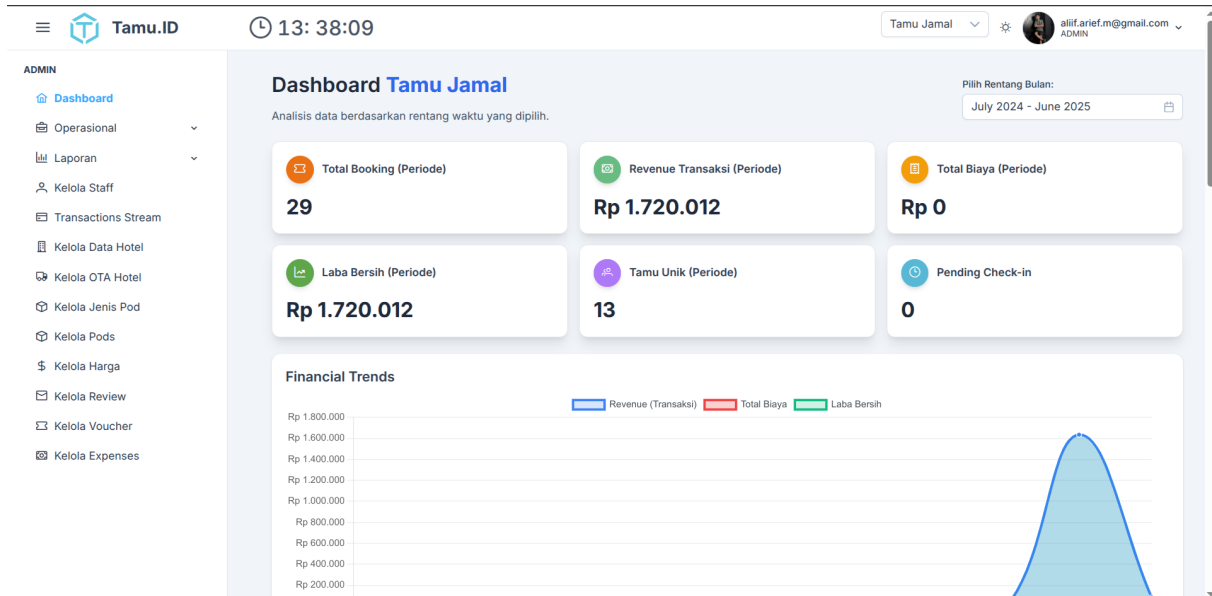
Pada Gambar 4.17 ditunjukkan halaman login, yang merupakan gerbang akses pertama bagi Admin. Untuk menjamin keamanan, sistem menyediakan dua metode autentikasi yang fleksibel, yaitu kombinasi email dan kata sandi atau melalui akun Google (SSO). Pilihan metode ini memberikan kemudahan sekaligus menjaga standar keamanan yang tinggi untuk melindungi data operasional cabang. Halaman ini menjadi garda terdepan dalam memastikan hanya pengguna terotorisasi yang dapat mengakses sistem.



Gambar 4.18 Tampilan Halaman Pilih Cabang Admin

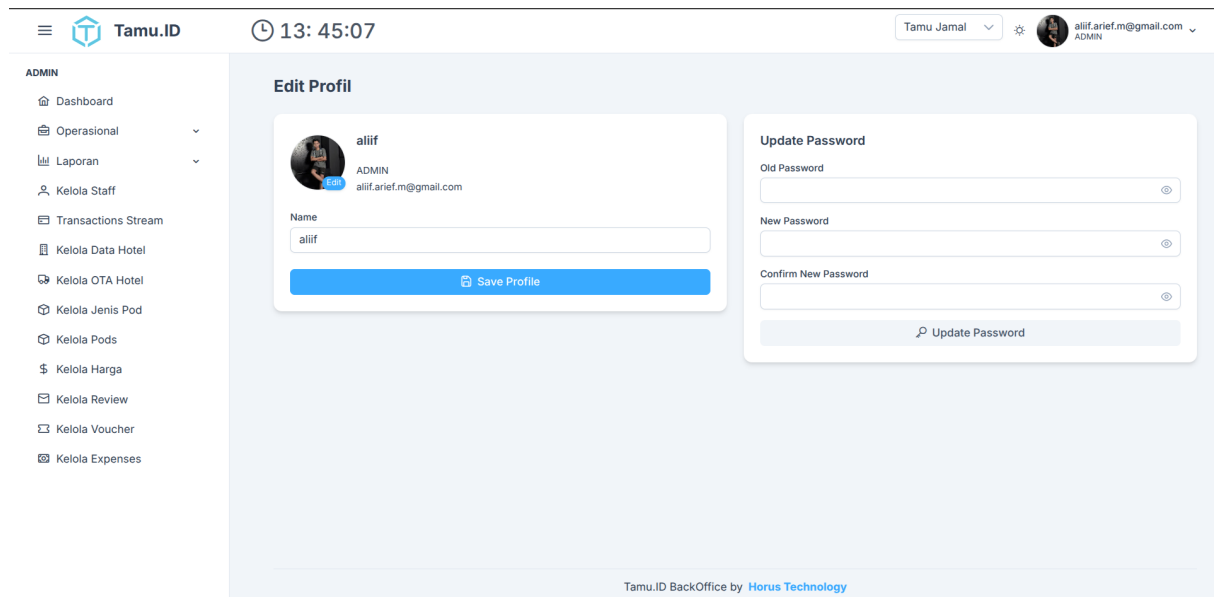
Setelah berhasil login, Admin diarahkan ke halaman pemilihan cabang seperti terlihat pada Gambar 4.18. Halaman ini muncul karena seorang Admin dapat ditugaskan untuk

mengelola lebih dari satu cabang. Sistem secara dinamis akan menampilkan daftar cabang yang sesuai dengan hak akses Admin tersebut. Dengan memilih salah satu cabang, Admin menetapkan konteks kerjanya, sehingga semua data dan fungsionalitas yang diakses selanjutnya akan spesifik untuk cabang yang dipilih.



Gambar 4.19 Tampilan Halaman Dasbor Admin Cabang

Dasbor utama cabang (Gambar 4.19) menyajikan informasi garis besar mengenai kondisi cabang yang sedang dikelola. Di halaman ini, Admin dapat memantau metrik kunci seperti ringkasan pendapatan, jumlah tamu yang sedang menginap, dan aktivitas reservasi terbaru. Selain itu, terdapat pula fitur analisis okupansi yang menampilkan data hunian secara **real-time**, perkiraan untuk hari-hari mendatang, serta performa historis, yang sangat membantu Admin dalam membuat keputusan strategis terkait harga dan ketersediaan pod.

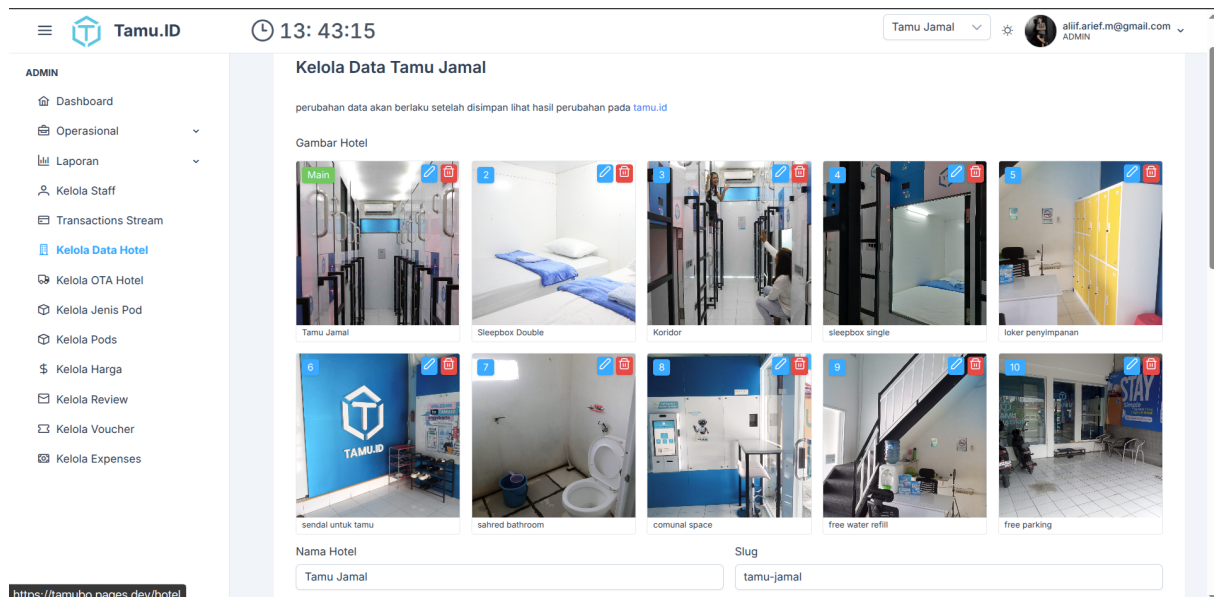


Gambar 4.20 Tampilan Halaman Profil Admin Cabang

Terakhir, pada halaman profil yang ditampilkan di Gambar 4.20, Admin dapat mengelola informasi akun pribadinya. Fitur ini memungkinkan Admin untuk melihat dan memperbarui data diri seperti nama, email, dan nomor telepon. Kemampuan untuk mengubah informasi profil secara mandiri penting untuk memastikan data kontak Admin selalu akurat dan terkini. Hal ini juga mendukung aspek keamanan akun dengan memungkinkan pembaruan informasi login jika diperlukan.

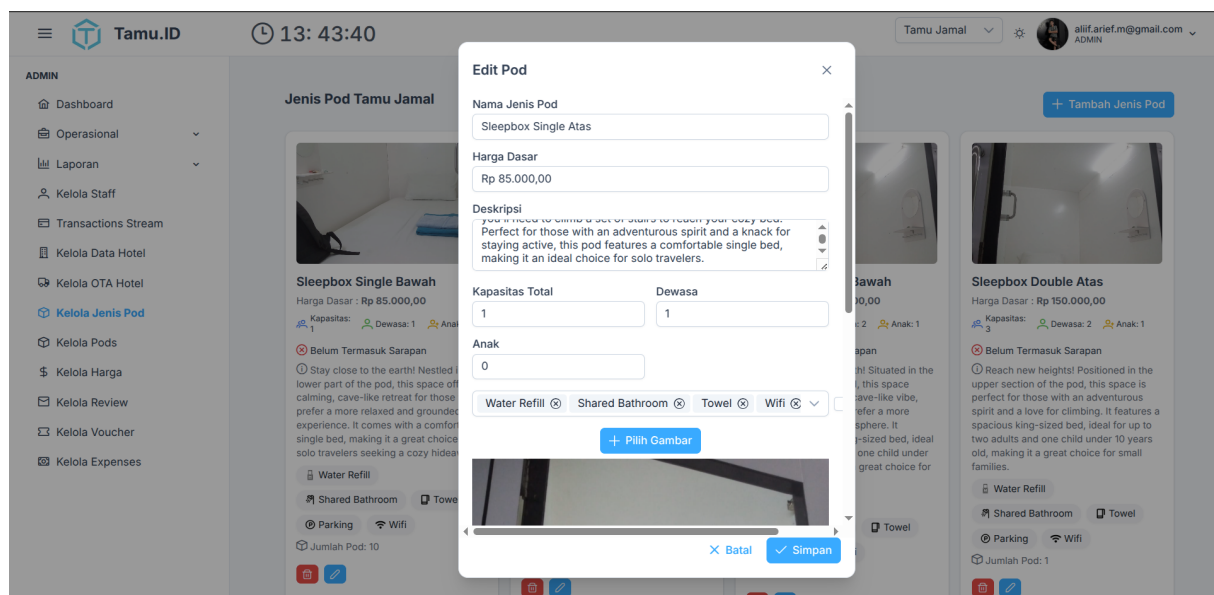
B. Kelola Data Cabang

Admin memiliki wewenang penuh untuk mengelola semua data master yang terkait dengan cabang yang ditanganinya. Pengelolaan ini mencakup pembaruan informasi umum cabang, pengaturan kamar atau pod, manajemen staf, hingga pembuatan promosi lokal. Kemampuan ini memastikan bahwa setiap cabang dapat menyesuaikan penawarannya dengan kondisi pasar lokal dan menjaga akurasi data yang ditampilkan kepada pengguna.

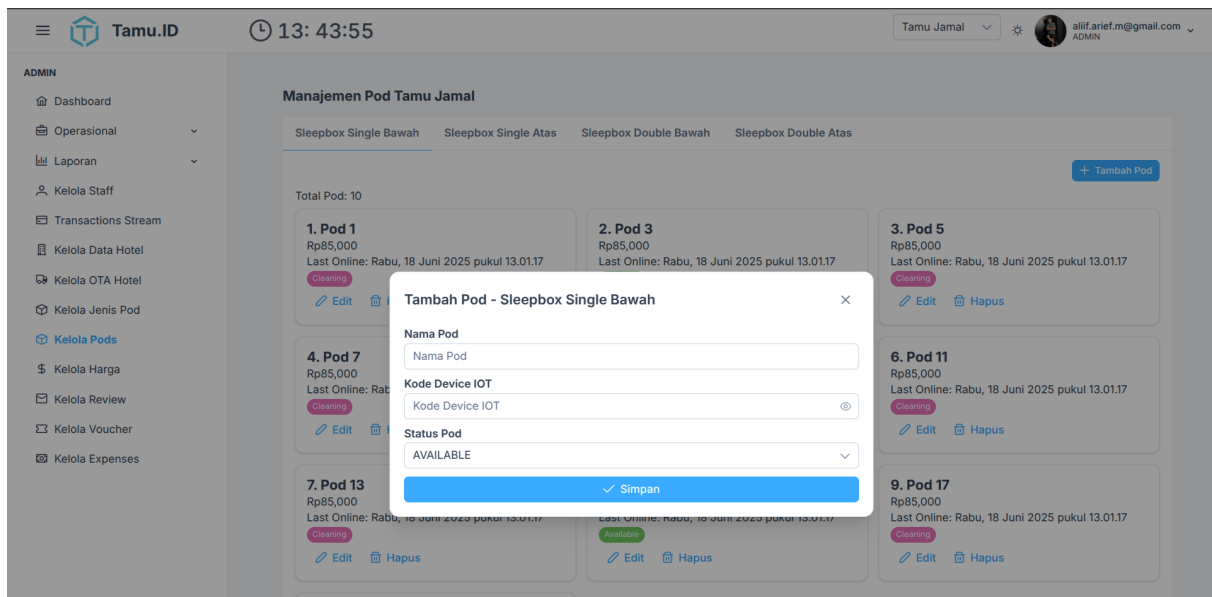


Gambar 4.21 Tampilan Admin Kelola Data Cabang

Pada halaman kelola data cabang (Gambar 4.21), Admin dapat mengubah informasi fundamental seperti nama, alamat, kontak, dan deskripsi cabang. Selain informasi umum, aset utama cabang yaitu pod juga dapat dikelola secara mendetail. Admin dapat mendefinisikan "Jenis Pod" terlebih dahulu (Gambar 4.22), misalnya berdasarkan ukuran atau fasilitas, lalu menambahkan unit-unit "Pod" individual ke dalam setiap jenis tersebut (Gambar 4.23). Struktur pengelolaan berjenjang ini memastikan inventaris kamar selalu terorganisir dan akurat.

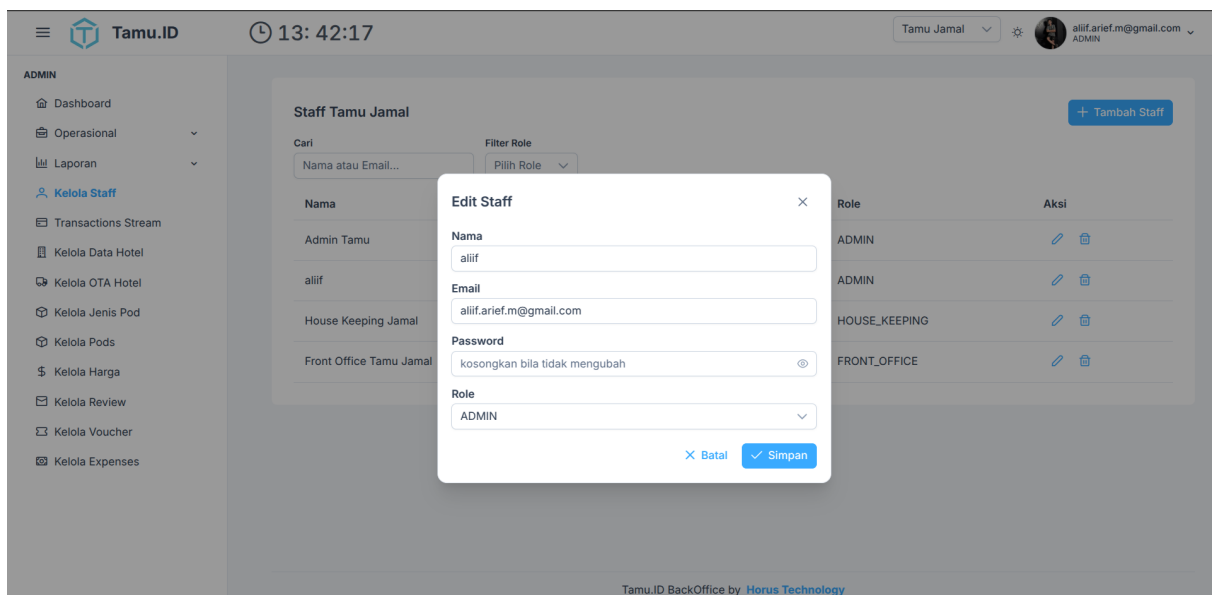


Gambar 4.22 Tampilan Admin Kelola Jenis Pod Cabang

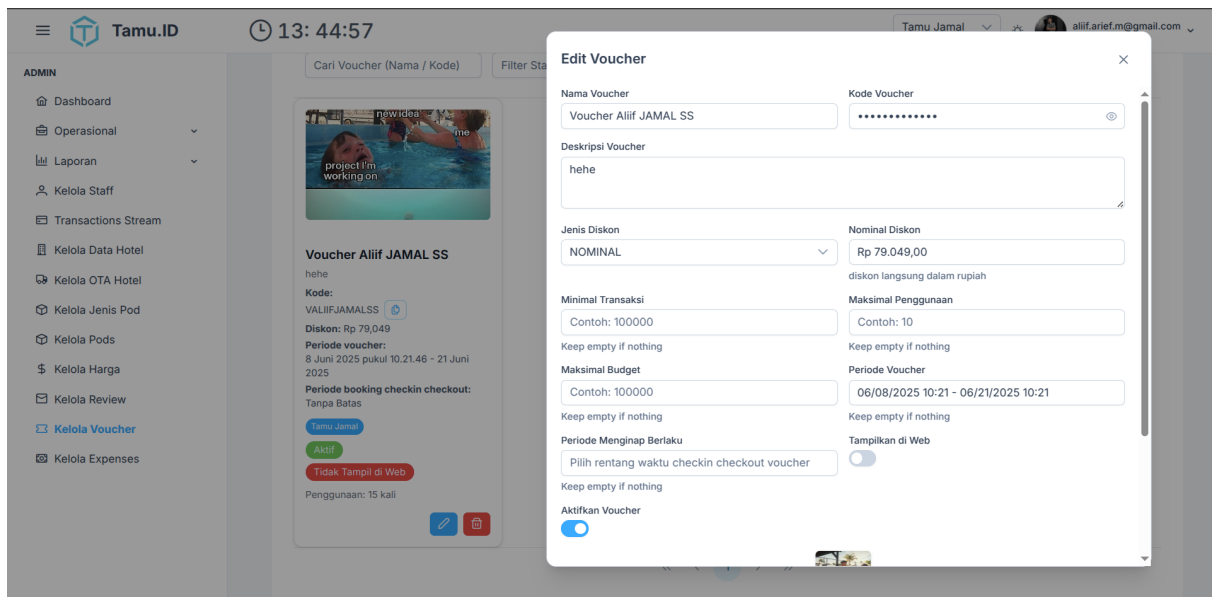


Gambar 4.23 Tampilan Admin Kelola Pod Cabang

Setiap cabang juga diberikan keleluasaan untuk mengelola sumber daya manusianya sendiri, seperti yang ditunjukkan pada Gambar 4.24. Seorang Admin dengan peran yang lebih tinggi dapat menambah, mengubah, atau menghapus akun staf lain di dalam cabang yang sama. Selain itu, untuk meningkatkan daya saing lokal, Admin dapat membuat dan mengelola voucher promosi yang hanya berlaku di cabangnya (Gambar 4.25). Fleksibilitas ini memungkinkan setiap cabang untuk merancang strategi pemasaran dan pengelolaan staf yang paling sesuai dengan kebutuhan unik mereka.



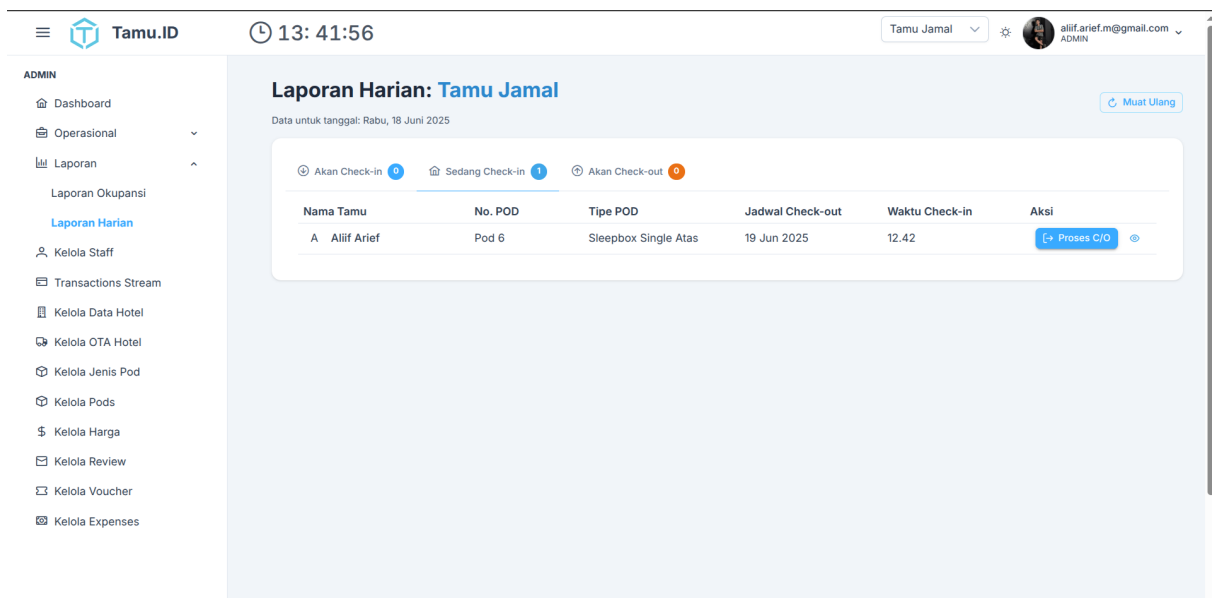
Gambar 4.24 Tampilan Admin Kelola Staff Cabang



Gambar 4.25 Tampilan Admin Kelola Voucher Cabang

C. Operasional Harian Cabang

Fungsi inti dari halaman Admin adalah untuk mendukung dan mempermudah operasional harian cabang. Fitur-fitur di bagian ini dirancang untuk memberikan visibilitas penuh terhadap pergerakan tamu, status kebersihan kamar, dan manajemen pemesanan, baik yang berasal dari platform Tamu.id maupun dari kanal eksternal.



Gambar 4.26 Tampilan Halaman Laporan Harian Admin Cabang

Untuk memantau aktivitas tamu, Admin dapat mengakses laporan harian seperti pada Gambar 4.26. Laporan ini memberikan informasi penting mengenai daftar tamu yang akan *check-in*, yang sedang menginap, dan yang akan *check-out* pada hari tersebut. Dari daftar

tamu yang sedang menginap, Admin memiliki opsi untuk melakukan tindakan operasional seperti memindahkan tamu ke pod lain jika terjadi kendala atau atas permintaan tamu. Seluruh riwayat perpindahan pod ini dicatat oleh sistem untuk keperluan audit dan pelacakan.

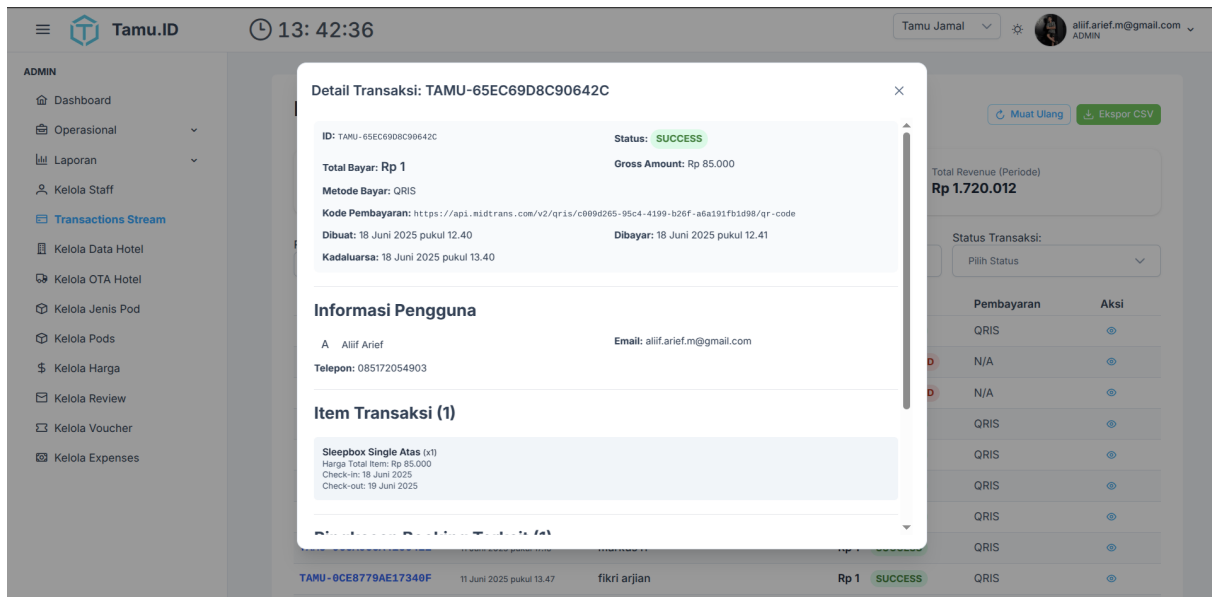
Tipe POD	Check In	Check Out	Petugas Kebersihan	Status Kebersihan	Aksi
Sleepbox Single Bawah	11 Juni 2025	12 Juni 2025	Belum Dibersihkan	Belum Dibersihkan	
Sleepbox Single Bawah	11 Juni 2025	12 Juni 2025	Belum Dibersihkan	Belum Dibersihkan	
Sleepbox Single Bawah	11 Juni 2025	12 Juni 2025	Belum Dibersihkan	Belum Dibersihkan	
Sleepbox Single Bawah	11 Juni 2025	12 Juni 2025	Belum Dibersihkan	Belum Dibersihkan	
Sleepbox Single Bawah	10 Juni 2025	11 Juni 2025	Belum Dibersihkan	Belum Dibersihkan	
Sleepbox Single Atas	10 Juni 2025	11 Juni 2025	Belum Dibersihkan	Belum Dibersihkan	

Gambar 4.27 Tampilan Halaman Daftar Pod yang Harus Dibersihkan

Manajemen kebersihan pod merupakan alur kerja penting lainnya yang difasilitasi oleh sistem. Setelah tamu melakukan *check-out*, pod akan secara otomatis muncul di daftar "Pod yang Harus Dibersihkan" (Gambar 4.27). Admin atau staf kebersihan dapat memulai proses pembersihan, di mana sistem akan menghasilkan kunci digital sementara yang valid selama tiga jam untuk mengakses pod tersebut. Setelah selesai, status pod diperbarui menjadi "Tersedia", memastikan pod siap untuk tamu berikutnya dan menjaga standar kebersihan cabang.

Gambar 4.28 Tampilan Admin Kelola Harga Dinamis Cabang

Untuk mengoptimalkan pendapatan, Admin dapat menerapkan strategi harga dinamis melalui halaman yang ditunjukkan pada Gambar 4.28. Fitur ini memungkinkan Admin untuk mengatur harga yang berbeda untuk tanggal atau periode tertentu, misalnya menaikkan harga pada akhir pekan atau hari libur. Harga yang telah diatur ini akan secara otomatis diterapkan pada saat pengguna melakukan reservasi. Kemampuan ini memberikan kendali penuh kepada cabang untuk menyesuaikan harga dengan permintaan pasar dan meningkatkan potensi pendapatan.



Gambar 4.29 Tampilan Transaksi pada Cabang

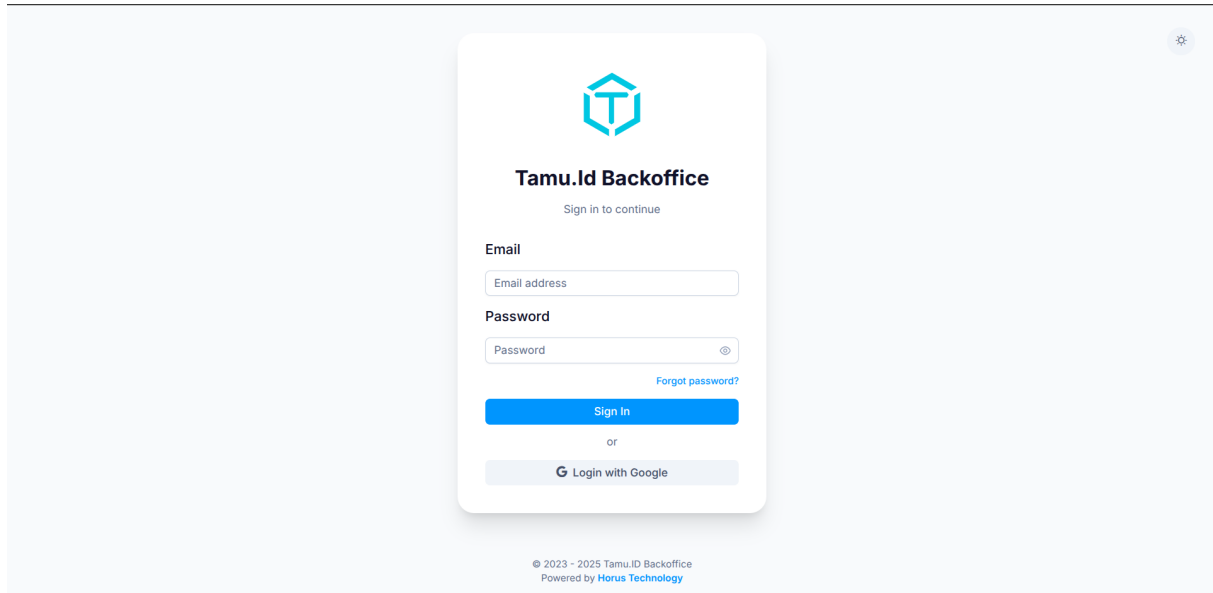
Sebagai pelengkap, Admin dapat memantau seluruh transaksi yang terjadi di cabangnya melalui halaman daftar transaksi (Gambar 4.29) untuk visibilitas finansial. Selain itu, karena integrasi dengan *Online Travel Agency* (OTA) belum berjalan otomatis, disediakan fitur untuk menginput data reservasi dari kanal OTA secara manual. Admin dapat memasukkan detail pemesanan dari platform seperti Traveloka atau Booking.com ke dalam sistem, yang kemudian akan memblokir ketersediaan pod yang relevan. Data reservasi manual ini juga dapat dilihat kembali dalam sebuah daftar terpisah untuk memastikan tidak ada pemesanan ganda dan semua data hunian tercatat secara terpusat.

4.1.3 Halaman Superadmin

Halaman *SuperAdmin* merupakan halaman yang digunakan oleh pemilik waralaba untuk mengelola operasional Tamu.id secara terpusat dan global. Halaman ini menjadi pusat kendali untuk semua aspek bisnis, mulai dari manajemen data fundamental hingga pemrosesan transaksi dan verifikasi pengguna. Dengan adanya halaman ini, pemangku kepentingan dapat memantau kinerja seluruh cabang dan mengambil keputusan strategis berdasarkan data yang terintegrasi dan akurat. Berikut adalah implementasi fungsionalitas utama pada halaman *SuperAdmin* yang telah dikembangkan.

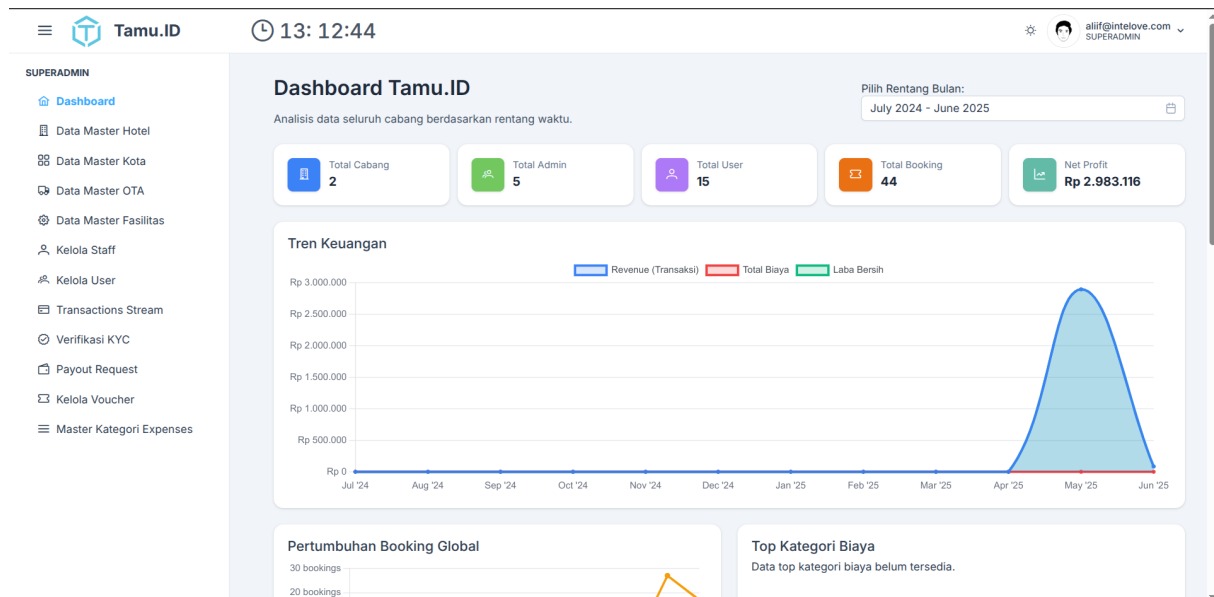
A. Login, Dashboard, dan Pengaturan Akun

Proses autentikasi dan manajemen akun merupakan fondasi keamanan dan personalisasi sistem bagi Superadmin. Fitur-fitur ini dirancang untuk memastikan bahwa hanya pengguna yang berwenang yang dapat mengakses data sensitif dan operasional. Selain itu, Superadmin diberikan kendali penuh atas informasi akun pribadinya untuk menjaga keamanan dan relevansi data.



Gambar 4.30 Tampilan Halaman Login Superadmin

Pada Gambar 4.30 diperlihatkan antarmuka halaman login yang berfungsi sebagai gerbang utama bagi Superadmin untuk mengakses sistem. Untuk masuk, Superadmin harus memasukkan kredensial yang valid, yaitu nama pengguna dan kata sandi, sebagai lapisan keamanan pertama. Sebagai alternatif, sistem juga menyediakan opsi autentikasi melalui akun Google, yang menawarkan kemudahan dan keamanan tambahan melalui protokol OAuth. Halaman ini secara esensial melindungi seluruh data dan fungsionalitas sistem dari akses yang tidak sah.



Gambar 4.31 Tampilan Halaman Dashboard Superadmin

Setelah berhasil login, Superadmin akan diarahkan ke halaman dasbor seperti yang ditunjukkan pada Gambar 4.31. Dasbor ini menyajikan gambaran umum dan rangkuman data vital dari seluruh cabang yang beroperasi dalam bentuk visual yang informatif. Metrik kunci seperti total transaksi, jumlah pengguna baru, dan tingkat hunian ditampilkan secara jelas untuk memfasilitasi pemantauan kinerja secara *real-time*. Berdasarkan data yang tersaji di sini, manajemen pusat dapat membuat keputusan strategis yang terinformasi dan responsif terhadap dinamika pasar.

The screenshot shows the 'Edit Profil' (Edit Profile) page for the Superadmin user 'Alif Arief'. The page has a sidebar menu identical to the dashboard. The main content area is divided into two sections. The left section, titled 'Edit Profil', shows the user's profile information: 'Alif Arief', 'SUPERADMIN', and 'alif@intelove.com'. There is an 'Edit' button next to the profile picture and a 'Save Profile' button at the bottom. The right section, titled 'Update Password', contains three input fields: 'Old Password', 'New Password', and 'Confirm New Password', each with a toggle for password visibility. An 'Update Password' button is located at the bottom of this section. The footer of the page reads 'Tamu.ID BackOffice by Horus Technology'.

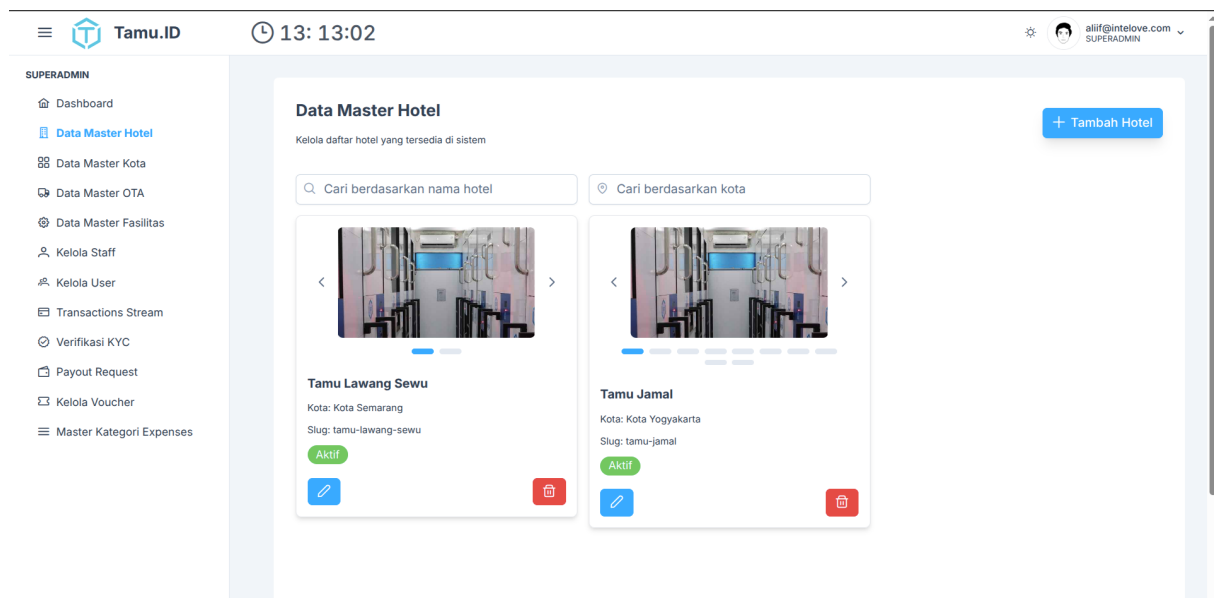
Gambar 4.32 Tampilan Halaman Pengaturan Akun Superadmin

Pada Gambar 4.32 terlihat halaman pengaturan akun yang memberikan Superadmin

kontrol penuh atas informasi profilnya. Melalui halaman ini, Superadmin dapat melakukan pembaruan detail pribadi serta mengubah kata sandi secara berkala untuk meningkatkan keamanan akun. Fitur ini sangat penting untuk memastikan bahwa informasi kontak dan kredensial Superadmin selalu akurat dan terlindungi. Dengan demikian, Superadmin memiliki fleksibilitas untuk mengelola akunnya secara mandiri sesuai dengan kebutuhan.

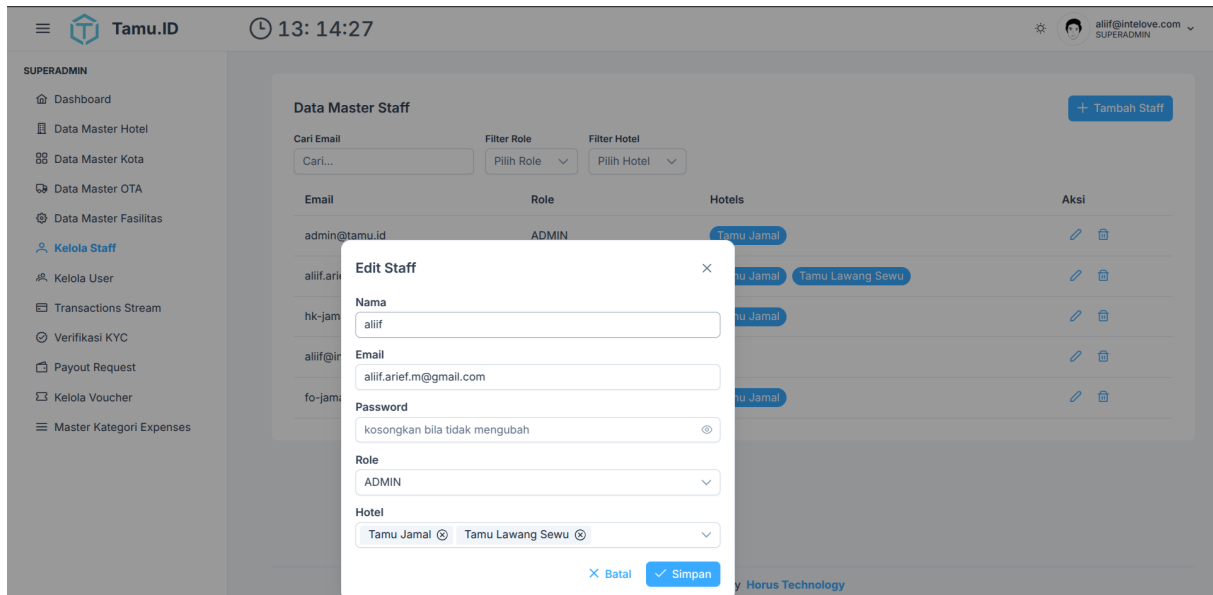
B. Kelola Data Master

Pengelolaan data master adalah fungsi krusial yang memungkinkan Superadmin untuk mengatur dan menjaga konsistensi informasi fundamental yang digunakan di seluruh sistem Tamu.id. Modul ini mencakup pengelolaan berbagai entitas inti seperti data kota, fasilitas, *Online Travel Agent* (OTA), cabang, staf, dan pengguna. Dengan adanya pengelolaan terpusat, Superadmin memastikan bahwa semua cabang beroperasi dengan standar dan data yang seragam, sehingga mengurangi redundansi dan potensi kesalahan data di seluruh platform.



Gambar 4.33 Tampilan Halaman Pengaturan Cabang Superadmin

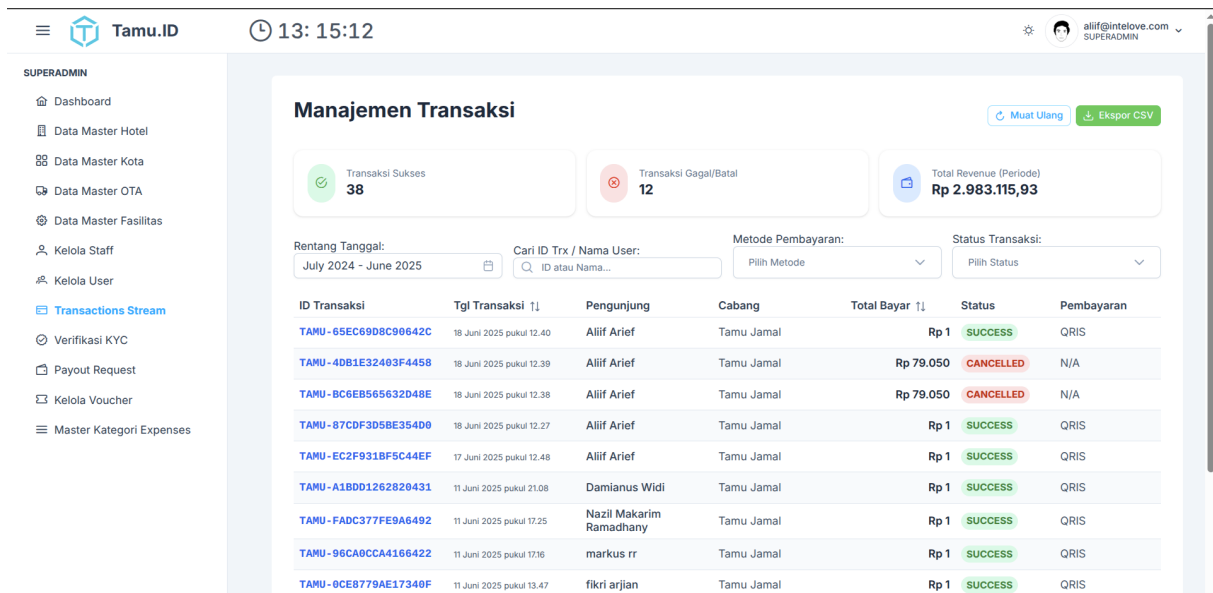
Salah satu contoh antarmuka pengelolaan data master adalah halaman pengaturan cabang, seperti yang terlihat pada Gambar 4.33. Halaman ini memberikan Superadmin kemampuan untuk mengelola informasi terkait setiap cabang operasional, termasuk menambahkan cabang baru, memperbarui detail lokasi, dan mengatur konfigurasi spesifik lainnya. Antarmuka serupa juga digunakan untuk mengelola data staf (Gambar 4.34), yang memungkinkan Superadmin mengatur akun, peran, dan penugasan staf ke cabang tertentu. Pengelolaan data master yang terstruktur ini sangat vital untuk skalabilitas bisnis dan menjaga operasional multi-lokasi yang terorganisir dan efisien.



Gambar 4.34 Tampilan Halaman Pengaturan Staff Superadmin

C. Pengelolaan Transaksi dan Pemasaran

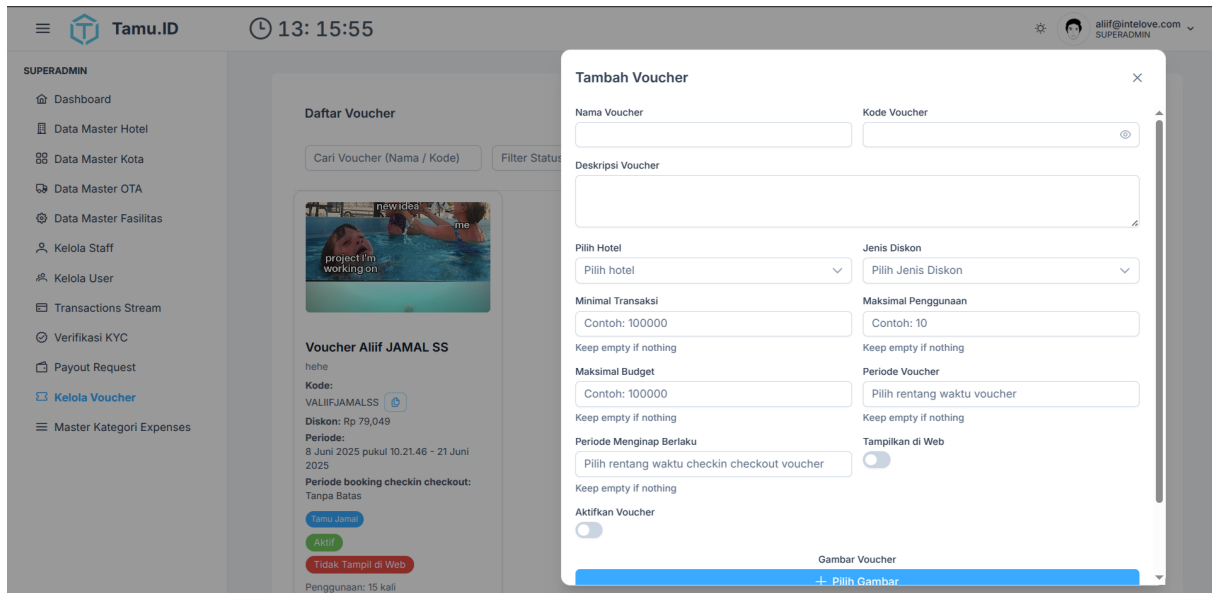
Superadmin memiliki kewenangan untuk memantau seluruh aktivitas finansial dan mengelola strategi pemasaran melalui modul khusus. Fitur ini dirancang untuk memberikan transparansi penuh terhadap semua transaksi yang terjadi serta menyediakan alat untuk meningkatkan keterlibatan dan retensi pengguna. Kemampuan untuk melihat detail transaksi dan membuat program promosi adalah kunci untuk audit finansial dan pertumbuhan bisnis.



Gambar 4.35 Tampilan Halaman List Transaksi Superadmin

Gambar 4.35 menunjukkan halaman daftar transaksi yang menyajikan ringkasan komprehensif dari semua transaksi dalam sistem. Dari halaman ini, Superadmin dapat

memantau, menyaring berdasarkan kriteria tertentu seperti tanggal atau cabang, dan melacak status setiap pembayaran. Untuk kebutuhan audit atau verifikasi, Superadmin dapat langsung melihat rincian mendalam dari setiap transaksi, yang mencakup informasi layanan, metode pembayaran, dan data relevan lainnya. Pengawasan terpusat ini esensial untuk menjaga akuntabilitas finansial dan menganalisis arus pendapatan perusahaan.



Gambar 4.36 Tampilan Halaman Pengaturan Voucher Superadmin

Untuk mendukung strategi pemasaran, Superadmin dilengkapi dengan modul pengaturan voucher seperti yang ditampilkan pada Gambar 4.36. Halaman ini berfungsi sebagai alat untuk membuat, mengelola, dan mendistribusikan berbagai program promosi atau diskon kepada pengguna. Superadmin dapat secara fleksibel menentukan kode unik, nilai diskon, periode validitas, serta syarat dan ketentuan penggunaan voucher. Fitur ini merupakan komponen penting dalam upaya menarik pelanggan baru dan meningkatkan loyalitas pelanggan yang sudah ada.

D. Proses dan Verifikasi Permintaan oleh Superadmin

Keamanan dan validitas pengguna serta kelancaran proses finansial merupakan prioritas utama. Oleh karena itu, Superadmin bertugas untuk memproses dan memverifikasi berbagai permintaan yang diajukan oleh pengguna. Proses ini mencakup verifikasi identitas (KYC) dan persetujuan penarikan dana (payout), yang keduanya dirancang untuk memastikan integritas dan keamanan ekosistem Tamu.id.

Tamu.ID

13: 15:28

aliif@intelove.com
SUPERADMIN

SUPERADMIN

- Dashboard
- Data Master Hotel
- Data Master Kota
- Data Master OTA
- Data Master Fasilitas
- Kelola Staff
- Kelola User
- Transactions Stream
- Verifikasi KYC
- Payout Request
- Kelola Voucher
- Master Kategori Expenses

Verifikasi Identitas User

Cari Nama/Email

Ketik nama atau email...

Filter Status

Semua Status

Nama	Email	Telepon	Status	Aksi
Aliif Arief	aliif.arief.m@gmail.com	085172054903	PENDING	Detail
Aliif	aliif@intelove.com	1223243	APPROVED	Detail

Tamu.ID BackOffice by [Horus Technology](#)

Gambar 4.37 Tampilan Halaman list Permintaan KYC User di Superadmin

Pada Gambar 4.37 ditampilkan daftar permintaan verifikasi identitas pengguna atau *Know Your Customer* (KYC). Halaman ini menjadi pusat bagi Superadmin untuk meninjau semua pengajuan verifikasi, di mana setiap permintaan dapat diproses untuk divalidasi lebih lanjut. Proses KYC ini diwajibkan bagi pengguna yang ingin mencairkan komisi dari program referral, sehingga mendorong pengguna untuk segera memverifikasi identitasnya. Dengan meninjau dokumen yang diunggah dan membandingkannya dengan data pengguna, Superadmin dapat menyetujui atau menolak permintaan, sebuah langkah krusial untuk mencegah penipuan dan mematuhi regulasi.

Tamu.ID

18: 46:34

aliif@intelove.com
SUPERADMIN

SUPERADMIN

- Dashboard
- Data Master Hotel
- Data Master Kota
- Data Master OTA
- Data Master Fasilitas
- Kelola Staff
- Kelola User
- Transactions Stream
- Verifikasi KYC
- Payout Request
- Kelola Voucher
- Master Kategori Expenses

Kelola Referral Pengguna

Cari Nama/Email

Ketik nama atau email...

Filter Status

Semua Status

Nama	Email	Rekening	No. Rekening	Nominal	Status	Aksi
Aliif	aliif@intelove.com	Gopay	1223243	Rp 17.850,00	APPROVED	Detail

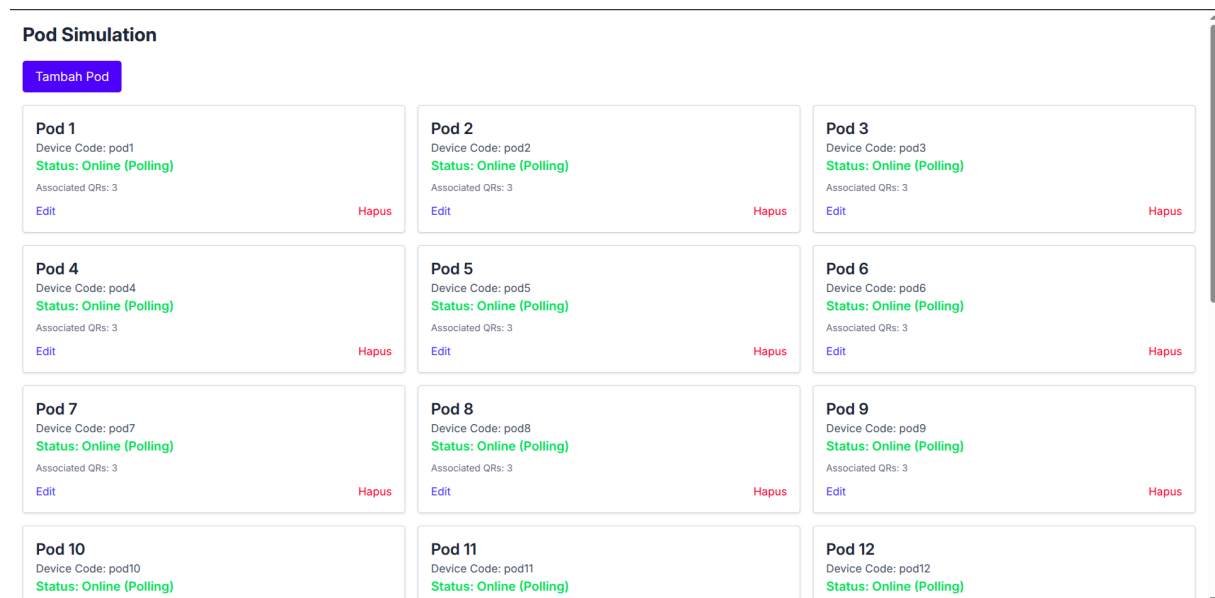
Tamu.ID BackOffice by [Horus Technology](#)

Gambar 4.38 Tampilan Halaman List Permintaan Payout User di Superadmin

Halaman daftar permintaan *payout* (Gambar 4.38) memberikan gambaran umum tentang semua permintaan penarikan dana yang diajukan oleh pengguna, khususnya yang berasal dari komisi program referral. Untuk meningkatkan partisipasi, program ini dirancang agar menguntungkan kedua belah pihak: pemilik kode referral mendapatkan komisi sebesar 7% dari total transaksi, dan pengguna kode tersebut juga menerima diskon langsung sebesar 7%. Superadmin bertugas memverifikasi setiap permintaan payout, memastikan dana tersebut sah, melakukan transfer ke rekening pengguna, dan terakhir mengunggah bukti transfer sambil memperbarui status permintaan menjadi "selesai".

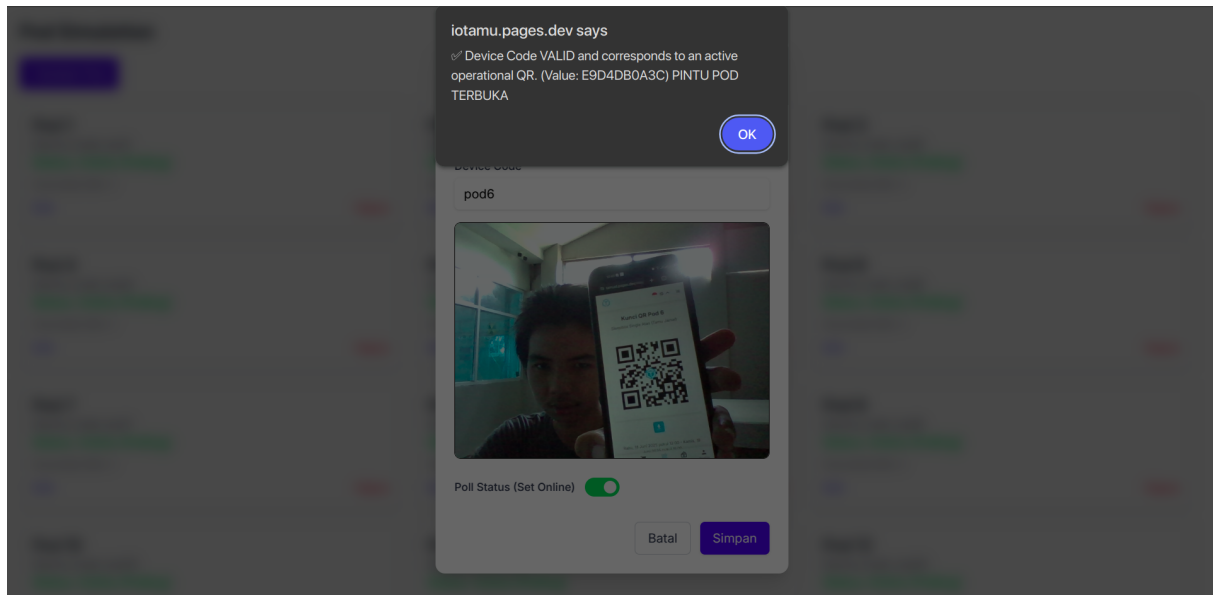
4.1.4 Halaman Simulasi Hardware

Halaman simulasi hardware adalah halaman yang ditujukan untuk mensimulasikan bagaimana integrasi antara backend dan hardware IoT dapat berjalan dengan baik.



Gambar 4.39 Tampilan List Hardware

Padaha gambar 4.39 dapat terlihat ada beberapa pod yang sudah dibuat dengan menginputkan device code untuk masing - masing pod nya sebagai identifikasi untuk pod saat terhubung dengan *Backend*.



Gambar 4.40 Tampilan QR Code Berhasil

Maka selanjutnya pada gambar 4.40 adalah tampilan setelah pengguna *Checkin* dan mendapatkan kunci berupa kode QR maka dapat discan ke pod yang ia pilih dan pod tersebut akan terbuka, namun bila pengguna salah pod maka pod tidak akan terbuka.

```
{
  "qr": [
    {
      "until": "1748670431",
      "value": "052322BED6"
    },
    {
      "until": "",
      "value": "A8982BC796"
    },
    {
      "until": "",
      "value": "tamumasterkey"
    }
  ]
}
```

Gambar 4.41 Struktur Data *Customized Linkedlist* dengan *Unix Timestamp* pada *Hardware IoT*

Pada kode 4.41 merupakan implementasi dari struktur data *Customized Linkedlist* dengan

Unix Timestamp pada *Hardware IoT* yang dapat membuat integrasi saat pengguna melakukan *Checkin* dan langsung menggunakan *QRCode* yang ia dapatkan secara *Seamless* tanpa delay dan walaupun Pod dalam kondisi *Offline* sesaat.

```
def get_pod_key():
    try:
        device_code = request.args.get("device_code")
        if not device_code:
            return jsonify({"error": "device_code is required"}), 400
        pod = Pod.query.filter_by(device_code=device_code).first()
        if not pod:
            return jsonify({"error": "Pod not found"}), 404
        pod.last_online = int(datetime.now().timestamp())
        db.session.commit()
        qrcode = QRCode.query.filter_by(pod_id=pod.id).order_by(QRCode.current_expired.desc()).first()
        response = {
            "qr": []
        }
        if qrcode:
            if qrcode.current_code:
                response["qr"].append({
                    "value": qrcode.current_code,
                    "until": str(qrcode.current_expired),
                })
            if qrcode.next_code:
                response["qr"].append({
                    "value": qrcode.next_code,
                    "until": "",
                })
        master_key = os.getenv("POD_MASTER_KEY")
        if master_key:
            response["qr"].append({
                "value": master_key,
                "until": "",
            })
        return jsonify(response), 200
    except Exception as e:
        logging.error(f"Error in get_pod_key: {e}", exc_info=True)
        return jsonify({"error": "Internal server error"}), 500
```

Gambar 4.42 Kode Implementasi Integrasi *Backend* dengan *Hardware IoT*

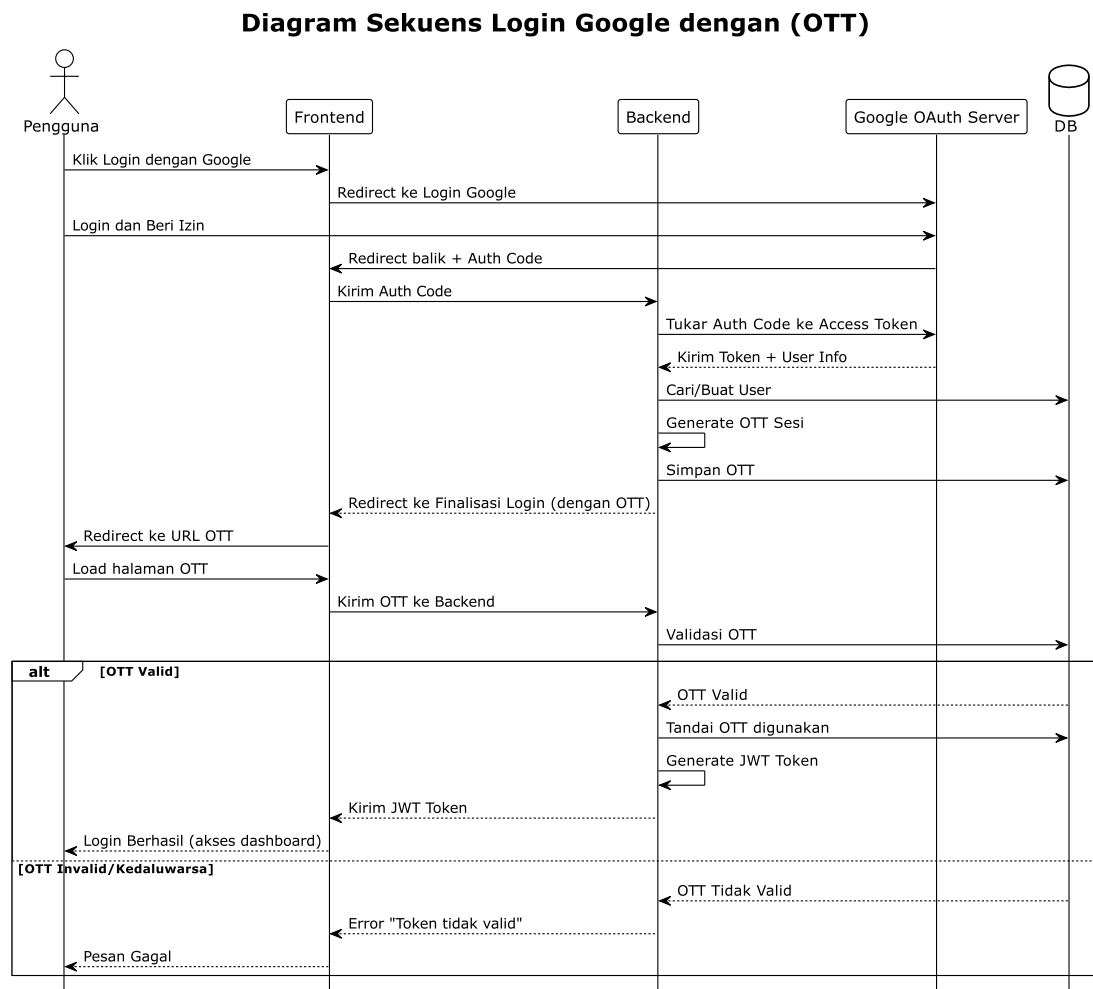
Kode 4.42 merupakan implementasi dari integrasi *Backend* dengan *Hardware IoT* yang dapat menghubungkan antara *Backend* dan *Hardware IoT* dengan baik, sehingga *Hardware IoT* selalu dapat terhubung dengan *Backend* dan *User* dapat masuk ke dalam Pod menggunakan kunci Pod yang telah didapatkan saat *Checkin*.

4.1.5 Analisis Implementasi OAuth2 Google dengan One Time Token (OTT)

Sistem autentikasi dan otorisasi modern harus menghadapi tantangan memastikan keaslian pengguna sekaligus menyediakan pengalaman yang mulus. Menggunakan sistem konvensional seperti email dan kata sandi seringkali menimbulkan hambatan, seperti pengguna harus mengingat kata sandi baru dan menunggu email verifikasi yang berpotensi masuk ke folder spam. Selain itu, pendekatan ini memakan biaya untuk layanan email dan memerlukan strategi terpisah untuk fitur lupa atau ganti kata sandi. Untuk mengatasi isu-isu ini, penulis menganalisis implementasi OAuth2 Google sebagai alternatif, namun tetap mempertahankan opsi autentikasi konvensional untuk mengakomodasi pengguna tanpa akun Google.

Oleh karena itu, penulis memadukan strategi One Time Token (OTT) untuk menjembatani

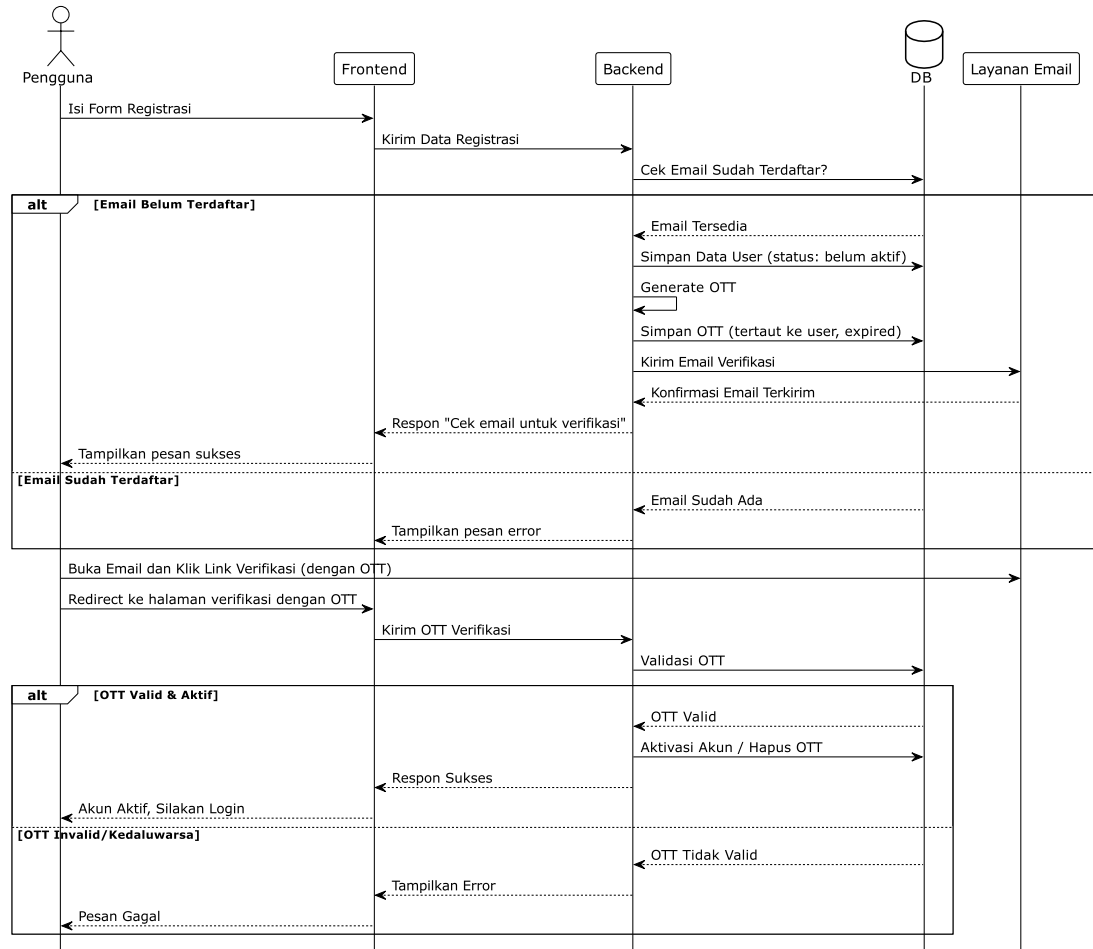
kedua sistem autentikasi ini ke dalam satu alur yang kohesif. Pendekatan ini memungkinkan pengguna untuk mendaftar atau masuk menggunakan sistem konvensional maupun langsung dengan akun Google mereka secara seamless. Diagram implementasi OAuth2 Google dengan One Time Token (OTT), yang menunjukkan alur otorisasi dan penggunaan OTT untuk finalisasi sesi, dapat dilihat pada Gambar 4.43.



Gambar 4.43 Diagram implementasi OAuth2 Google dengan One Time Token (OTT)

Implementasi One Time Token (OTT) pada pendaftaran pengguna konvensional diilustrasikan secara detail pada Gambar 4.44. Diagram ini secara spesifik menampilkan proses verifikasi email yang mengandalkan OTT untuk mengaktifkan akun pengguna baru, memberikan pengalaman registrasi yang lebih efisien dan aman.

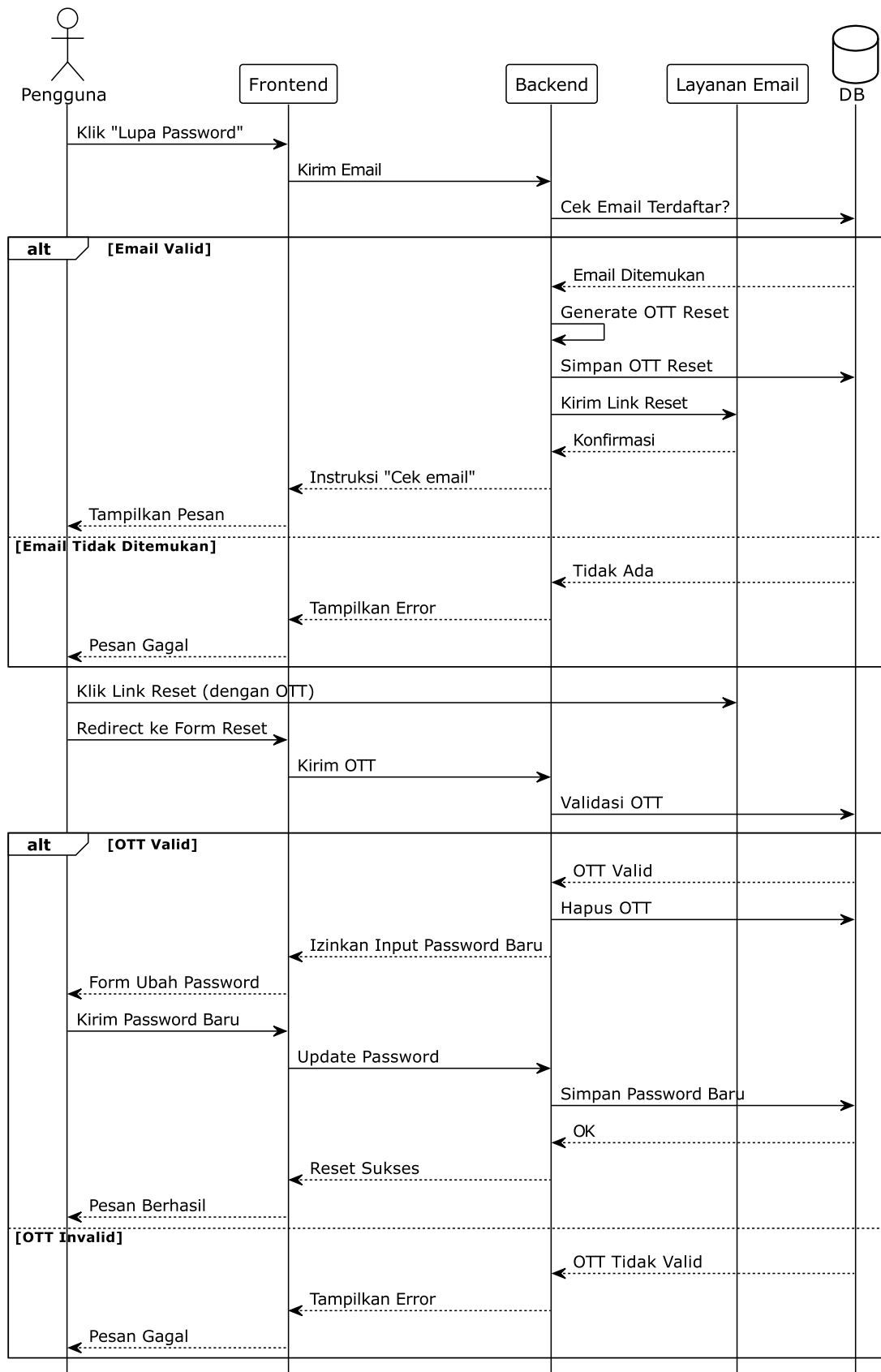
Diagram Sekuens Registrasi dengan Verifikasi Email (OTT)



Gambar 4.44 Diagram implementasi One Time Token (OTT) pada pendaftaran pengguna

OTT ini selanjutnya dipadukan dengan JWT (*JSON Web Token*), yang berfungsi sebagai produk akhir dari proses autentikasi dan menjadi kunci utama untuk otorisasi setelah pengguna berhasil masuk. Fleksibilitas OTT juga diperluas untuk mengatasi masalah lupa kata sandi. Diagram implementasi Lupa Kata Sandi dengan One Time Token (OTT) disajikan pada Gambar 4.45. Diagram ini menguraikan langkah-langkah pengguna dalam mereset kata sandi dengan aman menggunakan mekanisme OTT, memastikan keamanan dan kenyamanan di seluruh siklus hidup autentikasi.

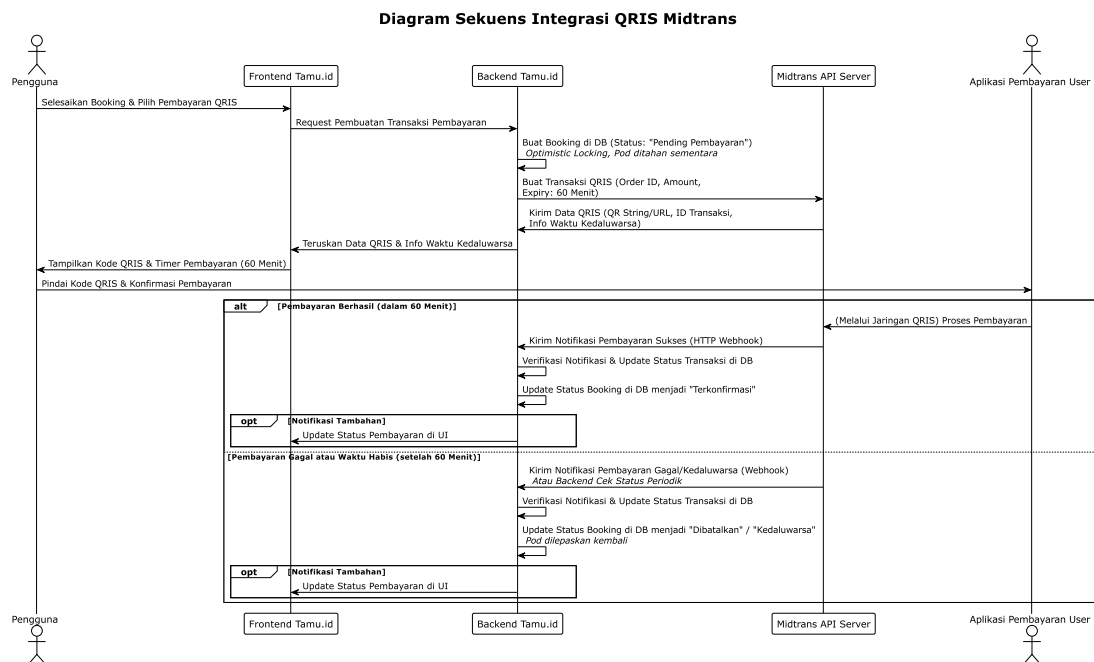
Diagram Sekuens Reset Password dengan OTT



Gambar 4.45 Diagram implementasi Lupa Kata Sandi dengan One Time Token (OTT)

4.1.6 Analisis Integrasi QRIS dengan Backend Sistem

QRIS dipilih sebagai metode pembayaran utama karena menawarkan biaya transaksi terendah (hanya 0,7% tanpa minimal transaksi) di antara semua platform pembayaran yang terintegrasi dengan payment gateway Midtrans. Keunggulan ini secara otomatis meningkatkan profitabilitas bagi bisnis dan efisiensi bagi pengguna. Selain itu, QRIS telah mencakup hampir semua kanal pembayaran di Indonesia, menghemat waktu dan sumber daya dalam proses pengembangan. Tantangan utama setelah pembayaran berhasil adalah bagaimana sistem backend dapat menerima notifikasi dan memperbarui status transaksi secara otomatis, mengingat hanya server QRIS yang pertama kali mengetahui keberhasilan pembayaran.



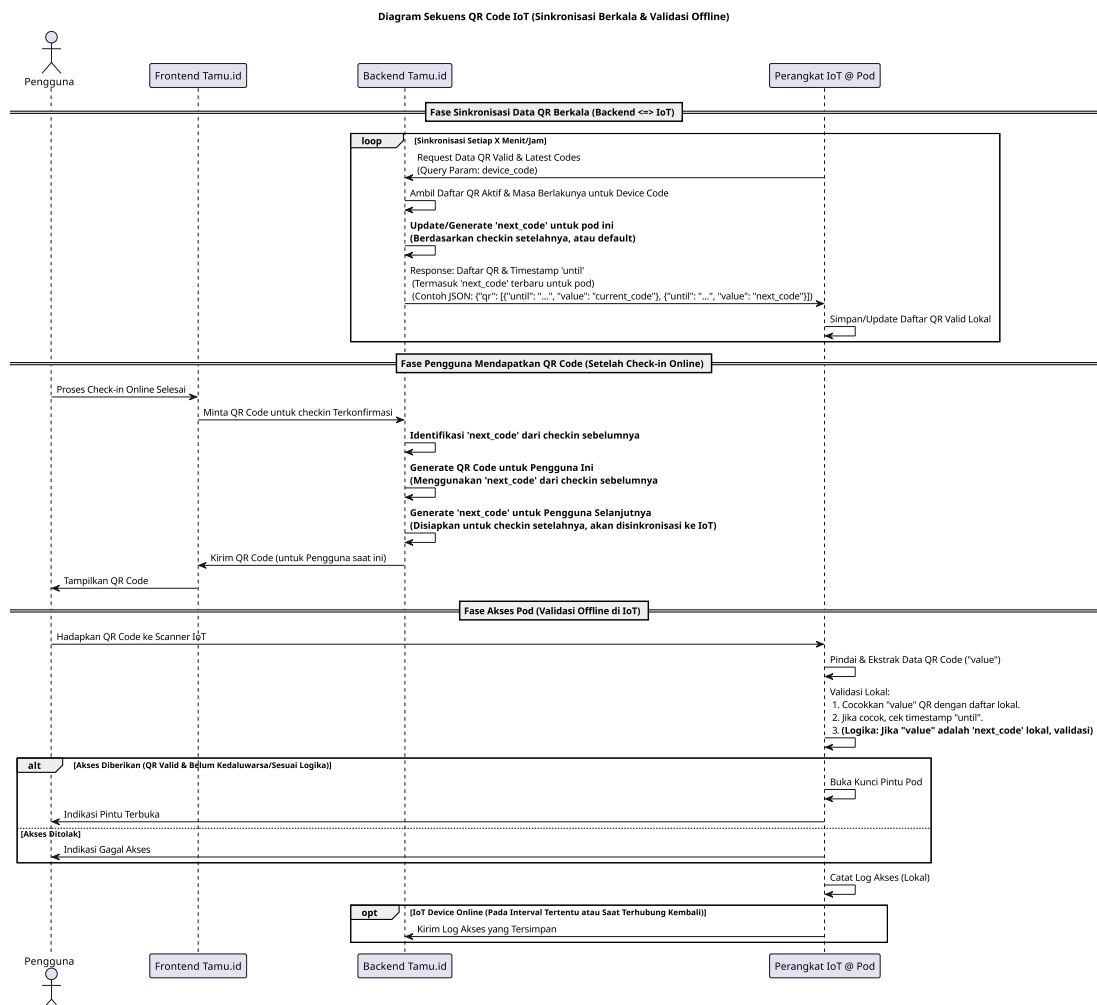
Gambar 4.46 Diagram implementasi Webhook pada Midtrans

Untuk mengatasi hal tersebut, implementasi *Webhook* menjadi solusi krusial. *Webhook* akan secara otomatis mengirimkan notifikasi *real-time* ke sistem backend segera setelah pembayaran berhasil. Ini memungkinkan sistem backend untuk langsung memperbarui status pembayaran tanpa intervensi manual, sehingga pengguna tidak perlu menunggu lama untuk mendapatkan konfirmasi. Pendekatan berbasis *Webhook* ini secara signifikan meningkatkan efisiensi dan otomatisasi keseluruhan proses pembayaran, baik dari sisi operasional backend maupun pengalaman pengguna. Diagram teknis sekuensial implementasi *Webhook* dapat dilihat pada Gambar 4.46.

4.1.7 Analisis Offline Fault Tolerance pada Hardware IoT

Ketika pengguna melakukan *Check-in*, sistem akan mengirimkan QR Code yang berfungsi sebagai kunci digital untuk membuka pintu Pod. Tantangan muncul ketika hardware IoT pada pintu Pod harus tetap berfungsi optimal bahkan saat koneksi internet terputus sesaat. Jika hardware IoT selalu bergantung pada koneksi online untuk memvalidasi setiap kunci baru,

ini akan menciptakan kerentanan terhadap *offline fault tolerance* dan dapat menyebabkan keterlambatan atau kegagalan akses. Oleh karena itu, analisis mendalam terhadap integrasi ini diperlukan untuk memastikan reliabilitas kunci digital dalam kondisi offline.



Gambar 4.47 Diagram implementasi *Offline Fault Tolerance* pada integrasi backend dengan hardware IoT

Untuk mencapai sifat *Offline Fault Tolerance* tersebut, penulis mengimplementasikan struktur data *Customized LinkedList* dengan *Unix Timestamp* langsung pada hardware IoT. Desain ini memungkinkan hardware untuk menyimpan dan memvalidasi daftar QR Code kunci yang valid secara lokal, lengkap dengan batas waktu berlakunya yang presisi. Dengan demikian, ketika pengguna melakukan *Check-in* dan menerima QR Code, mereka dapat segera menggunakannya untuk membuka pintu Pod secara *seamless* tanpa penundaan, bahkan jika Pod sedang dalam kondisi offline sesaat. Mekanisme teknis integrasi backend dengan hardware IoT, serta alur yang terjadi ketika pengguna melakukan *Check-in*, dijelaskan lebih lanjut dalam diagram sekuensial pada Gambar 4.47. Gambaran struktur data ini dapat dilihat pada Gambar 4.41.

4.2 Hasil Pengujian

Pengujian sistem dilakukan untuk mengetahui apakah sistem yang telah dibuat dapat berjalan sesuai dengan yang diharapkan. Pengujian ini dilakukan dengan cara melakukan uji coba terhadap setiap fitur yang ada pada sistem, baik dari sisi *User* maupun dari sisi admin.

4.2.1 Pengujian Blackbox

Pengujian Blackbox adalah pengujian yang dilakukan untuk mengetahui apakah sistem yang telah dibuat dapat berjalan sesuai dengan yang diharapkan. Pengujian ini dilakukan dengan cara melakukan uji coba terhadap setiap fitur yang ada pada sistem, baik dari sisi *User* pada tabel 4.1, *Admin* pada tabel 4.6, dan *SuperAdmin* pada tabel 4.3. Pengujian ini dilakukan tanpa melihat kode program yang ada pada sistem dan hanya berfokus pada input dan output dari interaksi antara pengguna dengan sistem.

Tabel 4.1 Hasil Pengujian Black Box (User)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Registrasi Akun			
Registrasi berhasil dengan email baru	1. Buka halaman registrasi. 2. Isi nama lengkap. 3. Masukkan email yang belum terdaftar (format valid). 4. Masukkan kata sandi kuat. 5. Konfirmasi kata sandi dengan benar. 6. (Jika ada) Centang persetujuan Syarat & Ketentuan. 7. Klik tombol "Daftar".	- Pesan sukses registrasi ditampilkan. - Pengguna diarahkan ke halaman login atau dashboard. - Email konfirmasi terkirim.	Sesuai
Registrasi gagal karena email sudah terdaftar	1. Buka halaman registrasi. 2. Isi nama lengkap. 3. Masukkan email yang sudah terdaftar. 4. Masukkan kata sandi. 5. Konfirmasi kata sandi. 6. Klik tombol "Daftar".	- Pesan error "Email sudah terdaftar" ditampilkan. - Pengguna tetap di halaman registrasi.	Sesuai

Registrasi gagal karena format email salah	1. Buka halaman registrasi. 2. Masukkan email dengan format salah (misal: user@.com, user.com). 3. Isi field lainnya. 4. Klik "Daftar".	- Pesan error "Format email tidak valid" ditampilkan.	Sesuai
Registrasi gagal karena konfirmasi kata sandi tidak cocok	1. Buka halaman registrasi. 2. Isi nama lengkap, email baru, kata sandi. 3. Masukkan konfirmasi kata sandi yang berbeda. 4. Klik tombol "Daftar".	- Pesan error "Konfirmasi kata sandi tidak cocok" ditampilkan.	Sesuai
Registrasi menggunakan akun Google (sukses, pengguna baru)	1. Buka halaman registrasi/login. 2. Klik tombol "Daftar/Masuk dengan Google". 3. Pilih akun Google yang valid dan belum terdaftar di sistem. 4. Izinkan akses jika diminta.	- Akun berhasil terbuat/terhubung. - Pengguna otomatis berhasil masuk.	Sesuai
Fitur Login Pengguna			
Login berhasil dengan email dan kata sandi valid	1. Buka halaman login. 2. Masukkan email terdaftar. 3. Masukkan kata sandi yang benar. 4. Klik tombol "Masuk".	- Login berhasil. - Pengguna berhasil masuk.	Sesuai
Login gagal (email tidak terdaftar)	1. Buka halaman login. 2. Masukkan email yang tidak terdaftar. 3. Masukkan kata sandi. 4. Klik tombol "Masuk".	- Pesan error "Email atau kata sandi salah" ditampilkan.	Sesuai

Login gagal (kata sandi salah)	1. Buka halaman login. 2. Masukkan email terdaftar. 3. Masukkan kata sandi yang salah. 4. Klik tombol "Masuk".	- Pesan error "Email atau kata sandi salah" ditampilkan.	Sesuai
Login gagal (akun belum diverifikasi email, jika ada)	1. Registrasi dengan email lalu jangan verifikasi email. 2. Coba login dengan email dan kata sandi tersebut.	- Pesan error "Akun belum diverifikasi. Silakan cek email Anda." atau sejenisnya.	Sesuai
Login menggunakan akun Google (sukses, akun sudah ada)	1. Buka halaman login. 2. Klik "Masuk dengan Google". 3. Pilih akun Google yang sudah terdaftar/terhubung sebelumnya.	- Login berhasil. - Pengguna diarahkan ke dashboard.	Sesuai
Fitur Lupa Kata Sandi			
Proses lupa kata sandi berhasil	1. Di halaman login, klik "Lupa Kata Sandi?". 2. Masukkan email terdaftar. 3. Klik "Kirim Instruksi". 4. Buka email, klik link reset. 5. Masukkan kata sandi baru dan konfirmasinya. 6. Klik "Reset Kata Sandi".	- Pesan sukses "Kata sandi berhasil direset". - Dapat login dengan kata sandi baru.	Sesuai
Lupa kata sandi dengan email tidak terdaftar	1. Di halaman login, klik "Lupa Kata Sandi?". 2. Masukkan email yang tidak terdaftar. 3. Klik "Kirim Instruksi".	- Pesan error "Email tidak ditemukan" atau instruksi tidak terkirim.	Sesuai
Fitur Kelola Profil Pengguna			

Mengubah semua data profil yang diizinkan (sukses)	1. Login. 2. Ke halaman "Profil Saya" lalu "Edit Profil". 3. Ubah nama, nomor telepon, alamat, gender, tanggal lahir. 4. Klik "Simpan".	- Pesan sukses "Profil berhasil diperbarui". - Semua perubahan tercermin dengan benar.	Sesuai
Mengubah data profil dengan format salah (misal: telepon)	1. Login. 2. Ke "Edit Profil". 3. Masukkan nomor telepon dengan format salah (misal: huruf). 4. Klik "Simpan".	- Pesan error "Format nomor telepon tidak valid". - Data tidak tersimpan.	Sesuai
Fitur Verifikasi Identitas (KYC)			
Mengajukan verifikasi KYC (KTP) dengan data valid	1. Login. 2. Ke halaman KYC. 3. Pilih jenis ID "KTP". 4. Masukkan nomor KTP valid. 5. Unggah foto KTP jelas. 6. Unggah foto selfie jelas. 7. Klik "Ajukan".	- Pengajuan KYC berhasil dikirim. - Status KYC menjadi "Menunggu Verifikasi".	Sesuai
Mengajukan KYC (Passport) dengan data valid	1. Login. 2. Ke halaman KYC. 3. Pilih jenis ID "Passport". 4. Masukkan nomor Passport valid. 5. Unggah foto Passport jelas. 6. Unggah foto selfie jelas. 7. Klik "Ajukan".	- Pengajuan KYC berhasil dikirim. - Status KYC menjadi "Menunggu Verifikasi".	Sesuai

Mengajukan KYC dengan file gambar terlalu besar/format salah	1. Login. 2. Ke halaman KYC. 3. Isi data. 4. Unggah file gambar dengan ukuran < batas maksimal atau format bukan JPG/PNG. 5. Klik "Ajukan".	- Pesan error "Ukuran file terlalu besar" atau "Format file tidak didukung".	Sesuai
Melihat status KYC (Pending, Approved, Rejected)	1. Login. 2. Navigasi ke halaman KYC atau Profil.	- Status KYC pengguna ditampilkan dengan benar sesuai kondisi aktual.	Sesuai
Fitur Pencarian dan Reservasi			
Mencari pod (tanggal check-out sebelum check-in)	1. Login. 2. Pilih Cabang. 3. Pilih tanggal check-in. 4. Pilih tanggal check-out lebih awal dari check-in. 5. Klik "Cari Pod".	- Pesan error "Tanggal check-out tidak valid".	Sesuai
Melakukan pemesanan pod dengan voucher kadaluarsa	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Di halaman pembayaran, masukkan kode voucher yang sudah kadaluarsa. 4. Klik "Terapkan Voucher".	- Pesan error "Voucher tidak valid atau sudah kadaluarsa".	Sesuai
Pemesanan pod dengan jumlah melebihi kapasitas tipe pod	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Coba pesan jumlah tamu yang melebihi kapasitas maksimal dewasa/anak tipe pod tersebut.	- Pesan error "Jumlah tamu melebihi kapasitas" atau input dibatasi.	Sesuai

Pemesanan pod, pembayaran berhasil (QRIS)	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Lanjutkan ke pembayaran. 4. Pilih metode QRIS, pindai QR. 5. Konfirmasi pembayaran di aplikasi e-wallet.	- Pembayaran berhasil. - Tampilan menunggu dibayar berubah menjadi terbayar - Transaksi tercatat.	Sesuai
Pemesanan pod, menggunakan kode referral teman (valid, aturan disesuaikan)	1. Lakukan pencarian pod. 2. Pilih tipe pod. 3. Di halaman pembayaran, masukkan kode referral valid milik teman. 4. Klik "Terapkan". 5. Selesaikan pembayaran.	- Diskon referral berhasil diterapkan. - Harga total berkurang. - Pemesanan berhasil.	Sesuai
Fitur Sinkronisasi OTA			
Sinkronisasi reservasi OTA dengan kode valid	1. Login. 2. Ke halaman "Sinkronisasi OTA" atau "Klaim Reservasi". 3. Pilih OTA. 4. Masukkan kode booking OTA / nomor order OTA yang valid (sudah diinput Admin Cabang). 5. Klik "Sinkronisasi".	- Reservasi OTA berhasil disinkronkan/diklaim. - Muncul di daftar booking pengguna.	Sesuai
Sinkronisasi reservasi OTA dengan kode tidak ditemukan/salah	1. Login. 2. Ke halaman "Sinkronisasi OTA". 3. Masukkan kode booking OTA yang salah. 4. Klik "Sinkronisasi".	- Pesan error "Reservasi tidak ditemukan" atau "Kode booking salah".	Sesuai
Fitur Operasional Menginap			

Check-in berhasil	1. Login. 2. Buka halaman stays lalu klik tab current 3. Coba klik tombol "Check-in" 4. Pilih pod yang diinginkan 5. klik checkin	- berhasil check-in dan mendapatkan kunci pod yang diinginkan berupa QRCode	Sesuai
Check-in gagal (di luar jam operasional atau sebelum waktunya check-in)	1. Login. 2. Buka halaman stays. 3. Coba klik tombol "Check-in" sebelum jam check-in yang diizinkan atau setelah batas waktu check-in.	- Tombol "Check-in" tidak ada untuk booking yang statusnya masih upcoming	Sesuai
Check-out	1. Login. 2. Buka detail booking aktif yang sudah check-in. 3. Klik tombol "Check-out" 4. muncul modal rating (user wajib mengisi rating untuk isi review opsional) 5. klik kirim	- Check-out berhasil. - Status booking "Completed". rating dan review juga berhasil dikirim	Sesuai
Fitur Referral			
Melihat riwayat pendapatan referral	1. Login. 2. Ke halaman Referral. 3. Pilih tab "Riwayat Pendapatan".	- Menampilkan daftar transaksi yang menghasilkan pendapatan referral dengan statusnya.	Sesuai
Mengajukan pencairan referral (KYC belum disetujui)	1. Login, KYC masih "Pending" atau "Rejected". 2. Ke halaman Referral. 3. Coba klik "Cairkan Dana".	- Tombol "Cairkan Dana" tidak aktif atau pesan error "Verifikasi KYC diperlukan untuk pencairan".	Sesuai
Fitur Ulasan			

Memberikan ulasan tanpa rating bintang	1. Login, setelah checkout. 2. Ke Riwayat Booking, pilih tab oldest booking. 3. Klik "Beri Ulasan". 4. Hanya tulis komentar, tidak memilih bintang. 5. Coba Kirim.	- tombol kirim review tidak dapat di klik	Sesuai
Mencoba memberi ulasan untuk booking yang belum selesai/dibatalkan	1. Login. 2. Cari booking yang statusnya "Pending Payment" atau "Cancelled". 3. Coba cari opsi "Beri Ulasan".	- Opsi "Beri Ulasan" tidak tersedia untuk booking tersebut.	Sesuai

Tabel 4.2 Hasil Pengujian Black Box (Admin)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Login dan Profil Admin Cabang			
Login Admin Cabang berhasil	1. Buka halaman login admin. 2. Masukkan email Admin Cabang valid. 3. Masukkan kata sandi benar. 4. Klik "Masuk".	- Login berhasil. - Diarahkan ke halaman pilih cabang yang ingin dikelola Cabang.	Sesuai
Login Admin Cabang gagal (kata sandi salah)	1. Buka halaman login admin. 2. Masukkan email Admin Cabang valid. 3. Masukkan kata sandi salah. 4. Klik "Masuk".	- Pesan error "Email atau kata sandi salah". - Tetap di halaman login.	Sesuai

Mengubah profil Admin Cabang (misal: nama)	1. Login sebagai Admin Cabang. 2. Navigasi ke "Profil Saya". 3. Klik "Edit Profil". 4. Ubah nama. 5. Klik "Simpan".	- Nama Admin Cabang berhasil diperbarui.	Sesuai
Fitur Pengelolaan Operasional Cabang			
Mengedit semua informasi detail cabang yang diizinkan	1. Login. 2. Ke "Pengaturan data Cabang". 3. Ubah nama, alamat, telepon, email, deskripsi, jam check-in/out, gambar utama. 4. Klik "Simpan".	- Semua perubahan berhasil disimpan dan ditampilkan dengan benar.	Sesuai
Menambah tipe pod baru dengan semua field valid	1. Login. 2. Ke "Menu jenis pod" 3. Klik "Tambah Baru". 4. Isi nama, deskripsi, harga, kapasitas (dewasa, anak), gambar, fasilitas terkait, status breakfast. 5. Klik "Simpan".	- Tipe pod baru berhasil ditambahkan dengan semua detailnya.	Sesuai
Menghapus tipe pod yang masih memiliki pod aktif	1. Login. 2. Ke "Manajemen Pod" - "Tipe Pod". 3. Pilih tipe pod yang memiliki pod terdaftar di bawahnya. 4. Klik "Hapus". 5. Konfirmasi.	- Pesan error "Tidak dapat menghapus tipe pod karena masih ada pod yang terhubung" atau sejenisnya. - Tipe pod tidak terhapus.	Sesuai

Menambah pod baru dengan device code IoT unik	1. Login. 2. Ke "Manajemen Pod" - "pilih jenis pod yang pod nya ingin ditambahkan". 3. Klik "Tambah Pod". 4. Isi nama/nomor pod, pilih tipe pod, masukkan device code IoT yang unik. 5. Klik "Simpan".	- Pod baru berhasil ditambahkan. - Status awal "Tersedia".	Sesuai
Menambah pod baru dengan device code IoT yang sudah ada	1. Login. 2. Ke "Manajemen Pod" - "Daftar Pod". 3. Klik "Tambah Pod". 4. Masukkan device code IoT yang sudah digunakan pod lain. 5. Klik "Simpan".	- Pesan error "Device code sudah digunakan". - Penambahan pod gagal.	Sesuai
Mengubah status pod menjadi "Perlu Dibersihkan" secara manual	1. Login. 2. Ke "Manajemen Pod" - "Daftar Pod". 3. Pilih pod yang "Tersedia", klik "Edit". 4. Ubah status ke "Perlu Dibersihkan". 5. Klik "Simpan".	- Status pod berhasil diubah.	Sesuai
Membuat voucher diskon nominal untuk cabang	1. Login. 2. Ke "Manajemen Voucher Cabang". 3. Klik "Tambah Voucher". 4. Isi kode, nama, deskripsi, tipe "NOMINAL", jumlah diskon, min transaksi, periode berlaku, maks penggunaan. 5. Klik "Simpan".	- Voucher cabang berhasil dibuat.	Sesuai

Membuat voucher diskon persentase dengan batas maks diskon	1. Login. 2. Ke "Manajemen Voucher Cabang". 3. Klik "Tambah Voucher". 4. Isi kode, nama, tipe "PERSENTASE", nilai persentase, diskon maksimal, min transaksi, periode, kuota. 5. Klik "Simpan".	- Voucher cabang berhasil dibuat.	Sesuai
Menetapkan harga dinamis untuk akhir pekan	1. Login. 2. Ke "Harga Dinamis". 3. Pilih tipe pod. 4. Pilih rentang tanggal mencakup beberapa akhir pekan. 5. Masukkan harga lebih tinggi. 6. Klik "Simpan".	- Harga dinamis untuk akhir pekan berhasil disimpan.	Sesuai
Menetapkan harga dinamis yang tumpang tindih dengan aturan lain	1. Login. 2. Ke "Harga Dinamis". 3. Buat aturan harga dinamis baru yang tanggalnya tumpang tindih dengan aturan yang sudah ada untuk tipe pod yang sama.	- Sistem memberikan peringatan bahwa aturan harga ada yang tumpang tindih dan aturan harga baru tidak berhasil disimman	Sesuai
Fitur Manajemen Reservasi dan Tamu Cabang			
Input manual transaksi OTA dengan semua field lengkap	1. Login. 2. Ke "Transaksi OTA". 3. Klik "Tambah". 4. Isi Jenis OTA, ID Order OTA, nama tamu jika ada, etanggal check-in/out, tipe pod, jumlah tamu, dan harga di OTA 5. Klik "Simpan".	- Transaksi OTA berhasil dicatat.	Sesuai

Input manual transaksi OTA dengan field wajib kosong	1. Login. 2. Ke "Transaksi OTA". 3. Klik "Tambah". 4. Kosongkan salah satu field wajib (misal: ID Order OTA). 5. Coba klik "Simpan".	- Tombol simpan tidak bisa diklik karena masih ada field wajib yang kosong	Sesuai
Memindahkan tamu ke pod lain karena ada masalah di pod asal	1. Login. 2. Cari booking tamu aktif. 3. Pilih "Pindah Pod", berikan alasan (misal: AC rusak). 4. Pilih pod lain yang kosong, tipe sama. 5. Konfirmasi.	- Tamu berhasil dipindahkan. - Pod asal mungkin statusnya berubah jadi "Maintenance".	Sesuai
Mencatat pembersihan pod dengan unggah foto bukti	1. Login. 2. Ke "Log Kebersihan" atau daftar pod. 3. Pilih pod "Perlu Dibersihkan". 4. Klik "Sudah Dibersihkan". 5. Unggah foto bukti kebersihan. 6. (Jika ada) Tambahkan catatan. 7. Simpan.	- Status pod "Tersedia". - Foto bukti dan catatan tersimpan di log.	Sesuai
Fitur Manajemen Ulasan Cabang			
Menyetujui ulasan pengguna yang konstruktif	1. Login. 2. Ke "Manajemen Ulasan". 3. Pilih ulasan "Pending" yang isinya baik. 4. Klik "Setujui/Tampilkan".	- Ulasan tampil di halaman publik.	Sesuai

Menolak ulasan pengguna yang mengandung spam/kata kasar	1. Login. 2. Ke "Manajemen Ulasan". 3. Pilih ulasan yang berisi spam. 4. Klik "Tolak/Sembunyikan".	- Ulasan tidak tampil dan statusnya diperbarui.	Sesuai
---	---	---	--------

Tabel 4.3 Hasil Pengujian Black Box (SuperAdmin)

Skenario Pengujian	Langkah Pengujian	Hasil yang diharapkan	Kesesuaian Hasil
Fitur Login & Dashboard Global SuperAdmin			
Login SuperAdmin berhasil	1. Buka halaman login admin. 2. Masukkan email SuperAdmin. 3. Masukkan kata sandi yang benar. 4. Klik "Masuk".	- Login berhasil. - Diarahkan ke dashboard SuperAdmin.	Sesuai
Melihat statistik global di dashboard SuperAdmin	1. Login sebagai SuperAdmin. 2. Amati informasi di dashboard (total cabang, total pengguna, total pendapatan global, dll.).	- Menampilkan data statistik agregat dari semua cabang yang akurat.	Sesuai
Fitur Manajemen Data Master Global			

Menambah data cabang (hotel) baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Cabang/Hotel". 3. Klik "Tambah Cabang Baru". 4. Isi semua field yang diperlukan (nama, alamat, kota, kontak, slug unik, dll.). 5. Klik "Simpan".	- Cabang baru berhasil ditambahkan ke sistem. - Cabang muncul di daftar cabang.	Sesuai
Mengedit data cabang (hotel) yang sudah ada	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Cabang/Hotel". 3. Pilih cabang, klik "Edit". 4. Ubah salah satu field (misal: email kontak). 5. Klik "Simpan".	- Data cabang berhasil diperbarui.	Sesuai
Menambah data kota baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Kota". 3. Klik "Tambah Kota". 4. Masukkan nama kota. 5. Klik "Simpan".	- Kota baru berhasil ditambahkan. - Kota dapat dipilih saat menambah/mengedit cabang.	Sesuai

Menambah akun Admin Cabang baru dan mengassign ke cabang	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Staff". 3. Klik "Tambah Staff". 4. Isi detail staff (nama, email, password), pilih peran "Admin Cabang". 5. Assign ke satu atau beberapa cabang. 6. Klik "Simpan".	- Akun Admin Cabang baru berhasil dibuat. - Admin Cabang tersebut dapat login dan mengelola cabang yang di-assign.	Sesuai
Menghapus akun staff (Admin Cabang)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Staff". 3. Cari akun Admin Cabang. 4. Klik opsi "Hapus".	- Akun Admin Cabang berhasil dihapus. - Admin Cabang tersebut tidak dapat login.	Sesuai
Menambah fasilitas global baru	1. Login sebagai SuperAdmin. 2. Navigasi ke "Manajemen Fasilitas". 3. Klik "Tambah Fasilitas". 4. Masukkan nama fasilitas dan unggah ikon. 5. Klik "Simpan".	- Fasilitas baru berhasil ditambahkan. - Fasilitas dapat di-assign ke hotel atau tipe pod.	Sesuai
Membuat voucher global untuk semua cabang	1. Login. 2. Ke "Manajemen Voucher". 3. Klik "Tambah Voucher". 4. Isi detail (kode, nama, tipe, nilai, periode, kuota). 5. Pilih opsi "Berlaku di semua cabang". 6. Klik "Simpan".	- Voucher global berhasil dibuat. - Dapat digunakan pengguna di semua cabang.	Sesuai
Fitur Verifikasi & Approval Sistem			

Menyetujui permintaan verifikasi KYC pengguna yang valid	1. Login sebagai SuperAdmin. 2. Navigasi ke "Verifikasi KYC". 3. Pilih permintaan KYC yang berstatus "Pending" dengan data dan foto valid. 4. Klik "Setujui".	- Status KYC pengguna berubah menjadi "Approved". - Pengguna mendapatkan pembaharuan. - Pengguna dapat mengakses fitur yang memerlukan KYC.	Sesuai
Menolak permintaan verifikasi KYC pengguna (misal: foto buram)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Verifikasi KYC". 3. Pilih permintaan KYC yang fotonya buram. 4. Klik "Tolak". 5. Masukkan alasan penolakan (misal: "Foto tidak jelas").	- Status KYC pengguna berubah menjadi "Rejected". - Pengguna mendapatkan pembaharuan beserta alasan penolakan.	Sesuai
Menyetujui permintaan pencairan saldo referral yang valid	1. Login sebagai SuperAdmin. 2. Navigasi ke "Permintaan Payout Referral". 3. Pilih permintaan yang valid (saldo pengguna cukup, data bank benar). 4. Klik "Setujui". 5. Proses transfer dana dan unggah buktinya.	- Status permintaan payout berubah menjadi "Approved". - Pengguna mendapatkan pembaharuan.	Sesuai

Menolak permintaan pencairan saldo referral (saldo kurang)	1. Login sebagai SuperAdmin. 2. Navigasi ke "Permintaan Payout Referral". 3. Pilih permintaan dimana saldo aktual pengguna kurang dari yang diajukan. 4. Klik "Tolak". 5. Masukkan alasan.	- Status permintaan payout berubah menjadi "Rejected". - Pengguna mendapatkan pembaharuan.	Sesuai
--	--	---	--------

4.2.2 User Acceptance Test

User Acceptance Test (UAT) dilakukan untuk mengetahui apakah sistem yang telah dibuat dapat diterima oleh *User*. UAT dilakukan dengan cara meminta *User* untuk mencoba sistem dan memberikan feedback terhadap sistem yang telah dibuat, berikut pada tabel 4.4 adalah daftar responden yang mengikuti UAT.

Tabel 4.4 Daftar Responden *User Acceptance Test* (UAT)

No	Nama	Peran (role)	Profesi	Umur
1	Reisika Jenny	<i>SuperAdmin</i> dan <i>Admin Cabang</i>	Business Development	22
2	Asraf Rafli Daifuloh	<i>User</i>	Fullstack Programmer	24
3	Damianus Cahyo Widi Nugroho	<i>User</i>	AI Engineer	22
4	Evi Nur Hidayah	<i>User</i>	Mahasiwa	21
5	Moh. Eka Syafrino Nazhifan	<i>User</i>	Fullstack Programmer	19
6	Salsa Ainnur Muharramah	<i>User</i>	Mahasiswa Profesi	22
7	Daffa Haj Tsaqif	<i>User</i>	IoT Engineer	25
8	Markus Rabin Ronaldo	<i>User</i>	Mahasiswa	23
9	Bela Dwi Primasari	<i>User</i>	Sistem Analis	23
10	Nazil Makarim Ramadhany	<i>User</i>	Graphic Designer	21

11	Bagus erwanto	User	IT Support	23
----	---------------	------	------------	----

Untuk mendapatkan hasil yang lebih terukur dan seimbang setelah para responden mencoba sistem, penulis memberikan kuesioner yang berisi 10 pertanyaan yang harus dijawab oleh para responden. Kuesioner ini bertujuan untuk mengetahui sejauh mana sistem yang telah dibuat dapat memenuhi kebutuhan *User*.

Tabel 4.5 Hasil Kuesioner *User Acceptance Test* - Perspektif *User*

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses pendaftaran akun dan login ke aplikasi Tamu.id mudah dipahami dan dilakukan?				1	9
2.	Apakah tampilan antarmuka aplikasi Tamu.id menarik dan mudah digunakan secara umum?			1	3	6
3.	Apakah informasi mengenai cabang dan tipe pod (harga, fasilitas, ketersediaan) disajikan dengan jelas?				3	7
4.	Apakah alur proses pencarian dan pemesanan pod mudah diikuti dan efisien?				3	7
5.	Apakah proses pembayaran saat melakukan pemesanan mudah dan terasa aman?			1	1	8
6.	Apakah proses check-in dan check-out melalui aplikasi berjalan dengan lancar?					10
7.	Apakah penggunaan QR Code untuk akses pod (jika ada fitur ini) mudah dan praktis?				1	9
8.	Apakah saya mudah menemukan riwayat pemesanan dan detail transaksi saya?			1	3	6
9.	Apakah fitur untuk memberikan ulasan dan rating setelah menginap mudah digunakan?				5	5
10.	Secara keseluruhan, apakah Anda puas dengan fungsionalitas dan kemudahan penggunaan aplikasi Tamu.id?				1	9
Total Bobot				3	21	76
Skor				9	84	380
Total Skor		473				

Dari tabel 4.5 di atas, kita dapat menghitung skor *User Acceptance Test* (UAT) untuk perspektif pengguna sebagai berikut:

$$\begin{aligned}
 \text{Skor UAT User}(\%) &= \left(\frac{A}{B \times N} \right) \times 100\% \\
 &= \left(\frac{473}{50 \times 10} \right) \times 100\% \\
 &= \left(\frac{473}{500} \right) \times 100\% \\
 &= 0.946 \times 100\% \\
 &= 94.6\%
 \end{aligned}$$

Kesimpulan dari hasil UAT perspektif pengguna menunjukkan bahwa aplikasi Tamu.id mendapatkan skor 94.6%, yang berarti aplikasi ini sangat diterima oleh pengguna dan masuk dalam kategori Sangat Layak berdasarkan tabel 2.8.

Tabel 4.6 Hasil Kuesioner *User Acceptance Test* - Perspektif Admin Cabang

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses login ke dashboard Admin Cabang mudah dan aman?					1
2.	Apakah tampilan dashboard Admin Cabang informatif untuk memantau operasional harian cabang?					1
3.	Apakah pengelolaan informasi umum cabang (seperti detail kontak, foto, atau deskripsi) mudah dilakukan?					1
4.	Apakah fitur untuk menambah dan mengelola tipe pod serta unit pod individual di cabang saya mudah digunakan?					1
5.	Apakah proses untuk mengatur harga pod (termasuk harga dinamis jika digunakan) cukup jelas dan mudah diterapkan?					1
6.	Apakah pengelolaan data reservasi tamu (melihat daftar booking, detail, dan status) efisien?					1

7.	Apakah fitur untuk menginput atau mengelola data reservasi yang berasal dari Online Travel Agent (OTA) mudah dioperasikan?					1
8.	Apakah proses untuk mengelola status kebersihan dan ketersediaan pod (misalnya, dari "Perlu Dibersihkan" menjadi "Tersedia") sederhana?					1
9.	Apakah fitur untuk melihat dan menanggapi (memoderasi) ulasan dari pengguna untuk cabang saya mudah digunakan?					1
10.	Secara keseluruhan, apakah dashboard Admin Cabang ini efektif membantu saya dalam mengelola operasional cabang sehari-hari?					1
Total Bobot						10
Skor						50
Total Skor		50				

Dari tabel 4.6 di atas, kita dapat menghitung skor *User Acceptance Test* (UAT) untuk perspektif Admin Cabang dengan menggunakan rumus 2.1 sebagai berikut:

$$\begin{aligned}
 \text{Skor UAT Admin}(\%) &= \left(\frac{A}{B \times N} \right) \times 100\% \\
 &= \left(\frac{50}{50 \times 1} \right) \times 100\% \\
 &= \left(\frac{50}{50} \right) \times 100\% \\
 &= 1 \times 100\% \\
 &= 100\%
 \end{aligned}$$

Kesimpulan dari hasil UAT perspektif Admin Cabang menunjukkan bahwa aplikasi Tamu.id mendapatkan skor 100%, yang berarti aplikasi ini sangat diterima oleh Admin Cabang dan masuk dalam kategori Sangat Layak berdasar tabel 2.8.

Tabel 4.7 Hasil Kuesioner *User Acceptance Test* - Perspektif *SuperAdmin*

No.	Pertanyaan	Jawaban				
		STS	TS	N	S	SS
1.	Apakah proses login ke dashboard SuperAdmin mudah dan aman?					1
2.	Apakah tampilan dashboard global SuperAdmin menyajikan informasi ringkasan sistem secara efektif dan mudah dipahami?					1
3.	Apakah pengelolaan data master sistem (seperti daftar Cabang/Hotel, Kota, OTA global) mudah dilakukan?					1
4.	Apakah fitur untuk menambah, mengedit, dan mengelola akun staf (Admin Cabang, SuperAdmin lain) serta hak aksesnya intuitif?					1
5.	Apakah proses verifikasi identitas pengguna (KYC) dari sisi SuperAdmin berjalan dengan alur yang jelas dan efisien?				1	
6.	Apakah proses approval atau penolakan permintaan pencairan saldo referral pengguna mudah dikelola dan dilacak?				1	
7.	Apakah fitur untuk membuat dan mengelola voucher yang berlaku secara global atau untuk banyak cabang mudah digunakan?					1
8.	Apakah sistem SuperAdmin menyediakan alat yang memadai untuk memantau aktivitas penting dan kesehatan sistem secara keseluruhan?					1
9.	Apakah laporan global atau data analitik yang dihasilkan sistem (jika ada) membantu dalam analisis dan pengambilan keputusan strategis?					1
10.	Secara keseluruhan, apakah antarmuka SuperAdmin efektif dalam membantu saya menjalankan tugas administrasi sistem dan pengawasan global?					1
Total Bobot					2	8

Skor				8	40
Total Skor	48				

Dari tabel 4.7 di atas, kita dapat menghitung skor *User Acceptance Test* (UAT) untuk perspektif SuperAdmin sebagai berikut:

$$\begin{aligned}
 \text{Skor UAT SuperAdmin}(\%) &= \left(\frac{A}{B \times N} \right) \times 100\% \\
 &= \left(\frac{48}{50 \times 1} \right) \times 100\% \\
 &= \left(\frac{48}{50} \right) \times 100\% \\
 &= 0.96 \times 100\% \\
 &= 96\%
 \end{aligned}$$

Kesimpulan dari hasil UAT perspektif SuperAdmin menunjukkan bahwa aplikasi Tamu.id mendapatkan skor 96%, yang berarti aplikasi ini sangat diterima oleh SuperAdmin dan masuk dalam kategori Sangat Layak berdasar tabel 2.8.

BAB V

PENUTUP

5.1 Kesimpulan

Dari rumusan masalah yang sudah diidentifikasi sebelumnya dan berdasarkan hasil penelitian yang telah dilakukan, maka dapat ditarik beberapa kesimpulan sebagai berikut:

1. Aplikasi Manajemen Hotel Kapsul multi-cabang berbasis website dengan *Frontend* Vue.js dan *Backend* Flask telah berhasil dikembangkan dan diimplementasikan dengan berbagai fitur seperti integrasi Oauth2 Google dalam sistem otentikasi, implementasi *Customized Linkedlist* dengan *Unix Timestamp* dalam rotasi kunci Pod yang berbasis QRCode untuk menangani delay dalam integrasi *Backend* dan *IoT* saat pengguna baru check-in sekaligus fitur *offline fault tolerance*, serta sistem manajemen hotel kapsul berhasil diimplementasikan dengan baik.
2. Gerbang Pembayaran Midtrans dengan metode QRIS yang diimplementasikan dengan Core API dipilih sebagai sistem pembayaran karena kemudahan integrasi dan keamanan yang ditawarkannya, serta dukungan untuk berbagai metode pembayaran digital di Indonesia.
3. Sistem yang dikembangkan telah diuji dan diterima oleh pengguna, dengan umpan balik positif mengenai kemudahan penggunaan, keamanan, dan kecepatan sistem dalam menangani transaksi dan manajemen hotel kapsul dengan tingkat penerimaan dari sisi *User* sebesar 94.6%, dari sisi *Admin* sebesar 100%, dan dari sisi *SuperAdmin* sebesar 96% dibuktikan dengan *User Acceptance Testing* yang dilakukan untuk setiap *Role* pengguna.

5.2 Saran

Berdasarkan hasil dari penelitian dan pengembangan yang sudah dilakukan maka berikut beberapa saran untuk pengembangan proyek ini kedepannya:

1. Menambahkan integrasi sistem manajemen dan pengelolaan yang lebih *Seamless* dan otomatis dengan berbagai *Online Travel Agency* (OTA) baik itu melalui Rest API dan juga webhook.
2. Menambahkan jenis pembayaran yang berlaku secara lebih luas dipasar internasional untuk saat ini seperti kartu kredit ataupun kartu debit dengan jaringan *VISA*, *Mastercard*, dan lain - lain.
3. Mengintegrasikan sistem dengan *Whatsapp* ataupun sistem *Messaging* yang lebih umum untuk kemudahan pengguna.